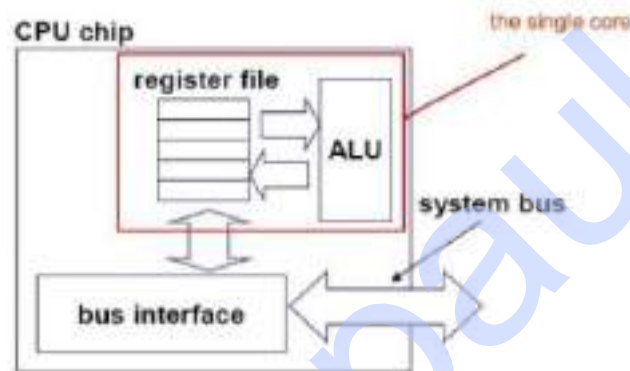


UNIT - I

*Single core to Multi-core architectures – SIMD and MIMD systems – Interconnection networks
Symmetric and Distributed Shared Memory Architectures – Cache coherence - Performance Issues –
Parallel program design.*

Single-Core Processors

Single core processors have only one processor in die to process instructions. All the processor developed by different manufacturers till 2005 were single core. In today's computers we use multicore processors but single core processor also perform very well. Single core processors have been discontinued in new computers, so these are available at very cheap rates.

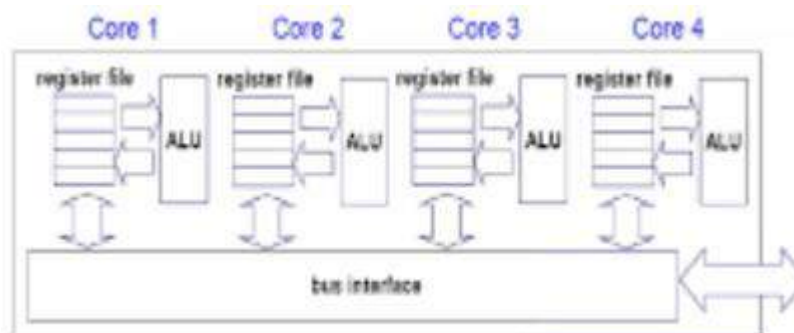


Problems of Single Core Processors:

- Due to increase in the clock speed of this processor, the amount of heat produced by the chip also increases.
- It is a big hindrance in the way of single core processors to continue evolving.

MULTI-CORE PROCESSORS

- Multicore processor are the latest processors which became available in the market after 2005.
- These processors use two or more cores to process instructions at the same time by using hyper threading.
- The multiple cores are embedded in the same die.
- The multicore processor may looks like a single processor but actually it contains two (dual-core), three (tricore), four (quad-core), six (hexa-core), eight (octa-core) or ten (deca-core) cores.
- Some processor even have 22 or 32 cores.
- Due to power and temperature constraint, the multicore processors are only practical solution for increasing the speed of future computers.



Problems with Multicore processors:

- According to Amdahl's law, the performance of parallel computing is limited by its serial components.
- So, increasing the number of cores may not be the best solution. There is need to increase the clock speed of individual cores.

Comparison of Single Core and Multi Cores :

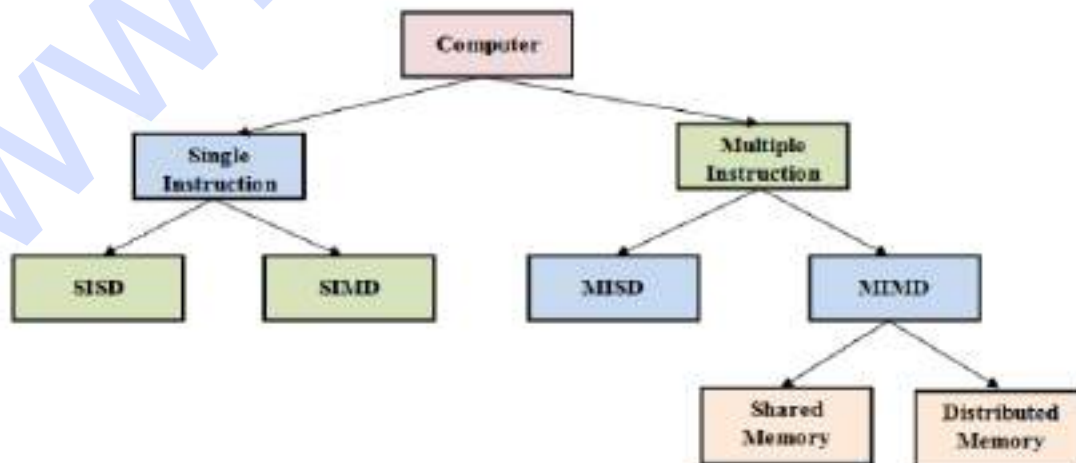
Parameter	Single Core	Multi Core
Number of cores on a die	Single	Multiple
Instruction Execution	Can execute Single instruction at a time Can execute multiple instructions by using	Can execute multiple instructions by using multiple cores
Gain	Speed up every program or software being executed	Speed up the programs which are designed for multi-core processors
Performance	Dependent on the clock frequency of the core	Dependent on the frequency, number of cores and program to be executed
Examples	Processor launched before 2005 like 80386, 486, AMD 29000, AMD K6, Pentium I, II, III etc.	Processor launched after 2005 like Core-2Duo, Athlon 64 X2, I3, I5 And I7 etc.

SIMD and MIMD Systems:

Computer architecture can be classified into four main categories. These categories are defined under the Flynn's Taxonomy. Computer architecture is classified by the number of instructions that are running in parallel and how its data is managed. The four categories that computer architecture can be classified under are:

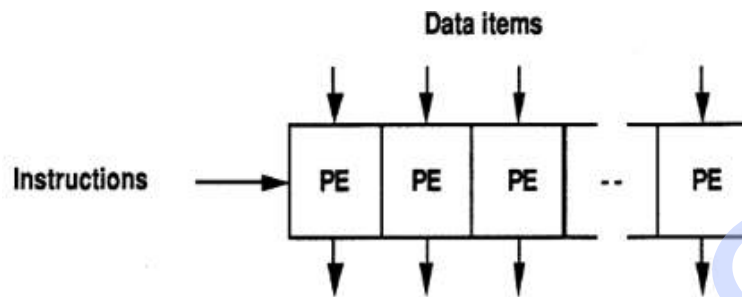
1. SISD: Single Instruction, Single Data
2. SIMD: Single Instruction, Multiple Data
3. MISD: Multiple Instruction, Single Data
4. MIMD: Multiple Instruction, Multiple Data

The following diagram explains about Flynn's technology



SIMD Architecture

Single Instruction stream, Multiple Data stream (SIMD) processors one instruction works on several data items simultaneously by using several processing elements, all of which carried out same operation.

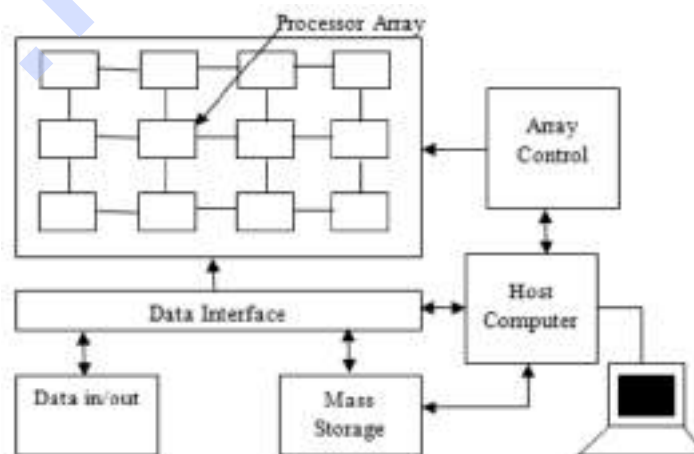


SIMD system comprise one of the three most commercially successful classes of parallel computers (the other being vector supercomputer and MIMD systems). A number of factors have contributed to this success including :

- Simplicity of concept and programming
- Regularity of structure
- Easy scalability of size and performance
- Straightforward applicability in a number of fields which demands parallelism to achieve necessary performance.

Basic Principles:

- There is a two-dimensional array of processing elements, each connected to its four nearest neighbors.
- All processors execute the same instruction simultaneously.
- Each processor incorporates local memory.
- The processors are programmable, that is, they can perform a variety of functions.
- Data can propagate quickly through the array.

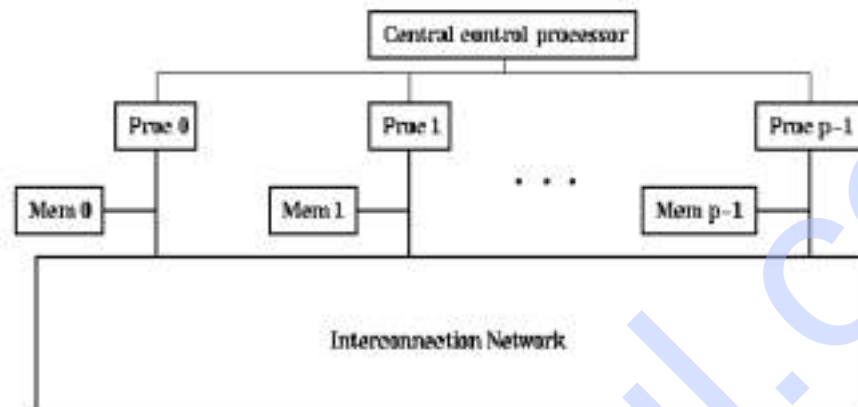


Implementing SIMD Architecture:

Two types of SIMD architectures exist:

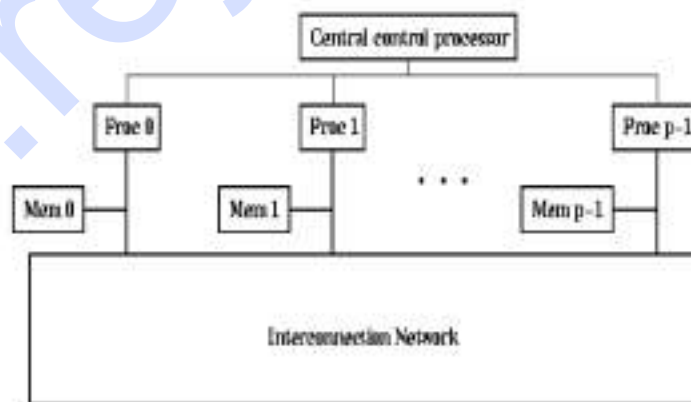
1. True SIMD
2. Pipelined SIMD

True SIMD architecture: True SIMD architectures can be determined by its usage of distributed memory and shared memory. Both true SIMD architectures possess similar implementation as seen on Fig.4, but differ on placement of processor and memory modules.



True SIMD architecture with distributed memory:

- A true SIMD architecture with distributed memory possesses a control unit that interacts with every processing element on the architecture.
- Each processor possesses their own local memory as observe on Fig.5.
- The processor elements are used as an arithmetic unit where the instructions are provided by the controlling unit.
- In order for one processing element to communicate with another memory on the same architecture, such as for information fetching, it will have to acquire it through the controlling unit.
- This controlling unit handles the transferring of the information from one processing element to another.
- The main drawback is with the performance time where the controlling unit has to handle the data transfer.

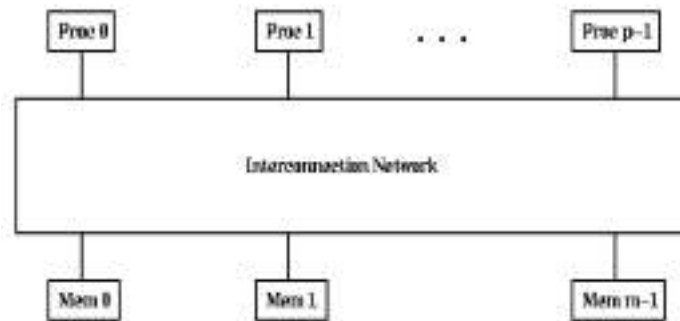


True SIMD architecture with Shared Memory:

- In this architecture, a processing element does not have a local memory but instead it's connected to a network where it can communicate with a memory component.
- In this architecture, the controlling unit is ignored when it comes to processing elements sharing information.
- However, provides an instruction to the processing elements for computational reasons.
- The disadvantage in this architecture is that if there is a need to expand this architecture, each

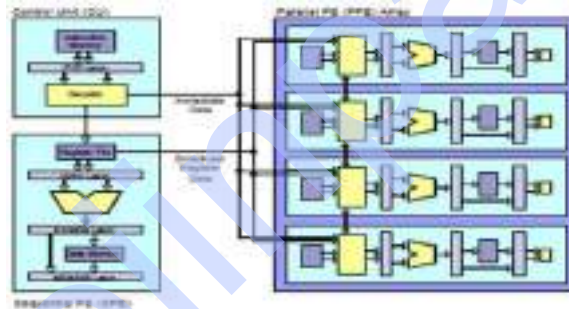
module (processing elements and memory) has to be added separately and configured.

- This architecture is still beneficial since it improved performance time and the information can be transferred more freely without the controlling unit.



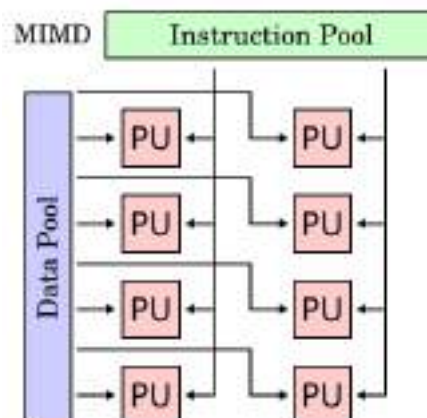
Pipelined SIMD Architecture:

- It implements the logic behind pipelining an instruction. Each processing element will receive an instruction from the controlling unit, using a shared memory, and will perform computation at multiple stages.
- The controlling unit provides the parallel processing elements with instructions.
- The sequential processing element is used to handle other instructions.



MIMD Architecture

MIMD stands for Multiple Instruction, Multiple Data. The MIMD class of parallel architecture is the most familiar and possibly most basic form of parallel processor. MIMD architecture consists of a collection of N independent, tightly coupled processors, each with memory that may be common to all processors, and /or local and not directly accessible by the other processors.

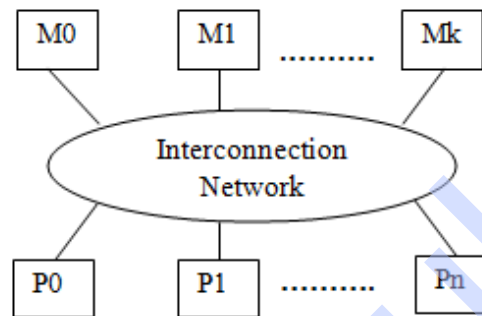


Two types of MIMD architecture:-

1. Shared Memory MIMD architecture
2. Distributed Memory MIMD architecture

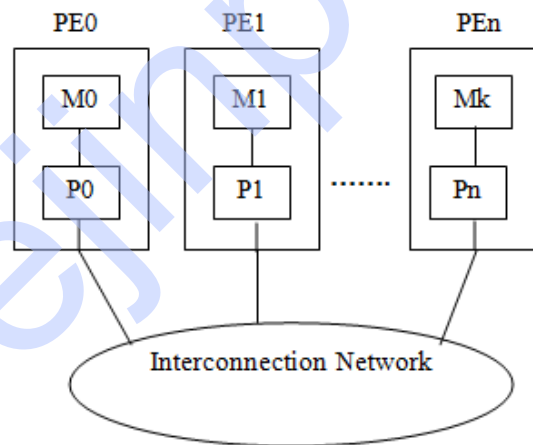
Shared Memory MIMD architecture:

- Create a set of processors and memory modules. Any processor can directly access any memory module via an interconnection network
- The set of memory modules defines a global address space which is shared among the processors.



Distributed Memory MIMD architecture:

- It replicates the processor/memory pairs and connects them via an interconnection network.
- The processor/memory pair is called processing element (PE).
- Each processing element (PE) can interact with each other via sending messages.



Comparison of SIMD and MIMD :

Features	SIMD	MIMD
Abbreviation	Single Instruction Multiple Data	Multiple Instruction Multiple Data
Ease of programming and Debugging	Single program, Processing element(PE) operate synchronously	Multiple communication programs, Processing element(PE) operate asynchronously
Lower program memory requirements	One copy of the program is stored	Each PE stores its own program
Lower instruction cost	One decoder in control unit	One decoder in each PE
Complexity of architectures	Simple	Complex
Cost	Low	Medium
Size and Performance	Scalability in size and performance	Complex size and good performance
Conditional Statements	Conditional statements depends	The multiple instruction stream o

	upon data local to processors, all of instructions of then block must broadcast, followed by all else block	MIMD allow for more efficient execution of conditional statement (e.g.-: if then else) because each processor can independently follow either decision path
Low synchronization overheads	Implicit in program	Explicit data structures and operations needed
Low PE-to-PE communication overheads	Automatic synchronization of all “send” and “receive” operations	Explicit synchronization and identification protocols needed.
Efficient execution of variable-time instructions	Total execution time equals the sum of maximal execution times through all processors	Total execution time equals the maximum execution time on a given processor

Using Flynn’s Taxonomy-: SIMD and MIMD. SIMD allow for more faster and multiple computation in this field where sacrifice cannot be made on the delay of time. SIMD processing architecture example-: a graphic processor processing instructions for translation or rotation or other operations are done on multiple data. MIMD processing architecture example is super computer or distributed computing systems with distributed or single shared memory

Interconnection networks:

- Networking strategy was originally employed in the 1950's by the telephone industry as a means of reducing the time required for a call to go through.
- Similarly, the computer industry employs networking strategy to provide fast communication between computer subparts, particularly with regard to parallel machines.
- Any parallel system that employs more than one processor per application program must be designed to allow its processors to communicate efficiently; otherwise, the advantages of parallel processing may be negated by inefficient communication.
- This fact emphasizes the importance of interconnection networks to overall parallel system performance.
- In many proposed or existing parallel processing architectures, an interconnection network is used to realize transportation of data between processors or between processors and memory modules.

Network Topology :

- Network topology refers to the layouts of links and switch boxes that establish interconnections.
- The links are essentially physical wires (or channels); the switch boxes are devices that connect a set of input links to a set of output links. There are two groups of network topologies: static and dynamic. Static networks provide fixed connections between nodes.
- With a static network, links between nodes are unchangeable and cannot be easily reconfigured. Dynamic networks provide reconfigurable connections between nodes.
- The switch box is the basic component of the dynamic network.
- With a dynamic network the connections between nodes are established by the setting of a set of interconnected switch boxes.

Static networks:

There are various types of static networks, all of which are characterized by their node degree; node degree is the number of links (edges) connected to the node. Some well-known static networks are the

following:

Degree 1: shared bus

Degree 2: linear array, ring

Degree 3: binary tree, fat tree, shuffle-exchange

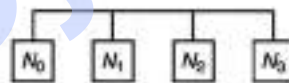
Degree 4: two-dimensional mesh (Illiac, torus)

Varying degree: n-cube, n-dimensional mesh, k-ary n-cube

A measurement unit, called diameter, can be used to compare the relative performance characteristics of different networks. More specifically, the diameter of a network is defined as the largest minimum distance between any pair of nodes. The minimum distance between a pair of nodes is the minimum number of communication links (hops) that data from one of the nodes must traverse in order to reach the other node.

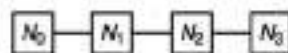
Shared bus :

- The shared bus, also called common bus, is the simplest type of static network.
- The shared bus has a degree of 1. In a shared bus architecture, all the nodes share a common communication link.
- The shared bus is the least expensive network to implement.
- Also, nodes (units) can be easily added or deleted from this network.
- However, it requires a mechanism for handling conflict when several nodes request the bus simultaneously. This mechanism can be achieved through a bus controller, which gives access to the bus either on a first-come, first-served basis or through a priority scheme.
- The shared bus has a diameter of 1 since each node can access the other nodes through the shared bus.



Linear array :

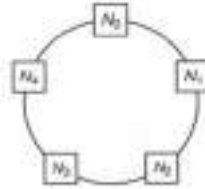
The linear array (degree of 2) has each node connected with two neighbors (except the far ends nodes). The linear quality of this structure comes from the fact that the first and last nodes are not connected. Although the linear array has a simple structure, its design can mean long communication delays, especially between far-end nodes. This is because any data entering the network from one end must pass through a number of nodes in order to reach the other end of the network. A linear array, with N nodes, has a diameter of $N-1$.



Ring :

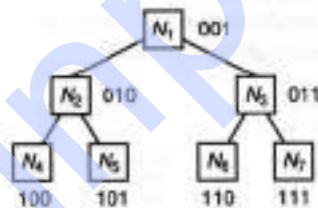
- Another networking configuration with a simple design is the ring structure.
- A ring network has a degree of 2.
- Similar to the linear array, each node is connected to two of its neighbors, but in this case the first and last nodes are also connected to form a ring.
- A ring can be unidirectional or bidirectional.

- In a unidirectional ring the data can travel in only one direction, clockwise or counterclockwise. Such a ring has a diameter of $N-1$, like the linear array.
- Bidirectional ring, in which data travel in both directions, reduces the diameter by a factor of 2, or less if N is even. A bidirectional ring with N nodes has a diameter of $2/N$. Although this ring's diameter is much better than that of the linear array, its configuration can still cause long communication delays between distant nodes for large N .



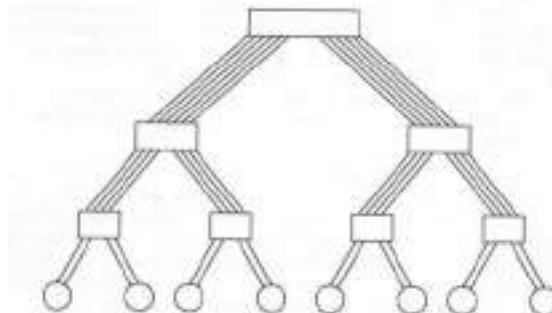
Binary tree :

- The top node is called the root, the four nodes at the bottom are called leaf (or terminal) nodes, and the rest of the nodes are called intermediate nodes.
- In such a network, each intermediate node has two children. The root has node address 1. The addresses of the children of a node are obtained by appending 0 and 1 to the node's address that is, the children of node x are labeled $2x$ and $2x+1$.
- A binary tree with N nodes has diameter $2(h-1)$, where $h = \log_2 N$ is the height of the tree. The binary tree has the advantages of being expandable and having a simple implementation.



Fat tree:

- The structure of the fat tree is based on a binary tree. Each edge of the binary tree corresponds to two channels of the fat tree.
- One of the channels is from parent to child, and the other is from child to parent.
- The number of communication links in each channel increases as we go up the tree from the leaves and is determined by the amount of hardware available.
- The fat tree can be used to interconnect the processors of a general-purpose parallel machine.
- Since its communication bandwidth can be scaled independently from the number of processors, it provides great flexibility in design.



Shuffle-exchange :

- Another method for establishing networks is the shuffle-exchange connection. A shuffle-exchange network is a combination of two functions: shuffle and exchange.
- Each is a simbijection function in which each input is mapped onto one and only one output.
- Let $s_{n-1}, s_{n-2} \dots s_0$ binary representation of a node address; then the shuffle function can be described as $\text{shuffle}(s_{n-1} s_{n-2} \dots s_0) = s_{n-2} s_{n-3} \dots s_0 s_{n-1}$.

For example, using the shuffle function for $N=8$ (i.e. 2 nodes) the following connections can be established between the nodes.

Source	→	Destination	Source	→	Destination
000	→	000	100	→	001
001	→	010	101	→	011
010	→	100	110	→	101
011	→	110	111	→	111

The reason that the function is called shuffle is that it reflects the process of shuffling cards. Given that there are eight cards, the shuffle function performs a perfect playing card shuffle as follows. First, the deck is cut in half, between cards 3 and 4. Then the two half decks are merged by selecting cards from each half in an alternative order.

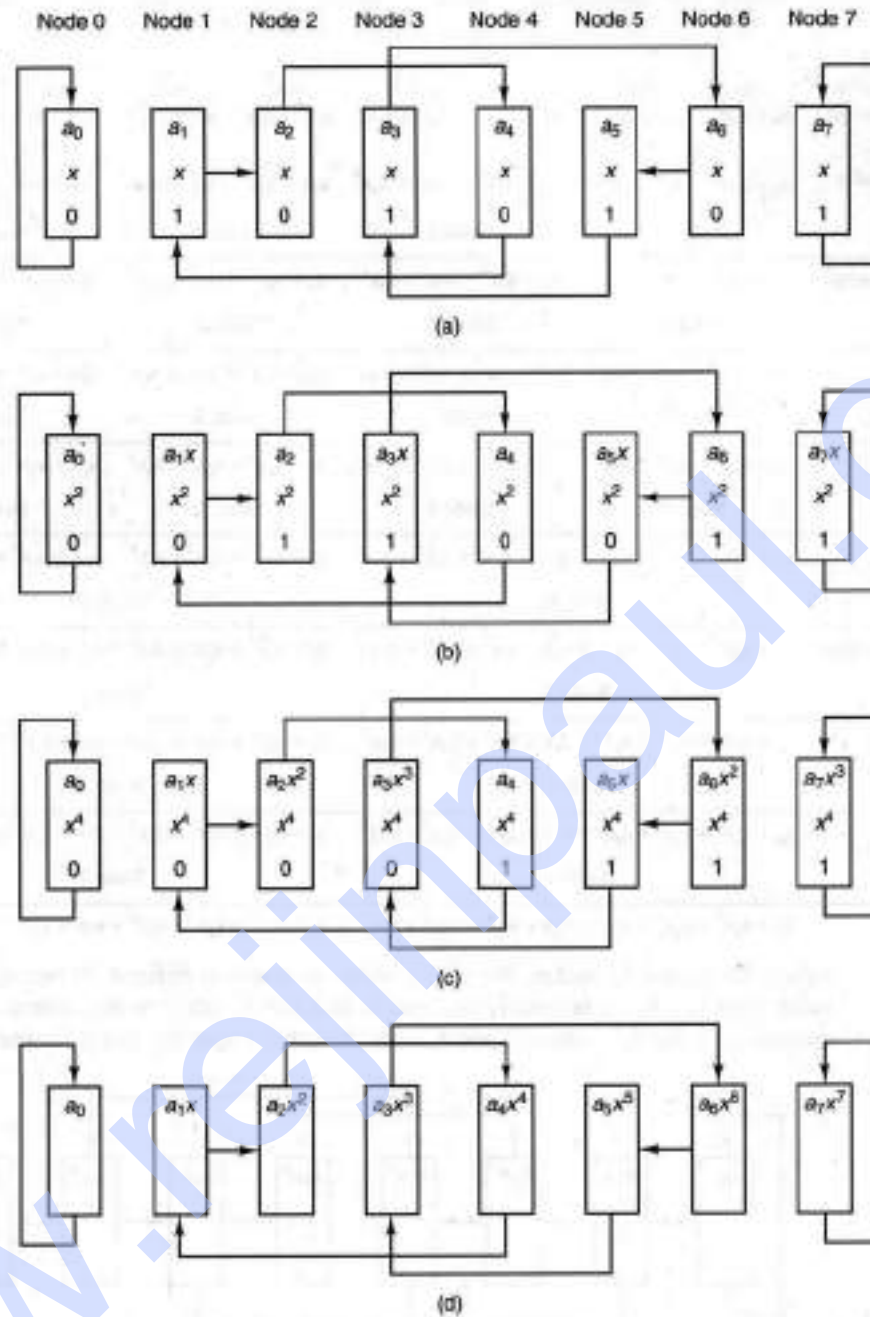
In general, a polynomial of degree N can be represented as

$$a_0 + a_1x + a_2x^2 + \dots + a_{N-1}x^{N-1} + a_Nx^N,$$

where $a_0, a_1, \dots, a_N$ are the coefficients and x is a variable. One way to evaluate such a polynomial is to use the architecture shown in figure. In this figure, assume that each node represents a processor having three registers: one to hold the coefficient, one to hold the variable x , and the third to hold a bit called the mask bit with three registers

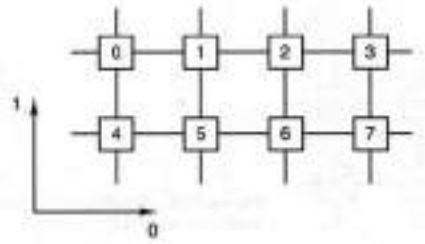


The following represents the steps involved in the computation of $a_i x^i$. Figure a shows the initial values of the registers of each node. The coefficient a_i , for $i=0$ to 7, is stored in node i . The value of the variable x is stored in each node. The mask register of node i , for $i=1, 3, 5$, and 7, is set to 1; others are set to 0. In each step of computation, every node checks the content of its mask register. When the content of the mask register is 1, the content of the coefficient register is multiplied with the content of the variable register, and the result is stored in the coefficient register. When the content of the mask register is zero, the content of the coefficient register remains unchanged. The content of the variable register is multiplied with itself. The contents of the mask registers are shuffled between the nodes using the shuffle network. Figures b, c, and d show the values of the registers after the first step, second step, and third step, respectively. At the end of the third step, each registers contains $a_i x^i$.

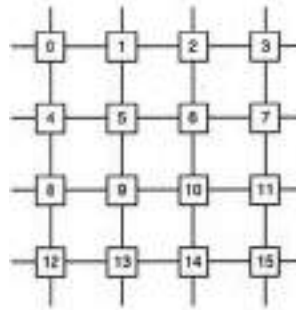


Two-dimensional mesh :

- A two-dimensional mesh consists of $k_1 \times k_0$ nodes, where $k_i \geq 2$ denotes the number of nodes along dimension i .
- The following represents a two-dimensional mesh for $k_0=4$ and $k_1=2$.
- There are four nodes along dimension 0, and two nodes along dimension 1. two-dimensional mesh network each node is connected to its north, south, east, and west neighbors.
- In general, a node at row i and column j is connected to the nodes at locations $(i-1, j)$, $(i+1, j)$, $(i, j-1)$, and $(i, j+1)$.
- The nodes on the edge of the network have only two or three immediate neighbors.



Following figure shows a mesh with 16 nodes. From this point forward, the term mesh will indicate a two dimensional mesh with an equal number of nodes along each dimension.



The routing of data through a mesh can be accomplished in a straightforward manner. The following simple routing algorithm routes a packet from source S to destination D in a mesh with n nodes.

1. Compute the row distance R as

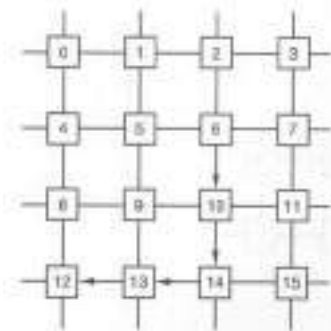
$$R = \lfloor D/n \rfloor - \lfloor S/n \rfloor$$

2. Compute the column distance C as

$$C = D(\bmod n) - S(\bmod n)$$

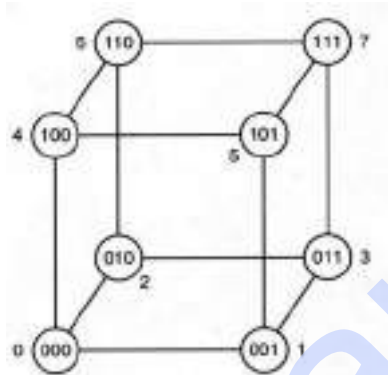
3. Add the values R and C to the packet header at the source node.
4. Starting from the source, send the packet for R rows and then for C columns.

- The values R and C determine the number of rows and columns that the packet needs to travel. The direction the message takes at each node is determined by the sign of the values R and C .
- When R (C) is positive, the packet travels downward (right); otherwise, the packet travels upward (left). Each time that the packet travels from one node to the adjacent node downward, the value R is decremented by 1, and when it travels upward, R is incremented by 1. Once R becomes 0, the packet starts traveling in the horizontal direction.
- Each time that the packet travels from one node to the adjacent node in the right direction, the value C is decremented by 1, and when it travels in the left direction, C is incremented by 1.
- When C becomes 0, the packet has arrived at the destination. For example, to route a packet from node 6 (i.e., $S=6$) to node 12 (i.e., $D=12$), the packet goes through two paths as shown in the figure



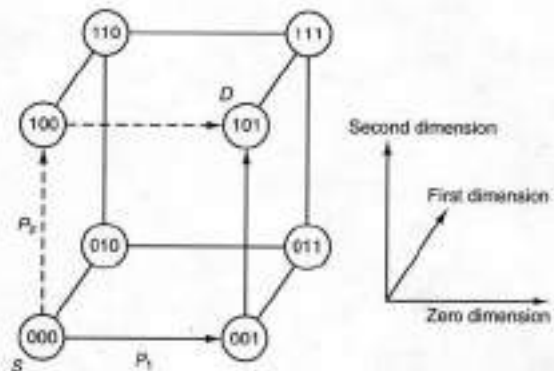
n-Cube or Hypercube :

- n is called the dimension of the n -cube network. When the node addresses are considered as the corners of an n dimensional cube, the network connects each node to its n neighbors.
- n -cube, individual nodes are uniquely identified by n -bit addresses ranging from 0 to $N-1$.
- Given a node with binary address d , this node is connected to all nodes whose binary addresses differ from d in exactly 1 bit.
- For example, in a 3cube, in which there are eight nodes, node 7(111) is connected to nodes 6(110), 5(101), and 3(011).



The above figure explains about the connection between various nodes.

- As can be seen in the 3-cube, two nodes are directly connected if their binary addresses differ by 1 bit.
 - This method of connection is used to control the routing of data through the network in a simple manner. The following simple routing algorithm routes a packet from its source $S = (s_{n-1} \dots s_0)$ to destination $D = (d_{n-1} \dots d_0)$.
1. Tag $T = S \oplus D = t_{n-1} \dots t_0$ is added to the packet header at the source node ($\oplus$ denotes an XOR gate).
 2. If $t_i \neq 0$ for some $0 \leq i \leq n-1$, then use i th-dimension link to send the packet to a new node with the same address as the current node except the i th bit, and change t_i to 0 in the packet header.
 3. Repeat step 2 until $t_i = 0$ for all $0 \leq i \leq n-1$.



- As shown in the above figure to route a packet from node 0 to node 5, the packet could go through two different paths, P_1 and P_2 . Here $T = 000 \oplus 101 = 101$.
- If we first consider the bit t_0 and then t_2 , the packet goes through the path P_1 . Since $t_0 = 1$, the packet is sent through the 0th-dimension link to node 1.

- At node 1, t_0 is set to 0; thus T now becomes equal to 100.
- Now, since $t_2=1$, the packet is sent through the second-dimension link to node 5.
- If, instead of t_0 , bit t_2 is considered first, the packet goes through P_2 .
- The maximum distance between nodes is 3. This is because the distance between nodes is equal to the number of bit positions in which their binary addresses differ.
- Since each address consists of 3 bits, the difference between two addresses can be at most 3 when every bit at the same position differs.
- In general, in an n -cube the maximum distance between nodes is n , making the diameter equal to n .
- The n -cube network has several features that make it very attractive for parallel computation.
- It appears the same from every node, and no node needs special treatment.

Symmetric and Distributed Shared Memory Architectures:

- Numerous designs on how to interconnect the processing nodes and memory modules were published in the literature.
- Examples of such message based systems included Intel Paragon, N-Cube, IBM' SP systems.
- As compared to shared memory systems, distributed memory (or message passing) systems can accommodate larger number of computing nodes.
- This scalability was expected to increase the utilization of message-passing architectures.

There are two principal types of MIMD systems:

- shared-memory systems and
- distributed-memory systems

In a **shared-memory system** a collection of autonomous processors is connected to a memory system via an interconnection network, and each processor can access each memory location. In a shared-memory system, the processors usually communicate implicitly by accessing shared data structures.

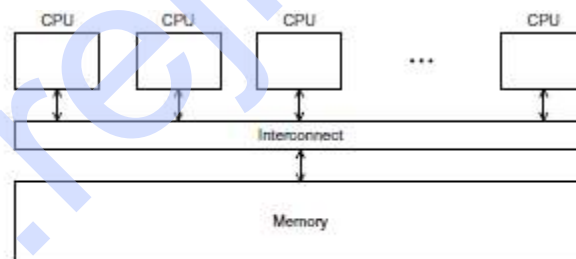


FIGURE 2.3

A shared-memory system

In a **distributed-memory system**, each processor is paired with its own private memory, and the processor-memory pairs communicate over an interconnection network. So in distributed-memory systems the processors usually communicate explicitly by sending messages or by using special functions that provide access to the memory of another processor.

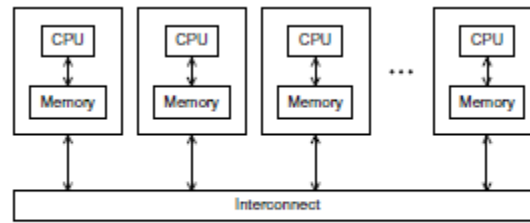


FIGURE 2.4
A distributed-memory system

Shared-memory systems

The most widely available shared-memory systems use one or more multicore processors. A multicore processor has multiple CPUs or cores on a single chip. Typically, the cores have private level 1 caches, while other caches may or may not be shared between the cores.

- In shared-memory systems to different multicore processors, the interconnect can either interface all the processors legitimately to primary memory or every processor can have an immediate association with a square of fundamental memory, and the processors can get to every others' squares of principle memory through exceptional equipment incorporated with the processors. See Figures 2.5 and 2.6.
- In the first type of system, the time to access all the memory locations will be the same for all the cores, while in the second type a memory location to which a core is directly connected can be accessed more quickly than a memory location that must be accessed through another chip.
- Thus, the first type of system is called a uniform memory access, or UMA, system, while the second type is called a nonuniform memory access, or NUMA, system.

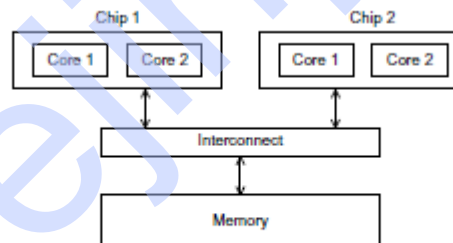


FIGURE 2.5
A UMA multicore system

- UMA frameworks are normally simpler to program, since the software engineer doesn't have to stress over various access times for various memory areas.
- This advantage can be offset by the faster access to the directly connected memory in NUMA systems. Furthermore NUMA systems have the potential to use larger amounts of memory than UMA systems.

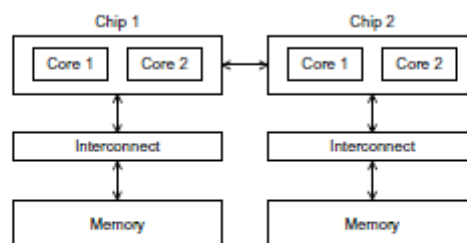


FIGURE 2.6
A NUMA multicore system

Distributed-memory systems

- The most widely available distributed-memory systems are called clusters.
- They are composed of a collection of commodity systems—for example, PCs—connected by commodity interconnection network—for example, Ethernet.
- In fact, the nodes of these systems, the individual computational units joined together by the communication network, are usually shared-memory systems with one or more multicore processors.
- To distinguish such systems from pure distributed-memory systems, they are sometimes called hybrid systems.
- These days, it's generally comprehended that a group will have shared-memory hubs. The framework gives the foundation important to transform huge systems of geologically appropriated PCs into a brought together dispersed memory framework. As a rule, such a framework will be heterogeneous, that is, the individual hubs might be worked from various sorts of equipment.

Differences in Types of implementation:

- There are two major differences in the two implementation, related to how the work is distributed and how the worker processes access the needed data.
- The Pthread version shows that each worker process is given the address of the data they need for their work.
- In the MPI version, the actual data is sent to the worker processors. The worker processes of the Pthread version access the needed data directly as if the data is local.
- It can also be seen that the worker processors directly accumulate their partial results in a single global variable (using mutual exclusion).
- The worker processes of the MPI program are supplied the actual data via messages, and they send their partial results back to the master for the purpose of accumulation.

Issues in Shared Memory Systems :

1. Data Coherency.
 - The use of cache memories is so pervasive in today's computer systems it is difficult to imagine processors without them.
 - Cache memories, along with virtual memories and processor registers form a continuum of memory hierarchies that rely on the principle of locality of reference.
2. Cache Coherency.
 - Techniques are needed to ensure that consistent data is available to all processors in a multiprocessor system.
 - Cache coherency can be maintained either by hardware techniques or software techniques

Entry and Scope Consistency :

- In both Weak ordering and Release consistency models, the shared memory is made consistent when any synchronization variable is released (or accessed).
- if we can associate a set of shared variable with each synchronization variable, then we only need to maintain consistency on these variables when the associated synchronization variable is released (or accessed).

- Entry consistency requires that the program specify the association between shared memory and synchronization variables.
- Scope consistency is similar to Entry consistency, but the associations between shared variables and synchronization variables is implicitly extracted.

Cache Coherence:

- CPU reserves are overseen by framework equipment: software engineers don't have direct power over them.
- A few significant ramifications for shared-memory frameworks. To comprehend these issues, assume we have a mutual memory framework with two centers, every one of which has its own private information reserve. See Figure 2.17. For whatever length of time that the two centers just read shared information, there is no issue.
- For instance, assume that x is a mutual variable that has been instated to 2, y_0 is private and claimed by center 0, and y_1 and z_1 are private and possessed by center 1. Presently assume the accompanying explanations are executed at the showed occasions

Time	Core 0	Core 1
0	$y_0 = x;$	$y_1 = 3 * x;$
1	$x = 7;$	Statement(s) not involving x
2	Statement(s) not involving x	$z_1 = 4 * x;$

- Then the memory location for y_0 will eventually get the value 2, and the memory location for y_1 will eventually get the value 6.
- However, it's not so clear what value z_1 will get. It might at first appear that since core 0 updates x to 7 before the assignment to z_1 , z_1 will get the value $4 * 7 = 28$.
- However, at time 0, x is in the cache of core 1. So unless for some reason x is evicted from core 0's cache and then reloaded into core 1's cache, it actually appears that the original value $x = 2$ may be used, and z_1 will get the value $4 * 2 = 8$.

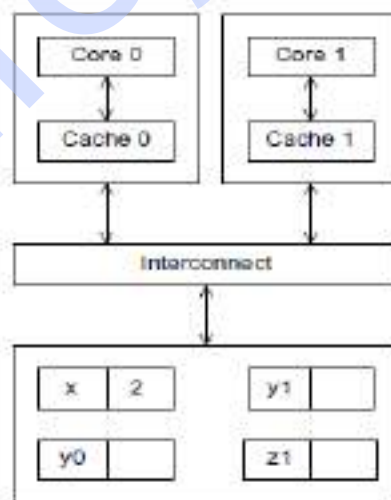


FIGURE 2.17

A shared-memory system with two cores and two caches

- Note that this unpredictable behavior will occur regardless of whether the system is using a write-through or a write-back policy.

- If it's using a write-through policy, the main memory will be updated by the assignment $x = 7$.
- However, this will have no effect on the value in the cache of core 1. If the system is using a write-back policy, the new value of x in the cache of core 0 probably won't even be available to core 1 when it updates $z1$.
- Clearly, this is a problem. The programmer doesn't have direct control over when the caches are updated, so her program cannot execute these apparently innocuous statements and know what will be stored in $z1$.

At the point when shared information are reserved, the mutual worth might be recreated in numerous stores. Notwithstanding the decrease in get to dormancy and required memory transfer speed, this replication likewise gives a decrease in dispute that may exist for shared information things that are being perused by various processors at the same time. This issue is called as reserve cognizance. There are two main approaches to insuring cache coherence:

- snooping cache coherence
- directory-based cache coherence.

Snooping cache coherence :

The idea behind snooping comes from bus-based systems:

1. When the cores share a bus, any signal transmitted on the bus can be "seen" by all the cores connected to the bus.
2. Hence, when center 0 updates the duplicate of x put away in its store, on the off chance that it additionally communicates this data over the transport, and if center 1 is "snooping" the transport, it will see that x has been refreshed and it can check its duplicate of x as invalid. This is pretty much how snooping reserve rationality functions.
3. The principal difference between our description and the actual snooping protocol is that the broadcast only informs the other cores that the cache line containing x has been updated, not that x has been updated.

A couple of points should be made regarding snooping.

1. First, it's not essential that the interconnect be a bus, only that it support broadcasts from each processor to all the other processors.
2. Second, snooping works with both write-through and write back caches.
3. In principle, if the interconnect is shared—as with a bus—with write through caches there's no need for additional traffic on the interconnect, since each core can simply "watch" for writes.
4. With write-back caches, on the other hand, an extra communication is necessary, since updates to the cache don't get immediately sent to memory.

Directory-based cache coherence :

1. Unfortunately, in large networks broadcasts are expensive, and snooping cache coherence requires a broadcast every time a variable is updated .
 2. So snooping cache coherence isn't scalable, because for larger systems it will cause performance to degrade.
 3. For example, suppose we have a system with the basic distributed-memory architecture (Figure 2.4). However, the system provides a single address space for all the memories. So, for example, core 0 can access the variable x stored in core 1's memory, by simply executing a statement such as $y = x$.
 4. Such a system can, in principle, scale to very large numbers of cores. However, snooping cache coherence is clearly a problem since a broadcast across the interconnect will be very slow relative to the speed of accessing local memory.
- Directory-based cache coherence protocols endeavor to take care of this issue using an information structure called a catalog.

- The registry stores the status of each reserve line. Commonly, this information structure is appropriated; in our model, each center/memory pair may be answerable for putting away the piece of the structure that determines the status of the store lines in its neighborhood memory.
- Along these lines, when a line is added something extra to, state, center 0's reserve, the registry passage comparing to that line would be refreshed demonstrating that center 0 has a duplicate of the line.
- At the point when a variable is refreshed, the index is counseled, and the store controllers of the centers that have that variable's reserve line in their reserves are refuted.
- Plainly there will be considerable extra stockpiling required for the index, however when a reserve variable is refreshed, just the centers putting away that variable should be reached

Performance:

➤ Speedup and efficiency

Usually the best we can hope to do is to equally divide the work among the cores, while at the same time introducing no additional work for the cores. If we succeed in doing this, and we run our program with p cores, one thread or process on each core, then our parallel program will run p times faster than the serial program.

- If we call the serial run-time T_{serial} and our parallel run-time T_{parallel} , then the best we can hope for is $T_{\text{parallel}} = T_{\text{serial}}/p$. When this happens, we say that our parallel program has **linear speedup**. So if we define the **speedup** of a parallel program to be

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}},$$

then linear speedup has $S = p$, which is unusual. Furthermore, as p increases, we expect S to become a smaller and smaller fraction of the ideal, linear speedup p .

- Another method for saying this is $S=p$ will most likely get littler and littler as p increases..This esteem, $S=p$, is in some cases called the effectiveness of the equal program. In the event that we substitute the recipe for S , we see that the proficiency is

$$E = \frac{S}{p} = \frac{\left(\frac{T_{\text{serial}}}{T_{\text{parallel}}}\right)}{p} = \frac{T_{\text{serial}}}{p \cdot T_{\text{parallel}}}.$$

- Many parallel programs are developed by dividing the work of the serial program among the processes/threads and adding in the necessary “parallel overhead” such as mutual exclusion or communication.
- Therefore, if T_{overhead} denotes this parallel overhead, it's often the case that $T_{\text{parallel}} = T_{\text{serial}}/p + T_{\text{overhead}}$.
- Furthermore, as the problem size is increased, T_{overhead} often grows more slowly than T_{serial} . When this is the case the speedup and the efficiency will increase.
- A final issue to consider is what values of T_{serial} should be used when reporting speedups and efficiencies.

➤ Amdahl's law

- Gene Amdahl mentioned an objective fact that is gotten known as Amdahl's law. It says, generally, that except if basically the entirety of a sequential program is parallelized, the conceivable speedup will be exceptionally constrained—paying little heed to the quantity of centers accessible.
- Suppose, for example, that we're able to parallelize 90% of a serial program.
- Further suppose that the parallelization is “perfect,” that is, regardless of the number of cores p we use, the speedup of this part of the program will be p .

- o If the serial run-time is $T_{\text{serial}} = 20$ seconds, then the run-time of the parallelized part will be $0.9 \times T_{\text{serial}}/p = 18/p$ and
- o The run-time of the “unparallelized” part will be $0.1 \times T_{\text{serial}} = 2$. The overall parallel run-time will be

$$T_{\text{parallel}} = 0.9 \times T_{\text{serial}}/p + 0.1 \times T_{\text{serial}} = 18/p + 2,$$

and the speedup will be

$$S = \frac{T_{\text{serial}}}{0.9 \times T_{\text{serial}}/p + 0.1 \times T_{\text{serial}}} = \frac{20}{18/p + 2}.$$

- o Now as p gets larger and larger, $0.9 \times T_{\text{serial}}/p = 18/p$ gets closer and closer to 0, so the total parallel run-time can't be smaller than $0.1 \times T_{\text{serial}} = 2$. That is, the denominator in S can't be smaller than $0.1 \times T_{\text{serial}} = 2$. The fraction S must therefore be smaller than

$$S \leq \frac{T_{\text{serial}}}{0.1 \times T_{\text{serial}}} = \frac{20}{2} = 10.$$

- o That is, $S \leq 10$. This is saying that even though we've done a perfect job in parallelizing 90% of the program, and even if we have, say, 1000 cores, we'll never get a speedup better than 10.

➤ Scalability

- o The word “scalable” has a wide variety of informal uses. Roughly speaking, a technology is scalable if it can handle ever-increasing problem sizes.
- o However, in discussions of parallel program performance, scalability has a somewhat more formal definition.
- o Suppose we run a parallel program with a fixed number of processes/threads and a fixed input size, and we obtain an efficiency E .
- o Suppose we now increase the number of processes/threads that are used by the program. If we can find a corresponding rate of increase in the problem size so that the program always has efficiency E , then the program is scalable.
- o As an example, suppose that $T_{\text{serial}} = n$, where the units of T_{serial} are in microseconds, and n is also the problem size. Also suppose that $T_{\text{parallel}} = n/p + 1$. Then

$$E = \frac{n}{p(n/p + 1)} = \frac{n}{n + p}.$$

- o To see if the program is scalable, we increase the number of processes/threads by a factor of k , and we want to find the factor x that we need to increase the problem size by so that E is unchanged. The number of processes/threads will be kp and the problem size will be xn , and we want to solve the following equation for x :

$$E = \frac{n}{n + p} = \frac{xn}{xn + kp}.$$

- o Well, if $x = k$, there will be a common factor of k in the denominator $xn + kp = kn + kp = k(n + p)$, and we can reduce the fraction to get

$$\frac{xn}{xn + kp} = \frac{kn}{k(n + p)} = \frac{n}{n + p}.$$

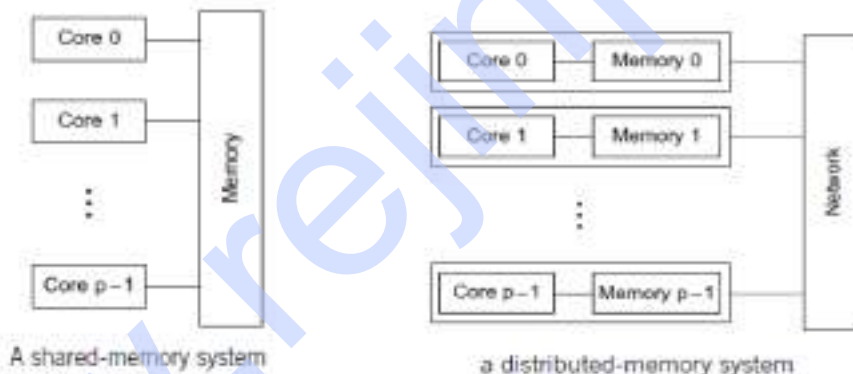
➤ Taking timings

There are a lot of different approaches, and with parallel programs the details may depend on the API. However, there are a few general observations we can make that may make things a little easier.

- The first thing to note is that there are at least two different reasons for taking timings.
- During program development we may take timings in order to determine if the program is behaving as we intend.
- For example, in a distributed-memory program we might be interested in finding out how much time the processes are spending waiting for messages, because if this value is large, there is almost certainly something wrong either with our design or our implementation.
- On the other hand, once we've completed development of the program, we're often interested in determining how good its performance is.
- Perhaps surprisingly, the way we take these two timings is usually different

Parallel Program Design:

- The two main types of parallel systems are: shared memory systems and distributed memory systems.
- In a shared-memory system, the cores can share access to the computer's memory; in principle, each core can read and write each memory location.
- In a shared-memory system, we can coordinate the cores by having them examine and update shared-memory locations.
- In a distributed-memory system, on the other hand, each core has its own, private memory, and the cores must communicate explicitly by doing something like sending messages across a network.



- The structural and computational patterns are composed to define software architecture.
- A software architecture defines the components that make up an application, the communication among components, and the fundamental computations that occur inside components.
- The software architecture, however, has little to say about how the software architecture is mapped onto the hardware of a parallel computer.
- To address parallelism, we combined the computational and structural patterns with the lower level, parallel programming design pattern languages from these parallel programming patterns define a distinct pattern language for parallel programming (PLPP).
- It's quite direct to compose a sequential program that creates a histogram. We have to choose what the containers are, decide the quantity of estimations in each receptacle, and print the bars of the histogram. Since we're not concentrating on I/O, we'll constrain ourselves to simply the initial two stages, so the info will be
 1. the number of measurements, data count
 2. an array of data count floats, data

3. the minimum value for the bin containing the smallest values, min
4. the maximum value for the bin containing the largest values, max
5. the number of bins, bin count;

- The output will be an array containing the number of elements of data that lie in each bin. To make things precise, we'll make use of the following data structures:

- bin_maxes. An array of bin_count floats
 - bin_counts. An array of bin_count ints

- The cluster receptacle maxes will store the upper destined for each container, and canister checks will store the quantity of information components in each canister. To be unequivocal, we can characterize

```
bin_width = (max_meas - min_meas)/bin_count
```

Then bin maxes will be initialized by

```
for (b = 0; b < bin_count; b++)  
    bin_maxes[b] = min_meas + bin_width*(b+1);
```

- We'll embrace the show that receptacle b will be all the estimations in the range

```
bin_maxes[b-1] <= measurement < bin_maxes[b]
```

- Obviously, this doesn't bode well if b = 0, and right now utilize the standard that receptacle 0 will be the estimations in the range

```
min_meas <= measurement < bin_maxes[0]
```

- This implies we generally need to treat receptacle 0 as an uncommon case, however this isn't excessively burdensome. When we've instated container maxes and doled out 0 to all the components of bin_counts, we can get the checks by utilizing the accompanying pseudo-code

```
for (i = 0; i < data_count; i++) {  
    bin = Find_bin(data[i], bin_maxes, bin_count, min_meas);  
    bin_counts[bin]++;  
}
```

- The Find_bin work restores the receptacle that data[i] has a place with. This could be a straightforward direct pursuit work: search through receptacle maxes until you discover a canister b that fulfills

UNIT - II

Performance – Scalability – Synchronization and data sharing – Data races – Synchronization primitives (mutexes, locks, semaphores, barriers) – deadlocks and livelocks – communication between threads condition variables, signals, message queues and pipes).

The past few decades have seen large fluctuations in the perceived value of parallel computing. At times, parallel computation has optimistically been viewed as the solution to all of our computational limitations. At different occasions, many have contended that it is a misuse of exertion given the rate at which processor paces and memory costs keep on improving. Recognitions keep on swaying between these two limits because of various variables, among them: the consistent changes in the "hot" issues being explained, the programming conditions accessible to clients, the supercomputing market, the merchants associated with building these supercomputers, and the scholarly network's concentration at some random point and time. The outcome is a fairly muddled picture from which it is difficult to impartially pass judgment on the worth and guarantee of equal registering.

- Picking a small workload will mislead you as to which parts of the code need to be optimized.
- You may have spent time optimizing the algorithmically simpler part of the code, when the performance of the application in a real-world situation will be dominated by the algorithmically complex part of the code.
- This emphasizes why it is important to select appropriate workloads for developing and testing the application.
- Different parts of the application will scale differently as the workload size changes, and regions that appear to take no time can suddenly become dominant.
- Another important point to realize is that a change of algorithm is one of the few things that can make an order of magnitude difference to performance.
- If 80% of the application's runtime was spent arranging a 1,000-component cluster, at that point changing from an air pocket sort to a quicksort could have a 300× effect to the exhibition of that work, making the time spent arranging 300× littler than it recently performed.
- The 80% of the runtime spent arranging would to a great extent vanish, and the application would wind up running around multiple times quicker..

The following table illustrates, it takes remarkably few elements for an $O(N^2)$ algorithm to start consuming significant amounts of time.

Elements	$O(1)$	$O(N)$	$O(N \log_2 N)$	$O(N^2)$
1	100ns	100ns	100ns	100ns
10	100ns	1,000ns	3,322ns	10,000ns
100	100ns	10,000ns	66,439ns	1,000,000ns
1,000	100ns	100,000ns	996,578ns	100,000,000ns
10,000	100ns	1,000,000ns	13,287,712ns	10,000,000,000ns

The challenges to realizing this potential can be grouped into two main problems: the hardware problem and the software problem.

Structure Impacts Performance

Three attributes of the construction of an application can be considered as “structure.”

1. The first of these is the assemble structure, for example, how the source code is appropriated between the source records.
 2. The second structure is the manner by which the source records are consolidated into applications and supporting libraries
 3. Finally, and likely the most self-evident, is that way information is sorted out in the application
- Performance and Convenience Trade-Offs in Source Code and Build Structures The structure of the source code for an application can make contrasts its exhibition.
 - Source code is frequently disseminated across source documents for the comfort of the engineers.
 - Performance openings are lost when the compiler sees just a solitary record at once.
 - The single record may not present the compiler with all the open doors for enhancements that it may have had if it somehow managed to see a greater amount of the source code.
 - This sort of restriction is obvious when a program utilizes an accessor work—a short capacity that profits the estimation of some factor.
 - A trivial optimization is for the compiler to replace this function call with a direct load of the value of the variable

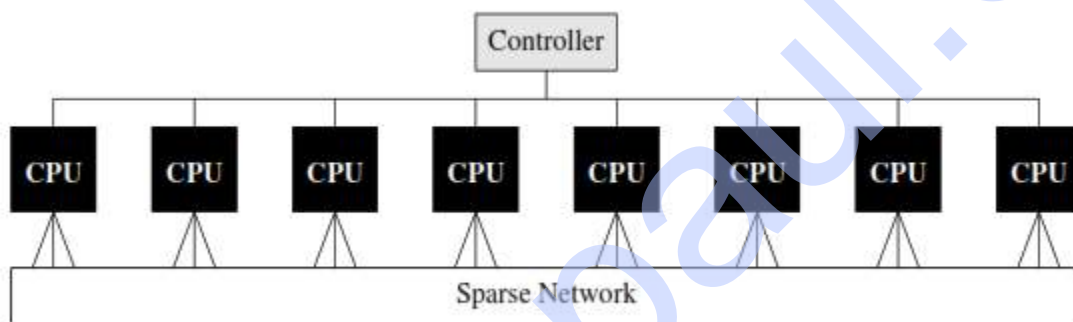
Parallel Architectures :

This dissertation categorizes parallel platforms as being one of three rough types:

1. Distributed memory
 2. Shared memory
 3. Shared address space.
- This taxonomy is somewhat coarse given the wide variety of parallel architectures that have been developed, but it provides a useful characterization of current architectures for the purposes of this dissertation.
 - Distributed memory machines are considered to be those in which each processor has a local memory with its own address space.
 - A processor's memory can't be gotten to legitimately by another processor, requiring the two processors to be included when imparting esteems starting with one memory then onto the next. Instances of circulated memory machines incorporate item Linux bunches.
 - Shared memory machines are those in which a solitary location space and worldwide memory are shared between different processors.
 - Each processor claims a neighborhood store, and its qualities are kept sound with the worldwide memory by the working framework.
 - Data can be traded between processors essentially by putting the qualities, or pointers to values, in a predefined area and synchronizing suitably.
 - Examples of shared memory machines include the SGI Origin series and the Sun Enterprise. Shared address space architectures are those in which each processor has its own local memory, but a single shared address space is mapped across the distinct memories.
 - Such architectures allow a processor to access the memories of other processors without their direct involvement, but they differ from shared memory machines in that there is no implicit caching of values located on remote machines.
 - The primary example of a shared address machine is Cray's T3D/T3E line.

The CTA Machine Model

- Compilation and execution on these diverse architectures by describing them using a single machine model known as the Candidate Type Architecture (CTA)
- The CTA is a reasonably vague model, and deliberately so. It characterizes parallel machines as a group of von Neumann processors, connected by a sparse network of unspecified topology. Each processor has a local memory that it can access at unit cost.
- Processors can also access other processors' values at a cost significantly higher than unit cost by communicating over the network.
- The CTA also specifies a controller used for global communications and synchronization, though that will not be of concern in this discussion. The following figure explains about CTA architecture



Using Libraries to Structure Applications :

Libraries are the usual mechanism for structuring applications as they become larger. There are some good technical reasons to use libraries:

- Common functionality can be extracted into a library that can be shared between different projects or applications. This can lead to better code reuse, more efficient use of developer time, and more effective use of memory and disk space.
- Placing functionality into libraries can lead to more convenient upgrades where only the library is upgraded instead of replacing all the executables that use the library.
- Libraries can provide better separation between interface and implementation. The implementation details of the library can be hidden from the users, allowing the implementation of the library to evolve while maintaining a consistent interface.
- Stratifying functionality into libraries according to frequency of use can improve application start-up time and memory footprint by loading only the libraries that are needed. Functionality can be loaded on demand rather than setting up all possible features when the application starts.
- Libraries can be used as a mechanism to dynamically provide enhanced functionality. The functionality can be made available without having to change or even restart the application.
- Libraries can enable functionality to be selected based on the runtime environment or characteristics of the system. For instance, an application may load different optimized libraries depending on the underlying hardware or select libraries at runtime depending on the type of work it is being asked to perform.

On the other hand, there are some nontechnical reasons why functionality gets placed into libraries. These reasons may represent the wrong choice for the user.

- Libraries frequently speak to an advantageous item for an authoritative unit. One gathering of engineers may be answerable for a specific library of code, however that doesn't naturally suggest that a solitary library speaks to the most ideal route for that code to be conveyed to the end clients.
- Libraries are likewise used to gather related usefulness. For instance, an application may contain a library of string-taking care of capacities. Such a library may be suitable in the event that it contains a huge group of code. Then again, in the event that it contains just a couple of little schedules, it may be progressively proper to join it with another library.

There are a few contributors to cost:

- Library calls might be executed utilizing a table of capacity addresses. This table might be a rundown of addresses for the schedules remembered for a library. A library routine calls into this table, which at that point hops to the real code for the everyday practice.
- Each library and its information are normally put onto new TLB passages. Calls into a library will as a rule additionally bring about an ITLB miss and conceivably a DTLB miss if the code gets to library-explicit information.
- If the library is being languid stacked (that is, stacked into memory on request), there will be costs related with circle access and setting up the addresses of the library capacities in memory.
- Unix stages regularly give libraries as position-free code. This empowers a similar library to be partaken in memory between various running applications. The expense of this is an expansion in code length. Windows makes the contrary exchange off; it utilizes position-subordinate code in libraries, lessening the chance of sharing libraries between running applications however delivering somewhat quicker code.

```
$ more liba.c
#include <stdio.h>

void ina()
{
    printf( "In library A\n" );
}

$ more libb.c
#include <stdio.h>

void inb()
{
    printf( "In library B\n" );
}
```

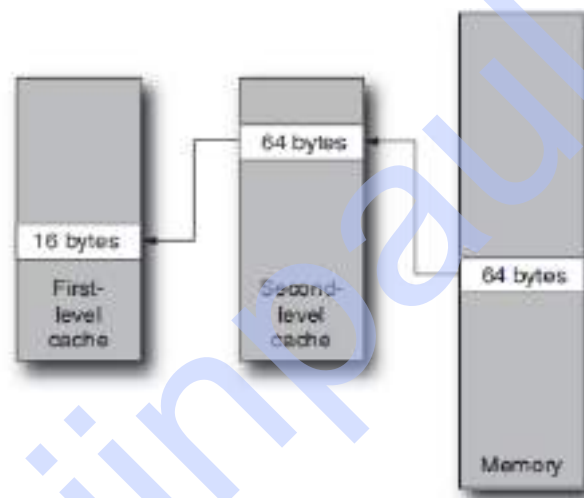
Impact of Data Structures on Performance :

- When an application needs an item of data, it fetches it from memory and installs it in cache.
- The idea with caches is that data that is frequently accessed will become resident in the cache.
- The cost of fetching data from the cache is substantially lower than the cost of fetching it from memory.
- Hence, the application will spend less time waiting for frequently accessed data to be retrieved from memory.

- The amount of data loaded into each level of cache by a load instruction depends on the size of the cache line.
- 64 bytes is a typical length for a cache line; however, some caches have longer lines than this, and some caches have shorter lines.
- Often the caches that are closer to the processor have shorter lines, and the lines further from the processor have longer lines.

The following figure illustrates what happens when a line is fetched into cache from memory.

- On a cache miss, a cache line will be fetched from memory and installed into the second level cache. The portion of the cache line requested by the memory operation is installed into the first-level cache.
- In this scenario, Accesses to data on the same 16-byte cache line as the original item will also be available from the first-level cache.



- Accesses to data that share the same 64-byte cache line will be fetched from the second-level cache.
- Accesses to data outside the 64-byte cache line will result in another fetch from memory.

If data is fetched from memory when it is needed, the processor will experience the entire latency of the memory operation. On a modern processor, the time taken to perform this fetch can be several hundred cycles.

There are techniques that reduce this latency:

1. Out-of-request execution is the place the processor will look the guidance stream for future guidelines that it can execute.
 2. Hardware prefetching of information streams Hardware prefetching can be exceptionally compelling in circumstances where information is brought as a stream or through a strided get to design. It can't prefetch information where the entrance design is less evident.
 3. Software prefetching is the demonstration of adding guidelines to bring information from memory before it is required.
- Another approach to covering memory latency costs is with CMT processors. When one thread stalls because of a cache miss, the other running threads get to use the processor resources of the stalled thread.

- This approach does not improve the execution speed of a single thread. This can enable the processor to achieve more work by sustaining more active threads, improving throughput rather than single-threaded performance.
- There are a number of common coding styles that can often result in suboptimal layout of data in memory.

Improving Performance Through Data Density and Locality :

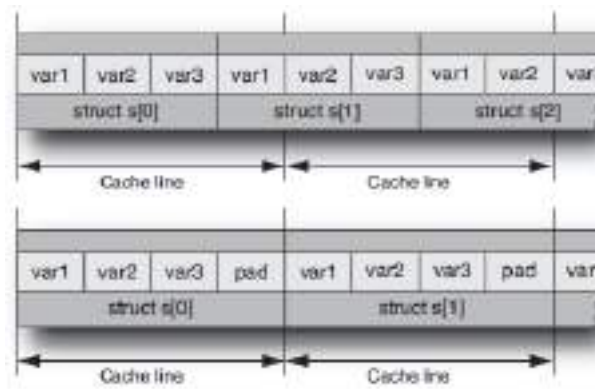
- Paying regard for the request in which factors are pronounced and spread out in memory can improve execution.
- When a heap acquires a variable from memory, it likewise gets the remainder of the store line in which the variable resides.
- Placing factors that are ordinarily gotten to together into a structure so they dwell on a similar store line will prompt execution gains.

```
struct s
{
int var1;
int padding1[15];
int var2;
int padding2[15];
}
```

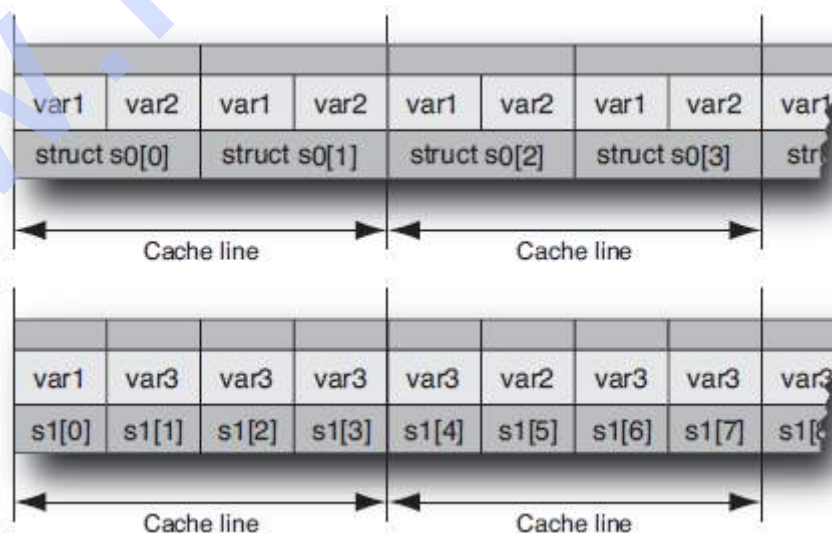
When the structure member var1 is accessed, the fetch will also bring in the surrounding 64 bytes. The size of an integer variable is 4 bytes, so the total size of var1 plus padding1 is 64 bytes. This ensures that the variable var2 is located on the next cache line. Important Structure Members Are Likely to Share a Cache Line

```
struct s
{
int var1;
int var2;
int padding1[15];
int padding2[15];
}
```

- If the structure doesn't fit precisely into the length of the reserve line, there will be circumstances when the neighboring var1 and var2 are part more than two cache lines.
- This presents an issue. Is it better to pack the structures as close as conceivable to fit however many of them as would be prudent into a similar reserve line, or is it better to add cushioning to the structures to make them reliably line up with the store line limits? The accompanying figure shows the two circumstances.
- The answer will rely upon different components. By and large, the best answer is likely to pack the structures as firmly as would be possible.
- This will imply that when one structure is gotten to, the entrance will likewise bring pieces of the encompassing structures.



- Where it is appropriate to add padding to the structure is when the structures are always accessed randomly, so it is more important to ensure that the critical data is not split across a cache line.
- The performance impact of poorly ordered structures can be hard to detect. The cost is spread over all the accesses to the structure over the entire application.
- Reordering the structure members can improve the performance for all the routines that access the structures.
- Determining the optimal layout for the structure members can also be difficult. One guideline would be to order the structure members by access frequency or group them by those that are accessed in the hot regions of code.
- It is also worth considering that changing the order of structure members could introduce a performance regression if the existing ordering happens to have been optimal for a different frequently executed region of code.
- A similar optimization is structure splitting, where an existing structure is split into members that are accessed frequently and members that are accessed infrequently.
- If the infrequently accessed structure members are removed and placed into another structure, then each fetch of data from memory will result in more of the critical structures being fetched in one action.
- Taking the previous example, where we assume that var3 is rarely needed, we would end up with a resulting pair of structures, as shown in following figure.



In this instance, the original structure *s* has been split into two, with *s0* containing all the frequently accessed data and *s1* containing all the infrequently accessed data. In the limit, this optimization is converting what might be an array of structures into a set of arrays, one for each of the original structure members.

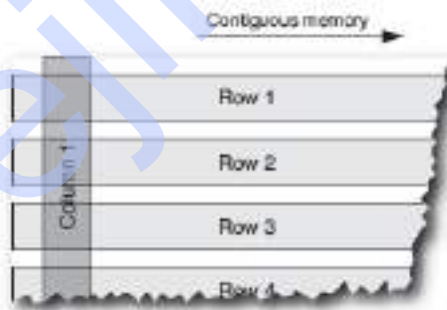
Selecting the Appropriate Array Access Pattern :

One common data access pattern is striding through elements of an array. The performance of the application would be better if the array could be arranged so that the selected elements were contiguous. The following code explains Non contiguous memory access pattern .

Noncontiguous Memory Access Pattern

```
{  
double ** array;  
double total=0;  
...  
for (int i=0; i<cols; i++)  
for (int j=0; j<rows; j++)  
total += array[j][i];  
...  
}
```

- These components won't be situated in bordering memory. In Fortran, the contrary requesting is followed, so neighboring components of the primary file are nearby in memory.
- This is called section significant request. Getting to components by a walk is a typical blunder in codes made an interpretation of from Fortran into C. shows how memory is tended to in C, where nearby components straight are neighboring in memory.



- Fortunately, most compilers are often able to correctly interchange the loops and improve the memory access patterns.
- However, there are many situations where the compiler is unable to make the necessary transformations because of aliasing or the order in which the elements are accessed in the loop.
- In these cases, it is necessary for the developer to determine the appropriate layout and then restructure the code appropriately

Scalability :

Data Races and the Scaling Limitations of Mutex Locks :

1. Bugs as a result of information races are the most evident appearance of a parallelization issue.
2. These identify with updates of factors without guaranteeing selective access to the factors.

3. These are normally settled by including synchronization natives, (for example, mutex locks) into the code to guarantee selective access to the factors.
 4. Although mutex locks can be utilized to guarantee that lone a solitary string approaches an asset at once, they can't authorize the requesting of gets to information.
- An alternative approach is required if there is an ordering constraint on the accesses to shared resources.
 - For example, if two threads need to update a variable, a mutex can ensure that they do not update the variable at the same time.
 - However, a mutex cannot force one of the two threads to be the last to perform the update. The problem with adding mutex locks into the code is that they serialize the access to the variables.
 - Only a single thread can hold the lock, so if there are multiple threads that need to access the data, the application effectively runs serially because only one thread can make progress at a time.
 - Even if requiring the mutex lock is a rare event, it can become a bottleneck if the lock is held for a long time or if there are many threads requiring access to the lock.

Here we find out scaling issues in code provided not only in application and also in libraries. The following is the Code to Testing Scaling of malloc() and free()

```
#include <stdlib.h>
#include <pthread.h>
int nthreads;
void *work( void * param )
{
    int count = 1000000 / nthreads;
    for( int i=0; i<count; i++ )
    {
        void *mem = malloc(1024);
        free( mem );
    }
}
int main( int argc, char*argv[] )
{
    pthread_t thread[50];
    nthreads = 8;
    if ( argc > 1 )
    {
        nthreads = atoi( argv[1] );
    }
    for( int i=0; i<nthreads; i++ )
    {
        pthread_create( &thread[i], 0, work, 0 );
    }
    for( int i=0; i<nthreads; i++ )
    {
```

```
        pthread_join( thread[i], 0 );  
    }  
    return 0;  
}
```

- If a default execution of malloc() and free() utilizes a solitary mutex lock then execution won't improve with numerous strings.
- Consider an option malloc() that utilizes an alternate calculation. Each string has its own load of memory, so it doesn't require a mutex lock. This option malloc() scales as the quantity of strings increments.
- As expected, the default execution doesn't scale, so the runtime doesn't improve. The expansion in runtime is a direct result of more strings battling for the single mutex lock..
- The elective execution shows awesome scaling. As the quantity of strings builds, the runtime of the application decreases.
- For the singlethreaded case, the default malloc() gives preferred execution over the elective usage. The calculation that gives improved scaling likewise adds an expense to the single-strung circumstance; it tends to be difficult to create a calculation that is quick for the single-strung case and scales well with different strings.

Superlinear Scaling :

- Doubling the resources yet get more than double the performance as a result.
- In most instances, going from one thread to two will result in, at most, a doubling of performance.
- However, there will be applications that do see super linear scaling—the application ends up running more than twice as fast.
- This is typically because the data that the application uses becomes cache resident at some point. Imagine an application that uses 4MB of data.
- On a processor with a 2MB cache, only half the data will be resident in the cache.
- Adding a second processor adds an additional 2MB of cache; then all the data becomes cache resident, and the time spent waiting on memory becomes substantially lower.

Challenges to Parallel Programming :

- Writing parallel programs is strictly more difficult than writing sequential ones.
- In sequential programming, the programmer must design an algorithm and then express it to the computer in some manner that is correct, clear, and efficient to execute.
- Parallel programming involves these same issues, but also adds a number of additional challenges that complicate development and have no counterpart in the sequential realm.
- These challenges include: finding and expressing concurrency, managing data distributions, managing interprocessor communication, balancing the computational load, and simply implementing the parallel algorithm correctly.

Hardware constraints applicable to improve scaling.

There are three critical areas that can make a large difference to scaling.

1. The amount of bandwidth to cache and the memory will be divided among the active threads on the system.

2. The design of the caches will determine how much time is lost because of capacity and conflict-induced cache misses.
3. The way that the processor core pipelines are shared between active software threads will determine how instruction issue rates change as the number of active threads increases.

Bandwidth Sharing Between Cores :

- Bandwidth is another asset shared between threads.
- The data transfer capacity limit of a framework relies upon the structure of the processor and the memory framework just as the memory chips and their area in the framework.
- The data transfer capacity a processor can expend is a component of the quantity of remarkable memory demands and the rate at which these can be returned.
- These memory solicitations can emerge out of either equipment or programming prefetches, just as from burden or store tasks.
- Since each string can give memory demands, the more strings that a processor can run, the more data transmission the processor can expend. string-taking care of library schedules, for example, strlen() or memset() can be enormous shoppers of memory transfer speed.
- The following code explains memset to define memory bandwidth

```
#include <stdio.h>
#include <stdlib.h>
#include <strings.h>
#include <pthread.h>
#include <sys/time.h>
#define BLOCKSIZE 1024*1025
int nthreads = 8;
char * memory;
double now()
{
    struct timeval time;
    gettimeofday( &time, 0 );
    return (double)time.tv_sec + (double)time.tv_usec / 1000000.0;
}
void *experiment( void *id )
{
    unsigned int seed = 0;
    int count = 20000;
    for( int i=0; i<count; i++ )
    {
        memset( &memory[BLOCKSIZE * (int)id], 0, BLOCKSIZE );
    }
    if ( seed == 1 ){ printf( "" ); }
}
int main( int argc, char* argv[] )
{
```

```
pthread_t threads[64];
memory = (char*)malloc( 64*BLOCKSIZE );
if ( argc > 1 )
{
    nthreads = atoi( argv[1] );
}
double start = now();
for( int i=0; i<nthreads; i++ )
{
    pthread_create( &threads[i], 0, experiment, (void*)i );
}
for ( int i=0; i<nthreads; i++ )
{
    pthread_join( threads[i], 0 );
}
double end = now();
printf( "%i Threads Time %f s Bandwidth %f GB/s\n", nthreads, (end - start), ((double)nthreads *
BLOCKSIZE * 20000.0) / ( end - start) / 1000000000.0 );
return 0;
}
```

the bandwidth measured by the test code for one to eight virtual CPUs on a system with 64 virtual CPUs.
Memory Bandwidth Measured on a System with 64 Virtual CPUs

1 Threads Time 7.082376 s Bandwidth 2.76 GB/s
2 Threads Time 7.082576 s Bandwidth 5.52 GB/s
3 Threads Time 7.059594 s Bandwidth 8.31 GB/s
4 Threads Time 7.181156 s Bandwidth 10.89 GB/s
5 Threads Time 7.640440 s Bandwidth 12.79 GB/s
6 Threads Time 11.252412 s Bandwidth 10.42 GB/s
7 Threads Time 14.723671 s Bandwidth 9.29 GB/s
8 Threads Time 17.267288 s Bandwidth 9.06 GB/s

For this particular system, the bandwidth scales nearly linearly with the number of threads until about six threads. After six threads, the bandwidth reduces.

There are several effects that can cause this :

- The threads are interfering on the processor.
- A second interaction effect is if the threads start interfering in the caches, such as multiple threads attempting to load data to the same set of cache lines.
- One other impact is the conduct of memory chips when they become soaked. the chips begin encountering lining latencies where the reaction time for each solicitation increments. Memory chips are organized in banks. Getting to a specific location will prompt a solicitation to a specific bank of memory. Each bank needs a hole between returning two reactions. In the event that different strings happen to hit a similar bank, at that point the reaction time becomes represented by the rate at which the bank can return memory back.

Memory Bandwidth Measured on a System with Four Virtual CPUs

1 Threads Time 7.437563 s Bandwidth 2.63 GB/s

2 Threads Time 15.238317 s Bandwidth 2.57 GB/s

3 Threads Time 24.580981 s Bandwidth 2.39 GB/s

4 Threads Time 37.457352 s Bandwidth 2.09 GB/s

False Sharing

False sharing is the situation where multiple threads are accessing items of data held on a single cache line. Although the threads are all using separate items of data, the cache line itself is shared between them so only a single thread can write to it at any one time. This is purely a performance issue

Example of False Sharing

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sys/time.h>

double now()
{
    struct timeval time;
    gettimeofday( &time, 0 );
    return (double)time.tv_sec + (double)time.tv_usec / 1000000.0;
}

#define COUNT 100000000
volatile int go = 0;
volatile int counters[20];
void *spin( void *id )
{
    int myid = (int)id + 1;
    while( !go ) {}
    counters[myid] = 0;
    while ( counters[myid]++ < COUNT ) {}
}

int main( int argc, char* argv[] )
{
    pthread_t threads[256];
    int nthreads = 1;
    if ( argc > 1 ) { nthreads = atoi( argv[1] ); }
    for( int i=1; i<nthreads; i++ )
    {
        pthread_create( &threads[i], 0, spin, (void*)i );
    }
    double start = now();
    go = 1;
    spin( 0 );
```

```
double end = now();
printf("Time %f ns\n", ( end – start ) );
for( int i=0; i<nthreads; i++ )
{
pthread_join( threads[i], 0 );
}
return 0;
}
```

If we run ABOVE CODE WITH a single thread, the thread completes its work in about nine seconds on a system with two dual-core processors. Using four threads on the same system results in a runtime for the code of about 100 seconds—a slowdown of about 10 times. It is very easy to solve false sharing by padding the accessed structures so that the variable used by each thread resides on a separate cache line.

Cache Conflict and Capacity :

- The two issues that can occur with shared caches: capacity misses and conflict misses. A conflict cache miss is where one thread has caused data needed by another thread to be evicted from the cache.
- Data structures such as stacks tend to be aligned on cache line boundaries, which increases the likelihood that structures from different processes will map onto the same address.
- The following code is used to Print the Stack Address for Different Threads

```
printf("Stack base address = %x for thread %i\n", &stack, (int)param);
```
- The expected output when this code is run on 32-bit Solaris indicates that threads are created with a 1MB offset between the start of each stack.
- For a processor with a cache size that is a power of two and smaller than 1MB, a stride of 1MB would ensure the base of the stack for all threads is in the same set of cache lines.
- The associativity of the cache will reduce the chance that this would be a problem.
- A cache with an associativity greater than the number of threads sharing is less likely to have a problem with conflict misses.

Data Races

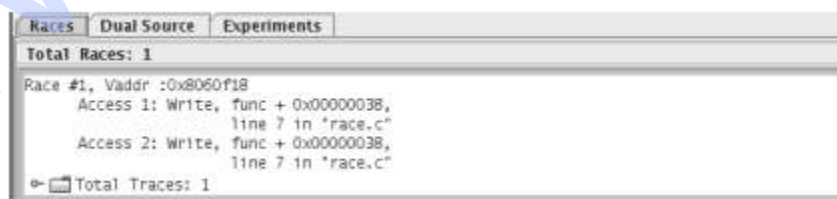
- Data races are the most widely recognized programming blunder found in equal code. An information race happens when numerous strings utilize similar information thing and at least one of those strings are refreshing it. It is best delineated by a model. Assume you have the code appeared in following code, where a pointer to a whole number variable is passed in and the capacity augments the estimation of this variable by 4.

```
void update(int * a)
{
    *a = *a + 4;
}
```

Value of variable a = 10	
Thread 1	Thread 2
ld [%0], %01 // Load %01 = 10	ld [%0], %01 // Load %01 = 10
add %01, 4, %01 // Add %01 = 14	add %01, 4, %01 // Add %01 = 14
st %01, [%0] // Store %01	st %01, [%0] // Store %01
Value of variable a = 14	

- In the example, each thread adds 4 to the variable, but because they do it at exactly the same time, the value 14 ends up being stored into the variable. If the two threads had executed the code at different times, then the variable would have ended up with the value of 18.
- Another circumstance may be the point at which one string is running, yet the other string has been setting turned off of the processor. Envision that the primary string has stacked the estimation of the variable an and afterward gets setting turned off the processor. At the point when it in the end runs once more, the estimation of the variable a will have changed, and the last store of the reestablished string will cause the estimation of the variable a to relapse to an old worth.
- "Using POSIX Threads." The code makes two threads, the two of which execute the routine func(). The fundamental thread at that point hangs tight for both the threads to finish their work. The two threads will endeavor to augment the variable counter. We can incorporate this code with GNU gcc and afterward use Helgrind, which is a piece of the Valgrind1 suite, to distinguish the information race. Valgrind is an tool that empowers an application to be instrumented and its runtime conduct inspected. The Helgrind tool utilizes this instrumentation to accumulate information about information races.
- The output from Helgrind shows that there is a potential data race between two threads, both executing line 7 in the file race.c.
- Another tool that is able to detect potential data races is the Thread Analyzer in Oracle Solaris Studio. This tool requires an instrumented build of the application, data collection is done by the collect tool, and the graphical interface is launched with the command tha. The following code performs detecting Data races using Sun studio Thread Analyzer.

```
$ cc -g -xinstrument=datarace race.c
$ collect -r on ./a.out
Recording experiment tha.1.er ...
$ tha tha.1.er
```



- The above figure explains list of data races detected by Solaris studio Thread Analyzer

Synchronization primitives:

- Synchronization is used to coordinate the activity of multiple threads. Most operating systems provide a rich set of synchronization primitives.
- It is usually most appropriate to use these rather than attempting to write custom methods of synchronization.
- The tools will be able to do a better job of detecting data races or correctly labeling synchronization costs.

Mutexes and Critical Regions :

- The simplest form of synchronization is a mutually exclusive (mutex) lock.
- Only one thread at a time can acquire a mutex lock, so they can be placed around a data structure to ensure that the data structure is modified by only one thread at a time.

```
int counter;

mutex_lock mutex;

void Increment()
{
    acquire( &mutex );
    counter++;
    release( &mutex );
}

void Decrement()
{
    acquire( &mutex );
    counter--;
    release( &mutex );
}
```

- In the example, the two routines Increment() and Decrement() will either increment or decrement the variable counter.
- To modify the variable, a thread has to first acquire the mutex lock. Only one thread at a time can do this; all the other threads that want to acquire the lock need to wait until the thread holding the lock releases it.
- The two schedules utilize the equivalent mutex; therefore, just each string in turn can either addition or decrement the variable counter.
- On the off chance that different strings are endeavoring to procure the equivalent mutex simultaneously, at that point just one string will succeed, and different strings should pause. This circumstance is known as a fought mutex.
- The district of code between the obtaining and arrival of a mutex lock is known as a basic area, or basic locale. Code right now be executed by just each string in turn. As an example of a critical section, imagine that an operating system does not have an implementation of malloc() that is thread-safe, or safe for multiple threads to call at the same time.
- One way to fix this is to place the call to malloc() in a critical section by surrounding it with a mutex lock.


```
void * threadSafeMalloc( size_t size )
{
    acquire( &mallocMutex );
    void * memory = malloc( size );
    release( &mallocMutex );
    return memory;
}
```

- On the off chance that all the calls to malloc() are supplanted with the threadSafeMalloc() call, at that point just each string in turn can be in the first malloc() code, and the calls to malloc() become string safe.
- Threads square on the off chance that they endeavor to secure a mutex lock that is as of now held by another string.
- Blocking implies that the strings are sent to rest either promptly or after a couple of ineffective endeavors to gain the mutex. One issue with this methodology is that it can serialize a program.
- On the off chance that numerous strings all the while call threadSafeMalloc(), just each string in turn will gain ground.
- This causes the multithreaded program to have just a solitary executing string, which prevents the program from exploiting various centers

Spin Locks

- Spin locks are basically mutex locks. The distinction between a mutex lock and a Spin lock is that a Thread holding on to secure a Spin lock will continue attempting to gain the lock without dozing.
- The benefit of utilizing Spin locks is that they will get the lock when it is discharged, while a mutex lock should be woken by the working framework before it can get the lock.
- The hindrance is that a turn lock will Spin on a virtual CPU cornering that asset.
- In correlation, a mutex lock will rest and free the virtual CPU for another string to utilize Semaphores
- Semaphores are counters that can be either incremented or decremented. An example might be a buffer that has a fixed size.
- Every time an element is added to a buffer, the number of available positions is decreased. Every time an element is removed, the number available is increased.
- Semaphores can also be used to mimic mutexes; if there is only one element in the semaphore, then it can be either acquired or available, exactly as a mutex can be either locked or unlocked.
- Semaphores will also signal or wake up threads that are waiting on them to use available resources; hence, they can be used for signaling between threads.
- For example, a thread might set a semaphore once it has completed some initialization. Other threads could wait on the semaphore and be signaled to start work once the initialization is complete.

Readers-Writer Locks :

- Data races are a worry just when shared information is changed. Various threads perusing the common information don't present an issue.
- Peruse just information doesn't, in this way, need assurance with a lock. In any case, once in a while information that is ordinarily perused just should be refreshed.
- A readerswriter lock (or different peruser lock) permits numerous threads to peruse the common information however would then be able to bolt the perusers strings out to permit one string to secure an author lock to alter the information.

- An essayist can't get the compose lock until all the perusers have discharged their peruser locks. Thus, the locks will in general be one-sided toward journalists; when one is lined, the lock quits permitting further perusers to enter.
- This activity causes the quantity of perusers holding the lock to decrease and will in the end the essayist to get selective access

```
int readData( int cell1, int cell2 )
{
    acquireReaderLock( &lock );
    int result = data[cell1] + data[cell2];
    releaseReaderLock( &lock );
    return result;
}

void writeData( int cell1, int cell2, int value )
{
    acquireWriterLock( &lock );
    data[cell1] += value;
    data[cell2] -= value;
    releaseWriterLock( &lock );
}
```

Barriers :

- There are circumstances where various strings have to all finish their work before any of the strings can begin the following assignment.
- For instance, assume various strings figure the qualities put away in a network.
- The variable complete should be determined utilizing the qualities put away in the network.
- A boundary can be utilized to guarantee that all the strings total their calculation of the lattice before the variable absolute is determined. The following code clarifies this idea

```
Compute_values_held_in_matrix();
Barrier();
total = Calculate_value_from_matrix();
```

- The variable total can be computed only when all threads have reached the barrier.
- This avoids the situation where one of the threads is still completing its computations while the other threads start using the results of the calculations.
- Notice that another barrier could well be needed after the computation of the value for total if that value is then used in further calculations.

The following code explains multiple barriers

```
Compute_values_held_in_matrix();
Barrier();
total = Calculate_value_from_matrix();
Barrier();
Perform_next_calculation( total );
```

Deadlocks and Live locks.

The fundamental ways to share access to resources between threads :

1. Deadlock
 2. Livelocks
- The deadlock, where two or more threads that they need are held by different threads. It is most effortless to clarify this with a model.
 - Assume two threads need to get mutex locks An and B to finish some errand.
 - On the off chance that thread 1 has just procured lock An and string 2 has just gained lock B, at that point A can't gain forward ground since it is hanging tight for lock B, and thread 2 can't gain ground

since it is sitting tight for lock A. The two threads are halted.

Thread 1	Thread 2
<pre>void update1() { acquire(A); acquire(B); <<< Thread 1 waits here variable1++; release(B); release(A); }</pre>	<pre>void update2() { acquire(B); acquire(A); <<< Thread 2 waits here variable1++; release(B); release(A); }</pre>

- The best way to avoid deadlocks is to ensure that threads always acquire the locks in the same order.
- So if thread 2 acquired the locks in the order A and then B, it would stall while waiting for lock A without having first acquired lock B.
- This would enable thread 1 to acquire B and then eventually release both locks, allowing thread 2 to make progress.
- A livelock traps threads in an unending loop releasing and acquiring locks. Livelocks can be caused by code to back out of deadlocks.
- If the thread cannot obtain the second lock it requires, it releases the lock that it already holds. The two routines update1() and update2() each have an outer loop.

<pre>int done=0; while (!done) { acquire(A); if (canAcquire(B)) { variable1++; release(B); release(A); done=1; } else { release(A); } }</pre>	<pre>int done=0; while (!done) { acquire(B); if (canAcquire(A)) { variable2++; release(A); release(B); done=1; } else { release(B); } }</pre>
---	---

- Routine update1() acquires lock A and then attempts to acquire lock B, whereas update2() does this in the opposite order.
- This is a classic deadlock opportunity, and to avoid it, the developer has written some code that causes the held lock to be released if it is not possible to acquire the second lock.
- The routine canAcquire(), in this example, returns immediately either having acquired the lock or having failed to acquire the lock.

Communication Between Threads and Processes - Condition Variables :

- Condition variables communicate readiness between threads by enabling a thread to be woken up when a condition becomes true.

- Without condition variables, the waiting thread would have to use some form of polling to check whether the condition had become true.
- For example, the producer consumer model can be implemented using condition variables. Suppose an application has one producer thread and one consumer thread.
- The producer adds data onto a queue, and the consumer removes data from the queue.
- If there is no data on the queue, then the consumer needs to sleep until it is signaled that an item of data has been placed on the queue.

```
Acquire Mutex();
Add Item to Queue();
If ( Only One Item on Queue )
{
Signal Conditions Met();
}
Release Mutex();
```

- The producer thread needs to signal a waiting consumer thread only if the queue was empty and it has just added a new item into that queue.
- If there were multiple items already on the queue, then the consumer thread must be busy processing those items and cannot be sleeping.

```
Acquire Mutex();
Repeat Item = 0;
If ( No Items on Queue() )
{
Wait on Condition Variable();
}
If (Item on Queue())
{
Item = remove from Queue();
}
Until ( Item != 0 );
Release Mutex();
```

- The producer thread can use two types of wake-up calls: Either it can wake up a single thread or it can broadcast to all waiting threads.

Consumer Thread Code with Potential Lost Wake-Up Problem

```
Repeat
Item = 0;
If ( No Items on Queue() )
{
Acquire Mutex();
Wait on Condition Variable();
Release Mutex();
```

```
}  
Acquire Mutex();  
If ( Item on Queue() )  
{  
Item = remove from Queue();  
}  
Release Mutex();  
Until ( Item!=0 );
```

Signals and Events

- Signals are a UNIX instrument where one procedure can impart a sign to another procedure and have a handler in the accepting procedure play out some errand upon the receipt of the message.
- Numerous highlights of UNIX are actualized utilizing signals. Halting a showing application to squeezing.
- Windows has a comparable instrument for occasions. The treatment of console presses and mouse moves are performed through the occasion system.
- Squeezing one of the catches on the mouse will make a tick occasion be sent to the objective window.
- Signs and occasions are truly upgraded for sending restricted or no information alongside the sign, and all things considered they are likely not the best component for correspondence when contrasted with different alternatives Installing and Using a Signal Handler

```
void signalHandler(void *signal)  
{ ... }  
int main()  
{  
installHandler( SIGNAL, signalHandler );  
sendSignal( SIGNAL );  
}
```

Message Queues

- A message queue is a structure that can be shared between multiple processes.
- Messages can be placed into the queue and will be removed in the same order in which they were added.
- Constructing a message queue looks rather like constructing a shared memory segment. The first thing needed is a descriptor, typically the location of a file in the file system.
- This descriptor can either be used to create the message queue or be used to attach to an existing message queue.
- Once the queue is configured, processes can place messages into it or remove messages from it. Once the queue is finished, it needs to be deleted.
- Creating and Placing Messages into a Queue ID = Open Message Queue Queue(Descriptor); Put Message in Queue(ID, Message); ... Close Message Queue(ID); Delete Message Queue(Description);

- Using the descriptor for an existing message queue enables two processes to communicate by sending and receiving messages through the queue.
- Opening a Queue and Receiving Messages ID=Open Message Queue ID(Descriptor); Message=Remove Message from Queue(ID); ... Close Message Queue(ID);

Named Pipes:

- UNIX uses pipes to pass data from one process to another. For example, the output from the command `ls`, which lists all the files in a directory, could be piped into the `wc` command, which counts the number of lines, words, and characters in the input.
- The combination of the two commands would be a count of the number of files in the directory. Named pipes provide a similar mechanism that can be controlled programmatically.
- Setting Up and Writing into a Pipe Make Pipe(Descriptor); ID = Open Pipe(Descriptor); Write Pipe(ID, Message, sizeof(Message)); ... Close Pipe(ID); Delete Pipe(Descriptor);
- Opening an Existing Pipe to Receive Messages ID=Open Pipe(Descriptor); Read Pipe(ID, buffer, sizeof(buffer)); ... Close Pipe(ID);

other approaches to Sharing Data Between Threads:

- There are several other approaches to sharing data. For example, data can be written to a file to be read by another process at a later point.
- This might be acceptable if the data needs to be stored persistently or if the data will be used at some later point.
- Still, writing to disk presents a long latency operation, which is not the best mechanism if the purpose is purely communication
- There is also operating system-specific approaches to sharing data between processes.
- Solaris doors allow one process to pass an item of data to another process and have the processed result returned.
- Doors are optimized for the round-trip and hence can be cheaper than using two different messages.

Evaluating Parallel Programs :

- For any of the parallel programming approaches described in the previous section, there are a number of metrics that can be used to evaluate its effectiveness.
- This section describes five of the most important metrics that will be used to evaluate parallel programming in this dissertation: performance, clarity, portability, generality, and a programmer's ability to reason about the implementation.

Performance :

- Performance is typically viewed as the bottom line in parallel computing.
- Since improved performance is often the primary motivation for using parallel computers, failing to achieve good performance reflects poorly on a language, library, or compiler.

Clarity :

- The importance of clarity is often brushed aside in favor of the all-consuming pursuit of performance. However, this is a mistake that should not be made.

- Clarity is perhaps the single most important factor that prevents more scientists and programmers from utilizing parallel computers today. Local-view libraries continue to be the predominant approach to parallel programming, yet their syntactic overheads are such that clarity is greatly compromised.
- This requires programmers to focus most of their attention on making the program

Generality :

- Generality simply refers to the ability of a parallel programming approach to express algorithms for varying types of problems.
- For example, a library which only supports matrix , multiplication operations is not very general, and would not be very helpful for writing a parallel quicksort algorithm.
- Conversely, a global-view functional language might make it

UNIT - III

OpenMP Execution Model – Memory Model – OpenMP Directives – Work-sharing Constructs - Library functions – Handling Data and Functional Parallelism – Handling Loops - Performance Considerations

Open specifications for Multi Processing via collaborative work between interested parties from the hardware and software industry, government and academia. OpenMP is an Application Program Interface (API) that may be used to explicitly direct multi-threaded, shared memory parallelism. API components: Compiler Directives, Runtime Library Routines, Environment Variables. OpenMP is a directive-based method to invoke parallel computations on share-memory multiprocessor

- OpenMP and Pthreads are both APIs for shared-memory programming, they have many fundamental differences.
- Pthreads requires that the programmer explicitly specify the behavior of each thread.
- OpenMP, on the other hand, sometimes allows the programmer to simply state that a block of code should be executed in parallel, and the precise determination of the tasks and which thread should execute them is left to the compiler and the run-time system.
- This suggests a further difference between OpenMP and Pthreads, that is, that Pthreads (like MPI) is a library of functions that can be linked to a C program, so any Pthreads program can be used with any C compiler, provided the system has a Pthreads library. OpenMP, on the other hand, requires compiler support for some operations, and hence it's entirely possible that you may run across a C compiler that can't compile OpenMP programs into parallel programs.
- Critical directives insure that only one thread at a time can execute the structured block. If multiple threads try to execute the code in the critical section, all but one of them will block before the critical section. As threads finish the critical section, other threads will be unblocked and enter the code.
- Named critical directives can be used in programs having different critical sections that can be executed concurrently. Multiple threads trying to execute code in critical section(s) with the same name will be handled in the same way as multiple threads trying to execute an unnamed critical section. However, threads entering critical sections with different names can execute concurrently.

OpenMP uses the fork-join model of parallel execution.

- o All OpenMP programs begin with a single master thread.
- o The master thread executes sequentially until a parallel region is encountered, when it creates a team of parallel threads (FORK).

- o When the team threads complete the parallel region, they synchronize and terminate, leaving only the master thread that executes sequentially (JOIN).

Simple Open MP hello world program:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void Hello(void); /* Thread function */

int main(int argc, char* argv[]) {
    /* Get number of threads from command line */
    int thread_count = strtol(argv[1], NULL, 10);

    #pragma omp parallel num_threads(thread_count)
    Hello();

    return 0;
} /* main */

void Hello(void) {
    int my_rank = omp_get_thread_num();
    int thread_count = omp_get_num_threads();

    printf("Hello from thread %d of %d\n", my_rank, thread_count);
} /* Hello */
```

5.1.1 Compiling and running OpenMP programs

To compile this with gcc we need to include the

```
$ gcc -g -Wall -fopenmp -o omp_hello omp_hello.c
```

To run the program, we specify the number of threads on the command line.

```
$ ./omp_hello 4
```

The following will be the output

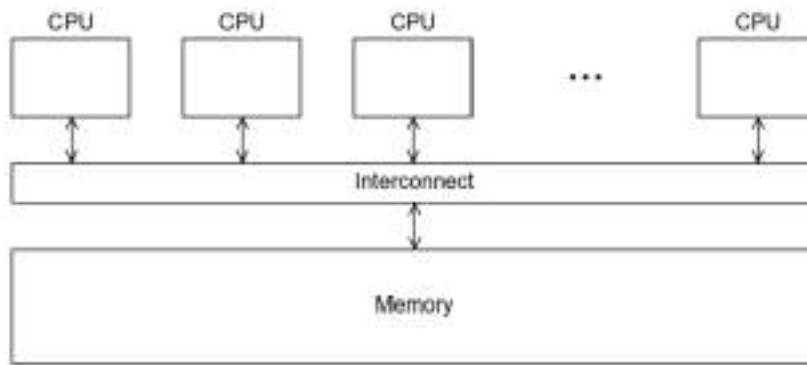
Hello from thread	0	of	4
Hello from thread	1	of	4
Hello from thread	2	of	4
Hello from thread	3	of	4

Open MP Memory Model :

- The OpenMP API gives a casual consistency, shared-memory model. All OpenMP strings approach a spot to store and to recover factors, called the memory.
- Likewise, each string is permitted to have its own impermanent perspective on the memory. The impermanent perspective on memory for each string is certainly not a necessary piece of the OpenMP memory model, yet can speak to any sort of mediating structure,
- for example, machine registers, reserve, or other neighborhood stockpiling, between the string and the memory.
- The brief perspective on memory permits the string to store factors and in this manner to abstain from going to memory for each reference to a variable.
- Each string likewise approaches another kind of memory that must not be gotten to by different strings, called thread private memory.
- The minimum size at which a memory update may likewise peruse and compose back nearby factors that are a piece of another variable (as exhibit or structure components) is usage characterized yet is no bigger than required by the base language.
- A solitary access to a variable might be actualized with various burden or store guidelines and, consequently, isn't destined to be nuclear regarding different gets to a similar variable.
- Gets to factors littler than the usage characterized least size or to C or C++ bit-fields might be executed by perusing, adjusting, and modifying a bigger unit of memory, and may consequently meddle with updates of factors or fields in a similar unit of memory.
- In the event that various strings compose without synchronization to a similar memory unit, including cases because of atomicity contemplations as depicted above, at that point an information race happens.
- Correspondingly, if in any event one string peruses from a memory unit and at any rate one string composes without synchronization to that equivalent memory unit,

Shared-Memory Model :

- The shared-memory model is an abstraction of the generic centralized multiprocessor.
- The underlying hardware is assumed to be a collection of processors, each with access to the same shared memory.
- Because they have access to the same memory locations, processors can interact and synchronize with each other through shared variables.



- A key difference, at that point, between the mutual memory model and the message passing model is that in the message-passing model all procedures regularly stay dynamic all through the execution of the program, though in the common memory model the quantity of dynamic strings is one at the program's beginning and finish and may change dynamically all through the execution of the program.
- To see, a consecutive program as an uncommon instance of a mutual memory equal program: it is just one with no fork/participates in it. Equal shared-memory programs run from those with just a solitary fork/join around a solitary circle to those in which the majority of the code sections are executed in equal.
- Henceforth the common memory model backings gradual parallelization, the way toward changing a successive program into an equal program each square of code in turn.
- The capacity of the common memory model to help steady parallelization is probably the best preferred position over the message-passing model. consider each square sick turn starting with the most tedious, parallelize each square manageable to resemble execution. what's more, stop when the required to accomplish further execution enhancements isn't justified.
- Consider, in contrast, message-passing programs. They have no shared memory 0 hold variables, and the parallel processes are active throughout the execution of the program. Transforming a sequential program into a parallel program is not incremental

Device Data Environments

- When an OpenMP program starts, every gadget has an underlying gadget information condition. The underlying gadget information condition for the host gadget is the information condition related with the underlying initial task region.
- Directives that accept data-mapping attribute clauses determine how an original variable is mapped to a corresponding variable in a device data environment.
- The original variable is the variable with the same name that exists in the data environment of the task that encounters the directive.

- If a comparing variable is available in the encasing gadget information condition, the new gadget information condition acquires the relating variable from the encasing gadget information condition.
- If a corresponding variable is not present in the enclosing device data environment, a new corresponding variable (of the same type and size) is created in the new device data environment.
- In the latter case, the initial value of the new corresponding variable is determined from the clauses and the data environment of the encountering thread.
- The corresponding variable in the device data environment may share storage with the original variable. Writes to the corresponding variable may alter the value of the original variable.
- When a task executes in the context of a device data environment, references to the original variable refer to the corresponding variable in the device data environment.
- The relationship between the value of the original variable and the initial or final value of the corresponding variable depends on the map-type. Details of this issue, as well as other issues with mapping a variable
- The unique variable in an information domain and the comparing variable(s) in at least one gadget information conditions may share stockpiling. Without mediating synchronization data races can happen.

The Flush Operation

- The memory model has relaxed-consistency because a thread's temporary view of memory is not required to be consistent with memory at all times.
- A value written to a variable can remain in the thread's temporary view until it is forced to memory at a later time. Likewise, a read from a variable may retrieve the value from the thread's temporary view, unless it is forced to read from memory.
- The OpenMP flush operation enforces consistency between the temporary view and memory.
- The flush operation is applied to a set of variables called the flush-set. The flush operation restricts reordering of memory operations that an implementation might otherwise do.

- Implementations must not reorder the code for a memory operation for a given variable, or the code for a flush operation for the variable, with respect to a flush operation that refers to the same variable.

OpenMP directives :

- OpenMP directives exploit shared memory parallelism by defining various types of parallel regions. Parallel regions can include both iterative and non-iterative segments of program code.
- Pragmas fall into these general categories:
 - ✓ Pragmas that let you define parallel regions in which work is done by threads in parallel (#pragma omp parallel). Most of the OpenMP directives either statically or dynamically bind to an enclosing parallel region.
 - ✓ Pragmas that let you define how work is distributed or shared across the threads in a parallel region (#pragma omp section, #pragma omp for, #pragma omp single, #pragma omp task).
 - ✓ Pragmas that let you control synchronization among threads (#pragma omp atomic, #pragma omp master, #pragma omp barrier, #pragma omp critical, #pragma omp flush, #pragma omp ordered) .
 - ✓ Pragmas that let you define the scope of data visibility across threads (#pragma omp threadprivate).
 - ✓ Pragmas for task synchronization (#pragma omp taskwait, #pragma omp barrier)

OpenMP compiler directives

- parallel, which goes before a square of code to be executed in equal by numerous strings for, which goes before a for circle with free iterations that might be separated among strings executing in equal
- parallel for, a combination of the parallel and for directives
- sections, which goes before a progression of hinders that might be executed in equal
- parallel sections, a combination of the parallel
- critical , which precedes a critical section
- single, which precedes a code block executed as a single thread

Four important OpenMP functions:

- **omp_get_num_procs**, which returns the number of CPUs in the multiprocessor on which this thread is executing
- **omp_get_num_threads**, which returns the number of threads active in the current parallel region

- **omp_get_thread_num**, which returns the thread identification number
- **omp_set_num_threads**, which allows you to fix the number of threads executing the parallel sections of code

Directive Format

The syntax of an OpenMP directive is formally specified by the grammar

```
#pragma omp directive-name [clause[ [,] clause]...] new-line
```

- Each directive starts with `#pragma omp`, to reduce the potential for conflict with other (non-OpenMP or vendor extensions to OpenMP) pragma directives with the same names.
- The remainder of the directive follows the conventions of the C and C++ standards for compiler directives. In particular, white space can be used before and after the `#`, and sometimes white space must be used to separate the words in a directive.
- Preprocessing tokens following the `#pragma omp` are subject to macro replacement.
- Directives are case-sensitive. The order in which clauses appear in directives is not significant. Clauses on directives may be repeated as needed, subject to the restrictions listed in the description of each clause.
- If variable-list appears in a clause, it must specify only variables. Only one directive-name can be specified per directive.

OpenMP fixed source form

The following formats for specifying directives are equivalent (the first line represents the position of the first 9 columns):

```
C23456789
!$OMP PARALLEL DO SHARED(A,B,C)

C$OMP PARALLEL DO
C$OMP+SHARED(A,B,C)

C$OMP PARALLELDOSHARED(A,B,C)
```

OpenMP free source form

The following formats for specifying directives are equivalent (the first line represents the position of the first 9 columns):

```
!23456789
  !$OMP PARALLEL DO &
    !$OMP SHARED(A,B,C)

!$OMP PARALLEL &
  !$OMP&DO SHARED(A,B,C)

!$OMP PARALLEL DO SHARED(A,B,C)
```

- One or more blanks or tabs must be used to separate adjacent keywords in directives
- Comments are allowed inside directives. Comments can appear on the same line as a directive.
- In free source form, the exclamation point initiates a comment; in fixed source form, it initiates a comment when it appears after column 6.
- Regardless of form, the comment extends to the end of the source line and is ignored. If the first nonblank character after the initial prefix (or after a continuation directive line in fixed source form) is an exclamation point, the line is ignored.

Conditional Compilation

- Fortran statements can be compiled conditionally as long as they are preceded by one of the following conditional compilation prefixes: !\$, C\$, or *\$.
- The prefix must be followed by a Fortran statement on the same line.
- During compilation, the prefix is replaced by two spaces, and the rest of the line is treated as a normal Fortran statement.
- The program must be compiled with the -mp option in order for the compiler to honor statements preceded by conditional compilation prefixes; without the mp command line option, statements preceded by conditional compilation prefixes are treated as comments.
- First define the _OPENMP symbol to be used for conditional compilation.

- This symbol is defined during OpenMP compilation to have the decimal value YYYYMM where YYYY and MM are the year and month designators of the version of the OpenMP Fortran API is supported.
- The !\$ prefix is accepted when compiling either fixed source form files or free source form files. The C\$ and *\$ prefixes are accepted only when compiling fixed source form. The source form you are using also dictates the following:
- In fixed source form, the prefixes must start in column one and appear as a single word with no intervening white space.
- Fortran fixed form line length, case sensitivity, white space, continuation, and column rules apply to the line. Initial lines must have a space or zero in column six, and continuation lines must have a character other than a space or zero in column six.
- In free source form, the !\$ prefix can appear in any column as long as it is preceded only by white space.
- It must appear as a single word with no intervening white space. Fortran free source form line length, case sensitivity, white space, and continuation rules apply to the line. Initial lines must have a space after the prefix.
- Continued lines must have an ampersand as the last nonblank character on the line prior to any comment appearing on the conditionally compiled line.
- Continuation lines can have an ampersand after the prefix, with optional white space before and after the ampersand.

Data dependences :

- If a for loop fails to satisfy one of the rules outlined in the preceding section, the compiler will simply reject it.
- For example, suppose we try to compile a program with the following linear search function:

```
int Linear_search(int key, int A[], int n) {  
    int i;  
    /* thread_count is global */  
    #pragma omp parallel for num_threads(thread_count)  
    for (i = 0; i < n; i++)  
        if (A[i] == key) return i;  
    return -1; /* key not in list */  
}
```

The gcc compiler reports:

Line 6: error: invalid exit from OpenMP structured block

- A more insidious problem occurs in loops in which the computation in one iteration depends on the results of one or more previous iterations. As an example, consider the following code, which computes the first n fibonacci numbers:

```
fibonacci[0] = fibonacci[1] = 1;
for (i = 2; i < n; i++)
    fibonacci[i] = fibonacci[i-1] + fibonacci[i-2];
```

Although we may be suspicious that something isn't quite right, let's try parallelizing the for loop with a parallel for directive:

```
fibonacci[0] = fibonacci[1] = 1;
#pragma omp parallel for num_threads(thread_count)
for (i = 2; i < n; i++)
    fibonacci[i] = fibonacci[i-1] + fibonacci[i-2];
```

The compiler will create an executable without complaint. However, if we try running it with more than one thread, we may find that the results are, at best, unpredictable. For example, on one of our systems if we try using two threads to compute the first 10 Fibonacci numbers, we sometimes get

1	1	2	3	5	8	13	21	34	55
---	---	---	---	---	---	----	----	----	----

How the Per-Data Environment ICVs Work

- At the point when an errand develop or equal build is experienced, the produced task(s) acquire the estimations of the information condition checked ICVs from the creating undertaking's ICV esteems.
- At the point when an assignment build is experienced, the produced task acquires the estimation of nthreads-var from the creating errand's nthreads-var esteem.
- At the point when an equal build is experienced, and the producing undertaking's nthreads-var list contains a solitary ICV Scope component, the created task(s) acquire that rundown as the estimation of nthreads-var.

- At the point when an equal develop is experienced, and the producing undertaking's nthreads-var list contains numerous components, the created task(s) acquire the estimation of nthreads-var as the rundown got by erasure of the principal component from the creating errand's nthreads-var esteem.
- The quandary var ICV is dealt with similarly as the nthreads-var ICV.
- At the point when a gadget build is experienced, the new gadget information condition acquires the estimations of the information condition perused ICVs from the encasing gadget information condition of the gadget that will execute the district.
- On the off chance that a groups develop with a thread_limit proviso is experienced, as far as possible var ICV of the new gadget information condition isn't acquired yet rather is set to a worth that is not exactly or equivalent to the worth determined in the statement.
- While experiencing a circle worksharing locale with schedule(runtime), all verifiable undertaking districts that comprise the coupling equal area must have a similar incentive for run-sched-var in their information surroundings.
- Something else, the conduct is unknownWork-sharing Constructs
- This is one of the category of Open MP language Extensions, it distribute work among threads using do/parallel and do/section directives

SPMD vs. worksharing

- A parallel construct by itself creates an SPMD or “Single Program Multiple Data” program i.e., each thread redundantly executes the same code.
- To split up pathways through the code between threads within a team. This is called worksharing.
#pragma omp c
 - Thread creation constructPARALLEL / parallel
- Original process (master thread) forked additional threads to run code enclosed in the parallel construct.
- Thread ID for master thread is 0. Work-sharing constructs
 - DO / for split up loop iterations among the threads in a parallel region
 - SECTIONS / sections
 1. divide consecutive but independent section(s) of code block amongst the threads
 2. barrier implied at the end unless the NOWAIT/nowait clause if used.
- code block to be executed by 1 thread
- barrier implied at the end unless the NOWAIT/nowait clause if used

- MASTER / master : code block to be executed by master thread only (no barrier on other threads implied)

Workshare :

- execution of structured block is divided into separate units of work, each to be executed once
- Structured block must consist only
 1. array or scalar assignment
 2. FORALL and WHERE statements
 3. FORALL and WHERE constructs
 4. atomic, critical or parallel constructs

Data scoping clauses

1. SHARED / shared : data are visible and accessible by all threads simultaneously. All variables in work-sharing region are shared by default, except the loop iteration.
2. PRIVATE / private : data is private to each thread. A private variable is not initialized. Loop iteration counter in work-sharing region is private.
3. DEFAULT / default : default data scoping in the work-sharing region (shared / private / none)

Synchronization clauses

1. CRITICAL / critical : enclosed code block to be executed by all threads, but only one thread at a time
2. ATOMIC / atomic : a mini-critical section specifying that a memory location must be updated atomically
3. ORDERED / ordered : iteration of enclosed code block is executed in same order as sequential
4. BARRIER / barrier : synchronizes all threads at the barrier. The barrier synchronization is implied at end of work-sharing construct.
5. NOWAIT / nowait : thread completing code block execution can proceed, at end of work-sharing construct where barrier is implied by default

Scheduling clauses

- SCHEDULE(type, [chunk]) / schedule(type, [chunk])
 1. STATIC / static : loop iterations are divided into chunk size and statically assigned to the threads. The iterations are evenly divided contiguously if no chunk is specified.

2. DYNAMIC / dynamic : loop iterations are divided into chunk size and dynamically scheduled amongst the threads. The default chunk size is 1.
 3. GUIDED / guided : For a chunk_size of 1, the size of each chunk is proportional to the number of unassigned iterations divided by the number of threads in the team, decreasing to
 4. RUNTIME /runtime : scheduling decision deferred until runtime by environment variable OMP_SECHULE
- AUTO / auto : scheduling decision delegated to the compiler and/or runtime system

The loop worksharing Constructs

The loop worksharing construct splits up loop iterations among the threads in a team

```
#pragma omp parallel
{
#pragma omp for
for (I=0;I<N;I++){ // The variable I is made “private” to each thread by default. We
                    could do this explicitly with a “private(I)” clause //
    NEAT_STUFF(I);
}
}
```

Library functions

- OpenMP is a library for parallel programming in the SMP (symmetric multi-processors, or shared-memory processors) model.
- When programming with OpenMP, all threads share memory and data. OpenMP supports C, C++ and Fortran.
- The OpenMP functions are included in a header file called omp.h .
- OpenMP program structure: An OpenMP program has sections that are sequential and sections that are parallel.
- In general an OpenMP program starts with a sequential section in which it sets up the environment, initializes the variables, and so on.
- When run, an OpenMP program will use one thread (in the sequential sections), and several threads (in the parallel sections).
- There is one thread that runs from the beginning to the end, and it's called the *master thread*.
- The parallel sections of the program will cause additional threads to fork. These are called the *slave threads*.

- A section of code that is to be executed in parallel is marked by a special directive (omp pragma). When the execution reaches a parallel section (marked by omp pragma), this directive will cause slave threads to form.
- Each string executes the equal segment of the code freely. At the point when a string completes, it joins the ace. At the point when all strings finish, the ace proceeds with code following the equal area.
- Each thread has an ID attached to it that can be obtained using a runtime library function (called `omp_get_thread_num()`). The ID of the master thread is 0.
- First we have to Modify/Check the number of threads
 - `omp_set_num_threads()`
 - `omp_get_num_threads()`
 - `omp_get_thread_num()`
 - `omp_get_max_threads()`

The active parallel region is found as `omp_in_parallel()`

The system to dynamically vary the number of threads from one parallel construct to another

- `omp_set_dynamic,`
- `omp_get_dynamic();`

To find number of processors in a system we use `omp_num_procs()` .Set the default number of threads to use.

`OMP_NUM_THREADS int_literal` sets the number of threads to use in a team

- This subroutine sets the quantity of strings that will be utilized in the following equal area. The dynamic strings component adjusts the impact of this daily schedule.
- Whenever empowered, indicates the most extreme number of strings that can be utilized for any equal area.
- Whenever impaired, indicates definite number of strings to use until next call to this everyday practice.
- This routine must be called from the sequential parts of the code. This bring has priority over the `OMP_NUM_THREADS` condition variable.`void omp_set_num_threads(int num_threads)`

Execution Environment Routine.

1. `omp_set_num_threads` :

- The `omp_set_num_threads` routine affects the number of threads to be used for subsequent parallel regions that do not specify a `num_threads` clause, by setting the value of the first element of the `nthreads-var` ICV of the current task.

Format

```
void omp_set_num_threads(int num_threads);  
subroutine omp_set_num_threads(num_threads)
```

2. Constraints on Arguments

- The value of the argument passed to this routine must evaluate to a positive integer, or else the behavior of this routine is implementation defined.
- The binding task set for an `omp_set_num_threads` region is the generating task.
- The effect of this routine is to set the value of the first element of the `nthreads-var` ICV of the current task to the value specified in the argument.

`omp_get_num_threads`

The `omp_get_num_threads` routine returns the number of threads in the current team.

```
int omp_get_num_threads(void);
```

- The binding region for an `omp_get_num_threads` region is the innermost enclosing parallel region.
 - The `omp_get_num_threads` routine returns the number of threads in the team executing the parallel region to which the routine region binds. If called from the sequential part of a program, this routine returns
- `omp_get_max_threads`
- The `omp_get_max_threads` routine returns an upper bound on the number of threads that could be used to form a new team if a parallel construct without a `num_threads` clause were encountered after execution returns from this routine.
 - The binding task set for an `omp_get_max_threads` region is the generating task.
 - The value returned by `omp_get_max_threads` is the value of the first element of the `nthreads-var` ICV of the current task. This value is also an upper bound on the number of threads that could be used to form a new team if a parallel region without a `num_threads` clause were encountered after execution returns from this routine.

`omp_get_thread_num`

- The `omp_get_thread_num` routine returns the thread number, within the current team, of the calling thread.

```
int omp_get_thread_num(void);
```

- The binding thread set for an `omp_get_thread_num` region is the current team. The binding region for an `omp_get_thread_num` region is the innermost enclosing parallel region.

- The `omp_get_thread_num` routine returns the thread number of the calling thread, within the team executing the parallel region to which the routine region binds.
- The thread number is an integer between 0 and one less than the value returned by `omp_get_num_threads`, inclusive. The thread number of the master thread of the team is 0. The routine returns 0 if it is called from the sequential part of a program.

`omp_in_parallel`

- The `omp_in_parallel` routine returns true if the active-levels-var ICV is greater than zero; otherwise, it returns false. `int omp_in_parallel(void);`
- The binding task set for an `omp_in_parallel` region is the generating task.
- The effect of the `omp_in_parallel` routine is to return true if the current task is enclosed by an active parallel region, and the parallel region is enclosed by the outermost initial task region on the device; otherwise it returns false.

Handling Data and Functional Parallelism

- First think about a calculation to process a connected rundown of undertakings. Think about a comparable calculation when planned an answer for the report grouping..
- In that design, assumed a message-passing model. Because that model has no shared memory, to gave a single process, which are called the manager, responsibility for maintaining the entire list of tasks.
- Worker tasks sent messages to the manager when they were ready to process another task.
- In contrase, the shared-memory model allows every thread to access the same “to=do” list. So there is no need for a separate manager thread.
- The following code segments are part of a program that processes work stored in a singly linked to-do list.

```
Int main(int argc,argv[])
{
Struct job_struct job_ptr;
Struct task_struct task_ptr;
---
Task_ptr=get_next_task(&job_ptr);
While(task_ptr!=NULL){
Complete_task(task_ptr);
Task_ptr=get_next_task(&job_ptr);
}
```

```
---  
}  
Char get-next_task(struct job_struct job_ptr){  
Struct task_struct answer;  
If(job_ptr==NULL) answer = Null;  
Else{  
Answer=(job_ptr)->task;  
Job_ptr=(job_ptr)-> next;  
}  
Return answer;  
}
```

An algorithm to process a linked list of tasks. The shared-memory model allows every thread to access the same “to-do” list, so there is no need for a separate manager thread.

The following code segments are part of a program that processes work stored in a singly linked to-do list.

To ensure that no two threads take the same task from the list. It is important to execute function `get_next_task` automatically.

Handling Loops

- A compiler order in C or C++ is called pragma. The word pragma is short for sober minded data.
- A pragma is an approach to impart data to the compiler.
- The data is superfluous as in the compiler may disregard the data and still produce a right article program.
- A pragma begin with the # character. A pragma in C or C++ has this syntax:

#pragma omp <rest of pragma>

- The first pragma we are going to consider is the parallel for pragma. The simplest for, of the parallel for pragma is :

#pragma omp parallel for

- Putting this line immediately before the for loop instructs the compiler to try to parallelize the loop:

#pragma omp parallel for

For (i=first; i<size; i+=prime) marked[i]=1;

- The for loop must not contain statements that allow the loop to be exited prematurely.
- Examples include the break statement, return statement, exit statement, and go to statement to labels outside the loop.

- The continue statement is allowed, however, because its execution does not affect the number of loop iterations.

```
for ( index = start ; index < end ; index++  
      index <= end ; ++index  
      index >= end ; index--  
      index > end ; --index )  
    index += incr  
    index -= incr  
    index = index + incr  
    index = incr + index  
    index = index - incr
```

- During parallel execution of the for circle, the ace string makes extra strings, and all strings cooperate to cover the emphases of the circle.
- Each string has its own execution setting; a location space containing the entirety of the factors the string may get to.
- The execution setting incorporate static factors, progressively assigned information structures in the pile, and factors on the run-time stack.
- The execution context includes its own additional fun-time stack, where the frames for functions it invokes are kept.
- Other variables may either be shared or private. A shared variables has the same address in the execution context of every thread.
- All threads have access to shared variables. A private variable has a different address in the execution context of every thread.
- A thread can access its own private variables, but cannot access the private variable of another thread.
- In the case of the parallel for pragma, variables are by default shared, with the exception that the loop index variables its private.

Conditionally Executing Loops

- If a loop does not have enough iterations , the time spent forking and joining threads may exceed the time saved by dividing the loop iteration among multiple threads.

- Consider, for example the parallel implementation of the rectangle rule examined earlier.

Area = 0.0

```
#pragma omp parallel for private(x) reduction(+:area)
```

```
For(i=0;i<n;i++)
```

```
{
```

```
X=(i+0.5)/n;
```

```
Area+=4.0 / (1.0 + x* x);
```

```
}
```

```
Pi = area /n;
```

- The following figure reveals the average execution time of this program segment on a sun enterprise server 4000, for various values of n and various numbers of threads.
- When n is 100, the sequential execution time is so small that adding threads only increased overall execution time.
- When n is 100,000 the parallel program executing on four threads achieves a speedup of 3.16 over the sequential program.
- If clause gives us the ability to direct the compiler to insert code that determines at run-time whether the loop should be executed in parallel or.

The clause has this syntax :

```
if(<scalar expression>)
```

if the scalar expression evaluated to true, the loop will be executed in parallel. Otherwise it will be executed serially.

Finding loop-carried dependences :

- Perhaps the first thing to observe is that when we're attempting to use a parallel for directive, we only need to worry about loop-carried dependences.
- We don't need to worry about more general data dependences. For example, in the loop

```
for (i = 0; i < n; i++) {  
    x[i] = a + i*h;  
    y[i] = exp(x[i]);  
}
```

- There is a data dependence between Lines 2 and 3. However, there is no problem with the parallelization

```
# pragma omp parallel for num_threads(thread_count)
for (i = 0; i < n; i++) {
    x[i] = a + i*h;
    y[i] = exp(x[i]);
}
```

Odd-even transposition sort :

- Odd-even transposition sort is a sorting algorithm that's similar to bubble sort, but that has considerably more opportunities for parallelism. that serial odd-even transposition sort can be implemented as follows:

```
for (phase = 0; phase < n; phase++)
    if (phase % 2 == 0)
        for (i = 1; i < n; i += 2)
            if (a[i-1] > a[i]) Swap(&a[i-1], &a[i]);
    else
        for (i = 1; i < n-1; i += 2)
            if (a[i] > a[i+1]) Swap(&a[i], &a[i+1]);
```

- The list a stores n ints, and the algorithm sorts them into increasing order. During an “even phase” (phase % 2 == 0), each odd-subscripted element, a[i], is compared to the element to its “left,” a[i-1], and if they’re out of order, they’re swapped.
- During an “odd” phase, each odd-subscripted element is compared to the element to its right, and if they’re out of order, they’re swapped.
- A theorem guarantees that after n phases, the list will be sorted.

Scheduling Loops:

- In some-loops the time needed to execute different loop iterations varies considerable. For example , consider the following doubly nested loop that initializes an upper triangular matrix:

```
for(i=0; i<n; i++)
{
    for(j=1; j<n; j++)
        a[i][j]=alpha_omega(a[i][j]);
}
```

- Assuming there are no data dependences among iterations, prefer to execute the outermost loop . in parallel in order to minimize fork/join overhead.

- If every call to function `alpha_omega` takes the same amount of time, then the first iteration of the outermost loop (when `I` equals 0) requires n times more work than the last iteration (when `I` equals $n-1$). Inverting the two loops will not remedy the imbalance.
- Suppose these n iterations are being executed on t threads. If each thread is assigned a contiguous block of either $\lceil n/t \rceil$ or $\lfloor n/t \rfloor$ threads, the parallel loop execution will have poor efficiency, because some threads will complete their share of the iterations much faster than others.

Performance Considerations

The following are some general techniques for improving performance of OpenMP applications.

- Minimize synchronization.
 - Avoid or minimize the use of `BARRIER`, `CRITICAL` sections, `ORDERED` regions, and locks.
 - Use the `NOWAIT` clause where possible to eliminate redundant or unnecessary barriers. For example, there is always an implied barrier at the end of a parallel region. Adding `NOWAIT` to a final `DO` in the region eliminates one redundant barrier.
 - Use named `CRITICAL` sections for fine-grained locking.
 - Use explicit `FLUSH` with care. Flushes can cause data cache restores to memory, and subsequent data accesses may require reloads from memory, all of which decrease efficiency.
- As a matter of course, idle threads will be taken care of after a specific break period. It may be the case that the default break period isn't adequate for your application, making the threads rest too early or past the point of no return. The `SUNW_MP_THR_IDLE` condition variable can be utilized to abrogate the default break period, even up to where the idle threads will never be taken care of and stay dynamic constantly.
- Parallelize at the highest level possible, such as outer `DO/FOR` loops. Enclose multiple loops in one parallel region. In general, make parallel regions as large as possible to reduce parallelization overhead.
- Sometimes transforming a sequential for loop into a parallel for loop can actually increase a program's execution time.

Inverting loops :

Consider the following code segment :

```
for(i=1;i<m;i++)  
for(j=1;j<m;j++)  
a[i][j]=2*a[i-1][j];
```

- Draw a data dependence diagram to help us understand data dependencies in this code. The diagram appears in following figure.
- The two rows may not be updated simultaneously, because there are data dependences between rows. The columns may be updated simultaneously.
- This means the loop indexed by j may be executed in parallel, but not the loop indexed by i. Insert a `parallel for` pragma before the inner loop, the resulting parallel program will execute correctly, but it may not exhibit good performance, because it will require m-1 fork/join steps, one per iteration of the outer loop.

```
#pragma parallel for private(i)
for(j=0;j<n;j++)
{
    for(i=1;i<m;i++)
        a[i][j]=2*a[i-j][j];
```

- Only a single's fork/join step is required. The data dependences have not changed, the iterations of the loop indexed by j are still independent of each other. In this respect definitely improved the code.
- How code transformations affect the cache hit rate. Each thread is now working through-columns, of a rather than rows. Since C matrices are stored in row-major order, inverting loops may lower the cache hit rate, depending upon m,n, the number of activate threads, and the architecture of the underlying system.

UNIT IV - DISTRIBUTED MEMORY PROGRAMMING WITH MPI

SYLLABUS

MPI program execution – MPI constructs – libraries – MPI send and receive – Point-to-point and Collective communication – MPI derived datatypes – Performance evaluation

Identifying MPI processes

- Common practice to identify processes by nonnegative integer ranks.
- p processes are numbered $0, 1, 2, \dots, p-1$

```
1 #include <stdio.h>      For strlen          1
2 #include <string.h>     For MPI functions, etc 1
3 #include <mpi.h>
4
5 const int MAX_STRING = 100;
6
7 int main(void) {8      char      greeting[MAX_STRING];
9      int      comm_sz; Number of processes 1
10     int      my_rank; My process rank      1
11
12     MPI_Init (NULL, NULL);
13     MPI_Comm_size(MPI_COMM_WORLD, &comm_sz );
14     MPI_Comm_rank (MPI_COMM_WORLD, &my_rank );
15
16     if (my_rank != 0) {
17         sprintf(greeting, "Greetings from process %d of %d!",
18             my_rank, comm_sz);
19         MPI_Send(greeting, strlen(greeting)+1, MPI_CHAR, 0, 0,
20             MPI_COMM_WORLD);
21     } else {
22         printf("Greetings from process %d of %d!\n", my_rank, comm_sz);
23         for (int q = 1; q < comm_sz; q++) {
24             MPI_Recv(greeting, MAX_STRING, MPI_CHAR, q,
25                 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
26             printf("ss\n", greeting);
27         }
28     }
29
30     MPI_Finalize();
31     return 0;
32 } main 1
```


Compilation

www.rejinpaul.com

wrapper script to compile

 *source file*

```
mpicc -g -Wall -o mpi_hello mpi_hello.c
```

*produce
debugging
information*

*create this executable file name
(as opposed to default a.out)*

turns on all warnings

mpixec -n <number of processes> <executable>

mpixec -n 1 ./mpi_hello

L *run with 1 process*

mpixec -n 4 ./mpi_hello

L *run with 4 processes*

```
mpiexec -n 1 ./mpi_hello
```

Greetings from process 0 of 1 !

```
mpiexec -n 4 ./mpi_hello
```

Greetings from process 0 of 4!

Greetings from process 1 of 4!

Greetings from process 2 of 4!

Greetings from process 3 of 4!

MPI Programs

www.rejinpaul.com

- Written in C.
 - Has main.
 - Uses `stdio.h`, `string.h`, etc.
- Need to add **mpi.h** header file.
- Identifiers defined by MPI start with "MPI".
- First letter following underscore is uppercase.
 - For function names and MPI-defined types.
 - Helps to avoid confusion.

MPI Components

■ MPI_Init

- Tells MPI to do all the necessary setup.

```
int MPI_Init(  
    int* argc_p, // in/out  
    char** argv_p // in/out );
```

■ MPI_Finalize

- Tells MPI we're done, so clean up anything allocated for this program.

```
int MPI_Finalize(void);
```

Basic Outline

```
...  
#include <mpi.h>  
...  
int main(int argc, char+ argv[]) {  
    ...  
    /* No MPI calls before this */  
    MPI_Init(&argc, &argv);  
    ...  
    MPI_Finalize();  
    /* No MPI calls after this */  
    ...  
    return 0;  
}
```

Communicators

- A collection of processes that can send messages to each other.
- MPI_Init defines a communicator that consists of all the processes created when the program is started.
- Called **MPI_COMM_WORLD**.

Communicators



```
int MPI_Comm_size(
    MPI_Comm comm,
    int *comm_Sz_P);
```

number of processes in the communicator

```
int MPI_Comm_rank(
    MPI_Comm comm,
    int *my_rank_p);
```

my rank
(the process making this call)

- Single-Program Multiple-Data
- We compile one program.
- Process 0 does something different.
 - Receives messages and prints them while the other processes do the work.
- The **if-else** construct makes our program **SPMD**.

Communication

```
int MPI_Send(
```

```
void +
```

```
int
```

```
MPI_Data_type
```

```
int
```

```
int
```

```
MPI_Comm
```

```
msg_buf
```

```
msg_size
```

```
msg_type
```

```
dest
```

```
tag
```

```
communicator
```

```
/+
```

```
in
```

```
/,
```

```
/+
```

```
in
```

```
/,
```

```
/+
```

```
in
```

```
/,
```

```
/+
```

```
in
```

```
/,
```

```
/+
```

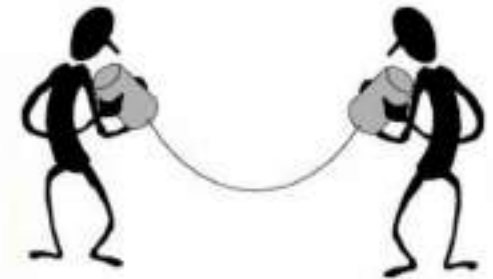
```
in
```

```
/,
```

```
/+
```

```
in
```

```
/);
```

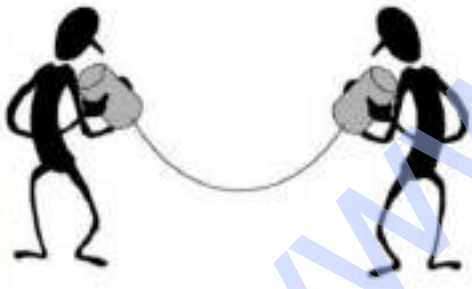


MPI datatype	C datatype
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_LONG_LONG	signed long long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	
MPI_PACKED	

Communication

```

int MPI_Reduce(
    void *       mSg_buf_p    /*+ out +/,
    int          buf_size     /*+ in  +/,
    MPI_Datatype datatype     /*+ in  +/,
    int          source        /*+ in  +/,
    int          tag           /*+ in  +/,
    MPI_Comm     communicator /*+ in  +/,
    MPI_Status   status       /*+ out +/);
    
```



Message matching

```
MPI_Send(send_buf_p, send_buf_sz, send_type, @esp.ssm_tad.  
<3cad_soi.
```

MPI_Send

src = q



MPI_Recv
dest = r

```
MPI_Recv(recv_buf_p, recv_buf_sz, reev_type, Groce_tad  
<res_eon>, &status):
```

q

Receiving messages

- A receiver can get a message without knowing:
 - the amount of data in the message,
 - the sender of the message,
 - or the tag of the message.



status_p argument

```
MPI_Recv(recv_buf_p, recv_buf_sz, recv_type, src, recv_tag,  
recv_comm, &status);
```

MPI_Status

```
MPI_Status* status;
```

```
status.MPI_SOURCE
```

```
status.MPI_TAG
```

MPI_SOURCE

MPI_TAG

MPI_ERROR

How much data am I receiving?

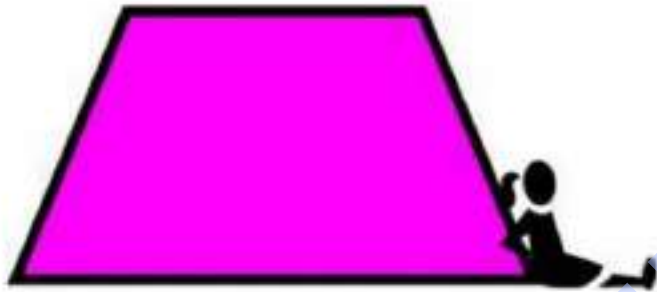
```
int MPI_Get_count (  
    MPI_Status+ status_P /* in */  
    MPI_Datatype type /* ir */  
    int + count_P /* out */);
```



Issues with send and receive

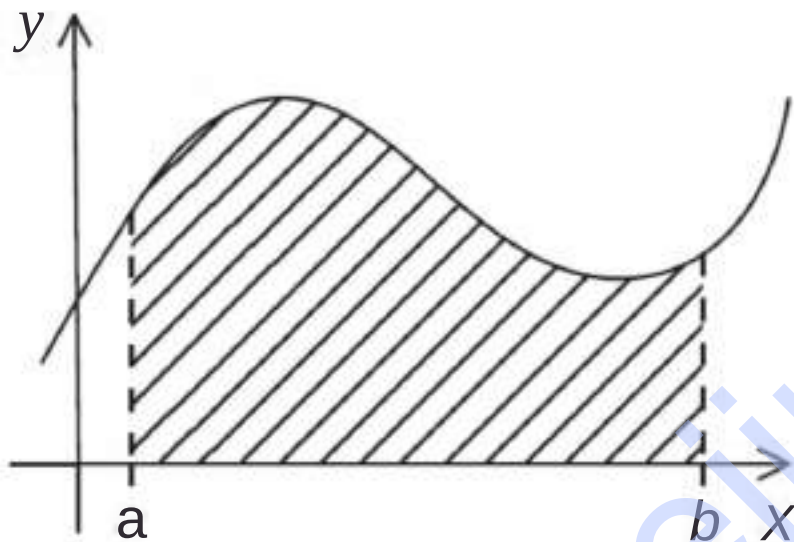
- Exact behavior is determined by the MPI implementation.
- MPI_Send may behave differently with regard to buffer size, cutoffs and blocking.
- MPI_Recv always blocks until a matching message is received.
- Know your implementation; don't make assumptions!



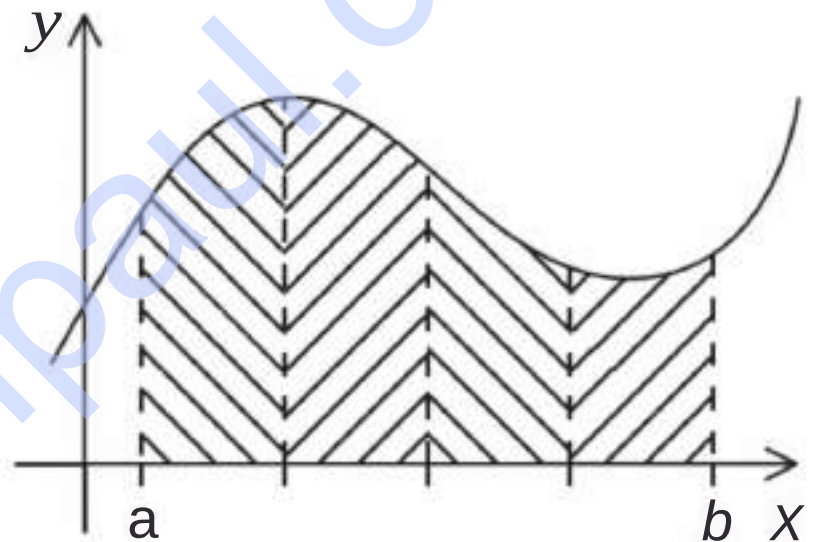


TRAPEZOIDAL RULE IN MPI

The Trapezoidal Rule



{a}



{b}

The Trapezoidal Rule

www.rejinpaul.com

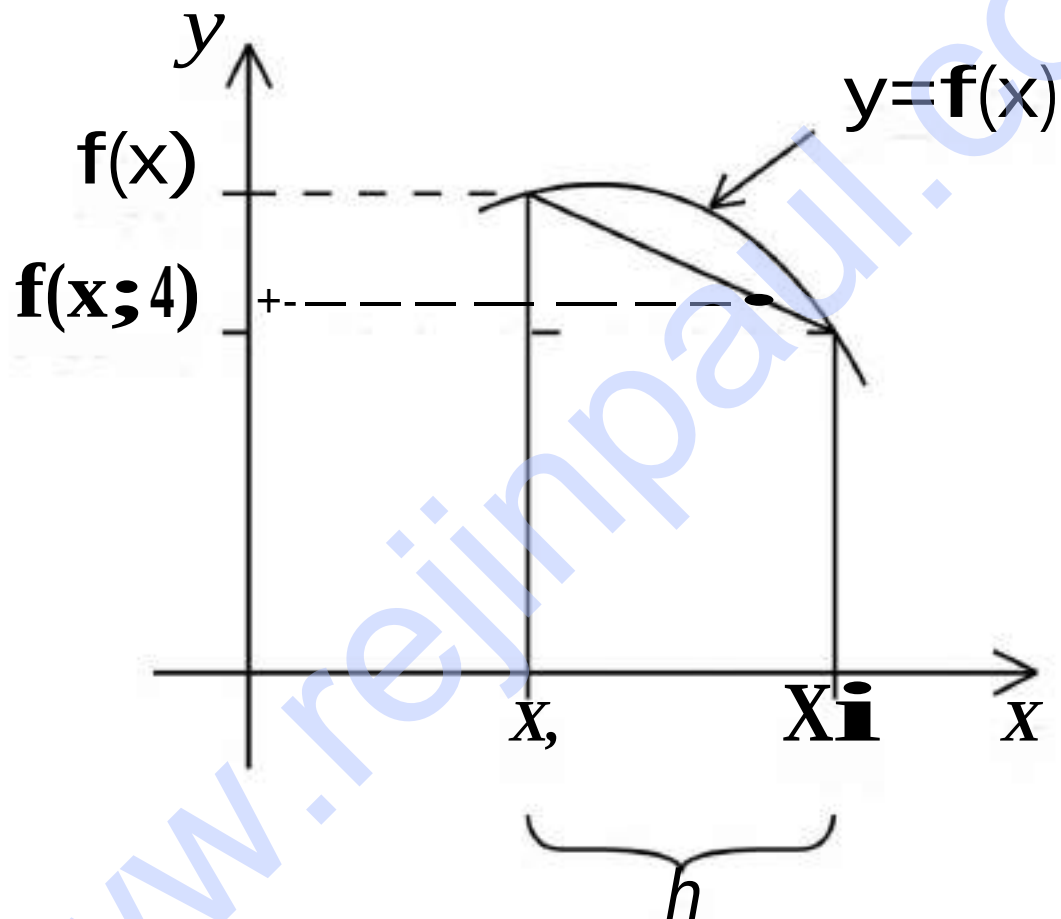
$$\text{Area of one trapezoid} = \frac{h}{2} [f(x_0) + f(x_n)]$$

$$\int_a^b \bullet \frac{b-a}{n}$$

$$x_0 = a, x_1 = a+h, x_2 = a+2h, \dots, x_{n-1} = a+(n-1)h, x_n = b$$

$$\text{Sum of trapezoid areas} = h \left[\frac{f(x_0)}{2} + f(x_1) + f(x_2) + \dots + f(x_{n-1}) + \frac{f(x_n)}{2} \right]$$

One trapezoid



Pseudo-code for a serial program

```
Input:  $a, b, n \in \mathbb{Z}$   
h = (b-a)/n;  
approx = (f(a) + f(b))/2.0;  
for (i = 0; i <= n-1; i++) {  
    x_i = a + i*h;  
    approx += f(x_i);  
}  
approx = h*approx;
```

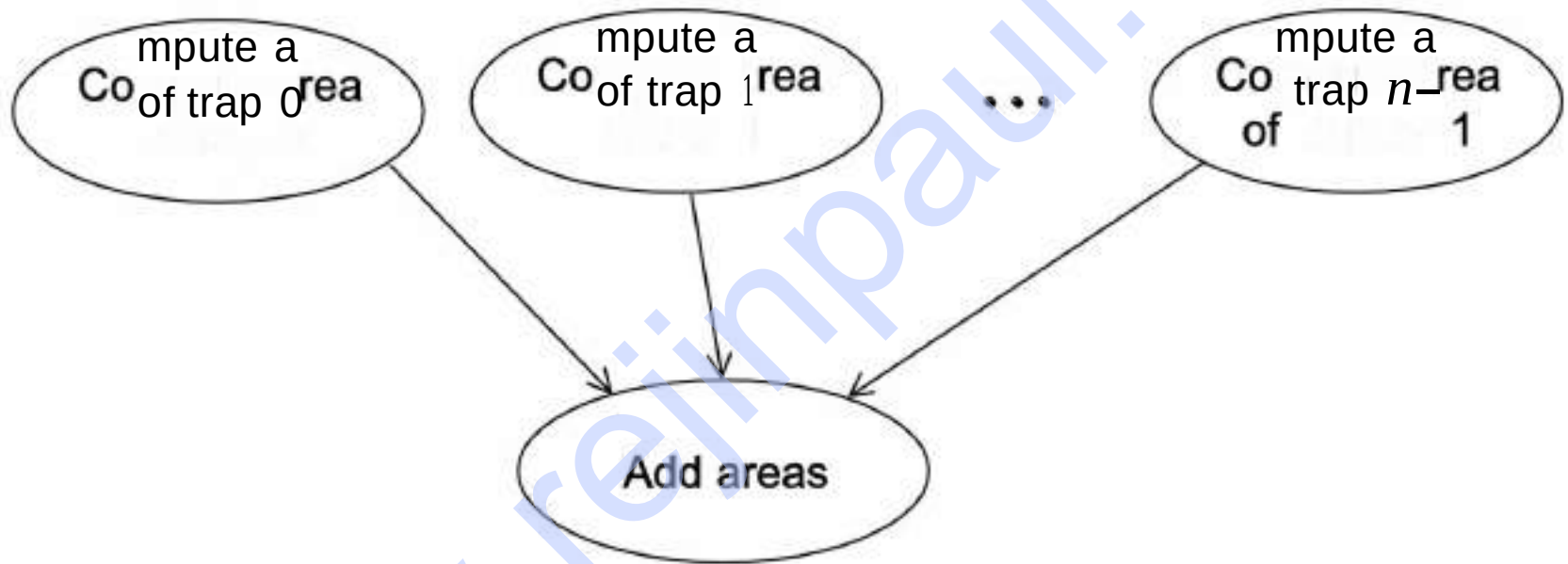

Parallelizing the Trapezoidal Rule

1. Partition problem solution into tasks.
2. Identify communication channels between tasks.
3. Aggregate tasks into composite tasks.
4. Map composite tasks to cores.

Parallel Pseudo-code

```
1   Get a, b, n;
2   h = (b-a)/n;
3   local_n = n/comm_sz;
4   local_a = a + my_rank*local_n;
5   local_b = local_a + local_n;
6   local_integral = Trap(local_a, local_b, local_n, h);
7   if (my_rank != 0)
8       Send local_integral to process 0;
9   else // my_rank == 0
10      total_integral = local_integral;
11      for (proc = 1; proc < comm_sz; proc++) {
12          Receive local_integral from proc;
13          total_integral += local_integral;
14      }
15
16   if (my_rank == 0)
17       print result;
```

Tasks and communications for Trapezoidal Rule



```
1  int main(void) {
2      int my_rank, comm_sz, n = 1024, local_n;
3      double a = 0.0, b = 3.0, h, local_a, local_b;
4      double local_int, total_int;
5      int source;
6
7      MPI_Init(NULL, NULL);
8      MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
9      MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
10
11     h = (b-a)/n;           / h is the same for all processes ■
12     local_n = n/comm_sz;   / So is the number of trapezoids ■
13
14     local_a = a + my_rank*h;
15     local_b = local_a + local_n*h;
16     local_int = Trap(local_a, local_b, local_n, h);
17
18     if (my_rank != 0) {
19         MPI_Send(&local_int, 1, MPI_DOUBLE, 0, 0,
20                 MPI_COMM_WORLD);
```

First version (2) ➤

```
21 } else {
22     total_int = local_int;
23     for (source = 1; source < comm_sz; source++) {
24         MPI_Recv(&local_int, 1, MPI_DOUBLE, source, 0,
25                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
26         total_int += local_int;
27     }
28 }
29
30 if (my_rank == 0) {
31     printf("With n = %d trapezoids, our estimate\n", n);
32     printf("of the integral from %f to %f = %.15e\n",
33           a, b, total_int);
34 }
35 MPI_Finalize();
36 return 0;
37 } // man al
```

First version (3)



```
1  double Trap(
2      double left_endpt  / in / ,
3      double right_endpt / in / ,
4      int trap_count    / in / ,
5      double base_len    / in / ) {
6      double estimate, x;
7      int i;
8
9      estimate = (f(left_endpt) + f(right_endpt))/2.0;
10     for (i = 1; i <= trap_count; i++){
11         x = left_endpt + i*base_len;
12         estimate += f(x);
13     }
14     estimate = estimate*base_len;
15
16     return estimate;
17 }
```

Dealing with I/O

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```
int main(void) {  
    int my_rank, comm_sz;
```

```
    MPI_Init(NULL, NULL);
```

```
    MPI_Comm_Size(MPI_COMM_WORLD, &comm_sz);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
```

```
    printf("Proc %d of %d > Does anyone have a toothpick?\n",  
           my_rank, comm_sz);
```

```
    MPI_Finalize();
```

```
    return 0;
```

```
} // main
```

*Each process just
prints a message.*

```
Proc 0 of 6 > Does anyone have a toothpick?  
Proc 1 of 6 > Does anyone have a  
toothpick? Proc 2 of 6 > Does anyone have a toothpick?  
Proc 4 of 6 > Does anyone have a toothpick?  
Proc 3 of 6 > Does anyone have a toothpick?  
Proc 5 of 6 > Does anyone have a toothpick?
```

unpredictable output



- Most MPI implementations only allow process 0 in **MPI_COMM_WORLD** access to `stdin`.
- Process 0 must read the data (`scanf`) and send to the other processes.

```
MPI_Comm_rank (MPI_COMM_WORLD, &my_rank);
MPI_Comm_Size (MPI_COMM_WORLD, &comm_sz );

Get_data(my_rank, comm_sz, &a, &b, &n);

h = (b-a)/n;
```

Function for reading user input

```
void Get_input(
    int      my_rank    /* in */,
    int      comm_sz    /* in */,
    double+  a_p        /* out */,
    double+  b_p        /* out */,
    int+     n_p        /* out */) {
    int dest;

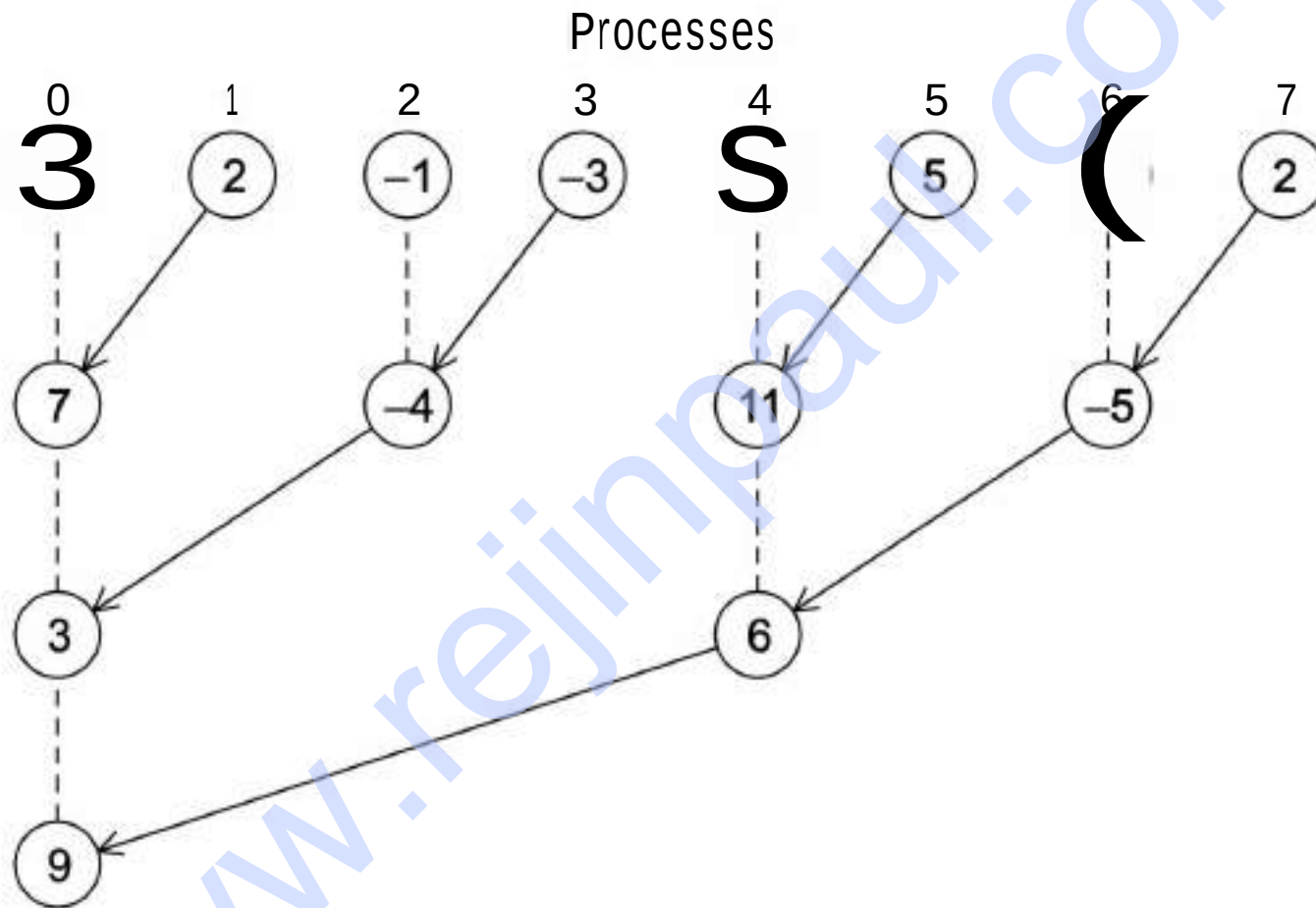
    if (my_rank == 0) {
        printf("Enter a, b, and n\n");
        scanf("%lf %lf %d", a_p, b_p, n_p);
        for (dest = 1; dest < comm_sz; dest++) {
            MPI_Send(a_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
            MPI_Send(b_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
            MPI_Send(n_p, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
        }
    } else { /* my_rank != 0 */
        MPI_Recv(a_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
        MPI_Recv(b_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
        MPI_Recv(n_p, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
    }
} /* Get_input */
```

COLLECTIVE COMMUNICATION

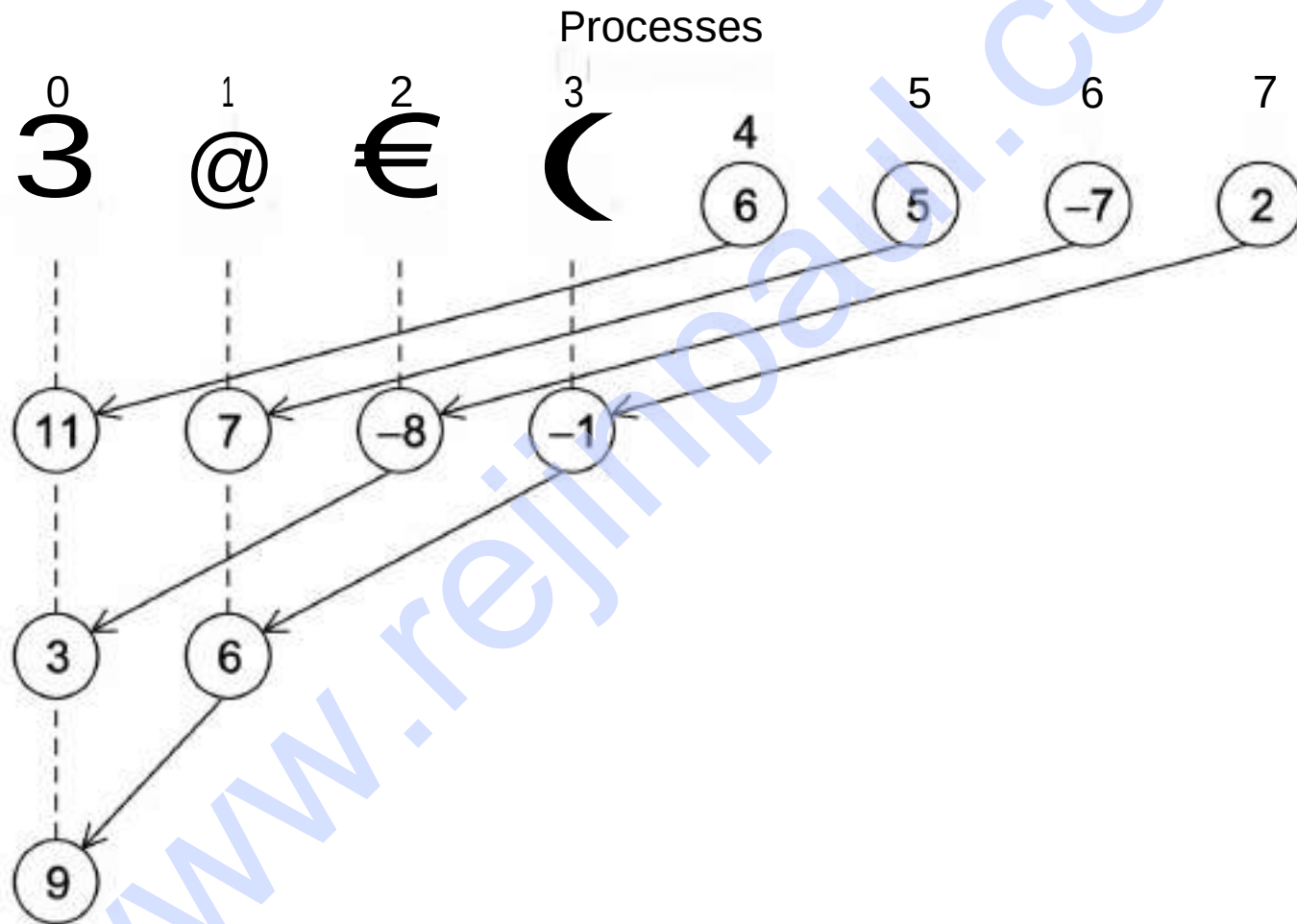
Tree structured communication

1. In the first phase:
 - (a) Process 1 sends to 0, 3 sends to 2, 5 sends to 4, and 7 sends to 6.
 - (b) Processes 0, 2, 4, and 6 add in the received values.
 - (c) Processes 2 and 6 send their new values to processes 0 and 4, respectively.
 - (d) Processes 0 and 4 add the received values into their new values.
2.
 - (a) Process 4 sends its newest value to process 0.
 - (b) Process 0 adds the received value to its newest value.

A tree structured global sum



An alternative tree-structured global sum



MPI_Reduce

```

int MPI_Reduce (
    void *      input_data_P  /* in  +
    void *      output_data_D /* out +/ ,
    int         count        /* in  +/ "
    MPI_Datatype datatype    /* +  in  +/ "
    MPI_Op       operator     /* %  in  / ,
    int         dest_process  /* %  in  +/ "
    MPI_Comm     comm         /* ≠  in  /);
  
```

```

MPI_Reduce(&local_int, &total_int, 1, MPI_DOUBLE, MPI_SUM, 0,
           MPI_COMM_WORLD);
  
```

```

double local_x[N], sum[N];
. . .
MPI_Reduce(local_x, sum, N, MPI_DOUBLE, MPI_SUM, 0,
           MPI_COMM_WORLD);
  
```

Predefined reduction operators

in MPI

Operation Value	Meaning
MP I_MAX	Maximum
MP I_MIN	Minimum
MP I_SUM	Sum
MP I_PROD	Product
MP I_LAND	Logical and
MP I_BAND	Bitwise and
MP I_LOR	Logical or
MPI...BOR	Bitwise or
MP I_LXOR	Logical exclusive or
MPI...BXOR	Bitwise exclusive or
MP I_MAXLOC	Maximum and location of maximum
MP I_MINLOC	Minimum and location of minimum

Collective vs. Point-to-Point Communications

- All the processes in the communicator must call the same collective function.
- For example, a program that attempts to match a call to `MPI_Reduce` on one process with a call to `MPI_Recv` on another process is erroneous, and, in all likelihood, the program will hang or crash.

Collective vs. Point-to-Point Communications

- The arguments passed by each process to an MPI collective communication must be "compatible."
- For example, if one process passes in 0 as the `dest_process` and another passes in 1, then the outcome of a call to `MPI_Reduce` is erroneous, and, once again, the program is likely to hang or crash.

Collective vs. Point-to-Point Communications

- The `output_data_p` argument is only used on `dest_process`.
- However, all of the processes still need to pass in an actual argument corresponding to `output_data_p`, even if it's just `NULL`.

Collective vs. Point-to-Point Communications

- Point-to-point communications are matched on the basis of tags and communicators.
- Collective communications don't use tags.
- They're matched solely on the basis of the communicator and the order in which they're called.

Example (1)

www.rejinpaul.com

Time	Process 0	Process 1	Process 2
0	a = 1; c = 2	a = 1; c = 2	a = 1; c = 2
1	MPI_Reduce(a, &b, ...)	MPI_Reduce(&c, &d, ...)	MPI_Reduce(&a, &b, ...)
2	MPI_Reduce(&c, &d, ...)	MPI_Reduce(&a, &b, ...)	MPI_Reduce(&c, &d, ...)

Multiple calls to MPI_Reduce

Example (2)

www.rejinpaul.com

- Suppose that each process calls `MPI_Reduce` with operator **`MPI_SUM`**, and destination process 0.
- At first glance, it might seem that after the two calls to `MPI_Reduce`, the value of `b` will be 3, and the value of `d` will be 6.

Example (3)

www.rejinpaul.com

- However, the names of the memory locations are irrelevant to the matching of the calls to `MPI_Reduce`.
- The order of the calls will determine the matching so the value stored in `b` will be $1+2+1 = 4$, and the value stored in `d` will be $2+1+2 = 5$.

MPI_Allreduce

www.rejinpaul.com

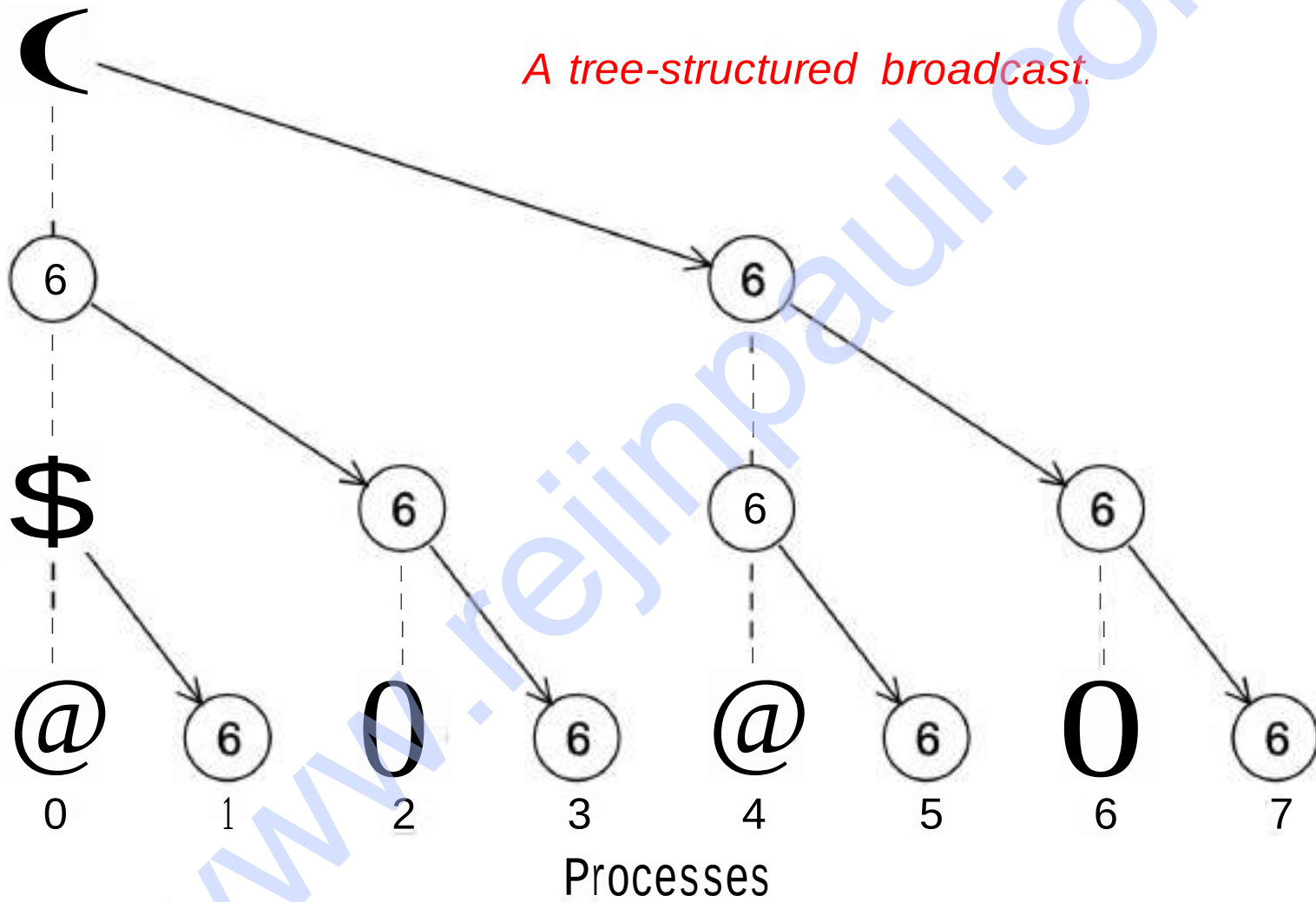
- Useful in a situation in which all of the processes need the result of a global sum in order to complete some larger computation.

```
int MPI_Allreduce(  
    void* input_data_p,      /*+ in+ */  
    void* output_data_p,    /*/k out k/ */  
    int count,               /*/ in+ */  
    MPI_Datatype datatype,   /*/+ in+ */  
    MPI_Op operator,         /*/+ in+ */  
    MPI_Comm comm           /*/+ in+ */) ;
```


Broadcast

- Data belonging to a single process is sent to all of the processes in the communicator.

```
int MPI_Bcast(
    void* data_p, int count, MPI_Datatype datatype,
    int source_proc, MPI_Comm comm) ;
```



A version of Get_input that uses MPI_Beast

```
void Get_input (
    int      my_rank    /*+ in */
    int      comm_sz    /*+ in */
    double *  a_p        /*+ out */
    double *  b_p        /*+ out */
    int *     n_p        /*+ out */) {

    if (my_rank == 0) {
        printf("Enter a, b, and n\n");
        scanf("%lf %lf %d", a_p, b_p, n_p);
    }

    MPI_Bcast(a_p, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    MPI_Bcast(b_p, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    MPI_Bcast(n_p, 1, MPI_INT, 0, MPI_COMM_WORLD);
} /* Get_input */
```

- Block partitioning
 - Assign blocks of consecutive components to each process.
- Cyclic partitioning
 - Assign components in a round robin fashion.
- Block-cyclic partitioning
 - Use a cyclic distribution of blocks of components.

Parallel implementation of vector addition

```
void Parallel_vector_sum(  
    double local_x []  
    double local_y []  
    double local_z []  
    int local_n  
    int local_i;  
  
    for (local_i = 0; local_i < local_n; local_i++)  
        local_z[local_i] = local_x[local_i] + local_y[local_i];  
} he Parallel_vector_sum
```

- MPI Scatter can be used in a function that reads in an entire vector on process 0 but only sends the needed components to each of the other processes.

```
int MPI_Scatter(
    void * send_buf_p  /* In */
    int send_count      /* n */
    MPI_Datatype send_type /* n */
    void * recv_buf_p  /* out */
    int recv_count      /* n */
    MPI_Datatype recv_type /* n */
    int src_proc        /* in */
    MPI_Comm comm       /* n */
);
```

Reading and distributing a vector

```
void Read_vector(  
    double    local_a[]    / out /,  
    int       local_n      /+ in /,  
    int       n            /+ in /,  
    char      vec_name[]   /+ in /,  
    int       my_rank      / in /,  
    MPI_Comm comm          /+ in /) {  
  
    double * a = NULL;  
    int i;  
  
    if (my_rank == 0) {  
        a = malloc(n*sizeof(double));  
        printf("Enter the vector %s\n", vec_name);  
        for (i = 0; i < n; i++)  
            scanf("%lf", &a[i]);  
        MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n, MPI_DOUBLE,  
                    0, comm);  
        free(a);  
    } else {  
        MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n, MPI_DOUBLE,  
                    0, comm);  
    }  
} / Read_vector 1
```

- Collect all of the components of the vector onto process 0, and then process 0 can process all of the components.

```

int MPI_Gather (
    void* k          send_buf_p    / 1n  / "
    int             send_count    / 1n  / "
    MPI_Datatype     send_type    / 1n  / "
    void* k          recv_buf_p   / out / "
    int             recv_count    / 1n  / "
    MPI_Datatype     recv_type    / 1n  / "
    int             dest_proc     / 1n  / "
    MPI_Comm         comm        / 1n  / );
    
```


Print a distributed vector (1)

```
void Print_vector(  
    double    local_b[]  
    int        local_n  
    int        n  
    char       title[]  
    int        my_rank  
    MPI_Comm   comm  
  
    double*    b = NULL;  
    int        i;
```

Print a distributed vector (2)

```
if (my_rank == 0) {
    b = malloc(n*sizeof(double));
    MPI_Gather(local_b, local_n, MPI_DOUBLE, b, local_n, MPI_DOUBLE,
               0, comm);
    printf("%s\n", title);
    for (i = 0; i < n; i++)
        printf("%E ", b[i]);
    printf("\n");
    free(b);
} else {
    MPI_Gather(local_b, local_n, MPI_DOUBLE, b, local_n, MPI_DOUBLE,
               0, comm);
}
h» Print_vector I
```

- Concatenates the contents of each process' `send_buf_p` and stores this in each process' `recv_buf_p`.
- As usual, `recv_count` is the amount of data being received from each process.

```
int MPI_Allgather(  
    void * send_buf_p, /* n */  
    int send_count, /* In */  
    MPI_Datatype send_type, /* In */  
    void * recv_buf_p, /* out */  
    int recv_count, /*+ In +/ */  
    MPI_Datatype recv_type, /*+ In +/ */  
    MPI_Comm comm /*+ In */);
```

Multiply a matrix by a vector

```
/* For each row of A */  
for (i = 0; i < m; i++) {  
    /* Form dot product of ith row with x */  
    v[i] = 0.0;  
    for (j = 0; j < n; j++)  
        v[i] += A[i][j]*x[j];  
}
```

Serial pseudo-code

C style arrays

$$\begin{pmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \end{pmatrix}$$

stored as

0 1 2 3 4 5 6 7 8 9 10 11

Serial matrix-vector multiplication

```
void Mat_vect_mult(
    double A[]  // In
    double x[]  // 1n
    double y[]  // out
    int m      // 1n
    int n      // 1n ) {
    int i, j;

    for (i = 0; i < m; i++) {
        y[i] = 0.0;
        for (j = 0; j < n; j++)
            y[i] += A[i+n*j+x[j]];
    }
} /* Mat_vect_mult */
```

An MPI matrix-vector

multiplication function (1)

```
void Mat_vect_mult(  
    double local_A [L / 100 in // "  
    double local_x[] / in // "  
    double local_y[l / out // "  
    int local_m I in // "  
    int n - / in // "  
    int local_n / in // "  
    MPI_Comm comm - / in // "  
  
    double* x;  
    int local_i, j;  
    int local_ok = 1;  
    - =
```

An MPI matrix-vector

multiplication function (2)

```
x = malloc(n*sizeof(double));
MPI_Allgather(local_x, local_n, MPI_DOUBLE,
              x, local_n, MPI_DOUBLE, comm);

for (local_i = 0; local_i < local_m; local_i++) {
    local_y[local_i] = 0.0;
    for (j = 0; j < n; j++)
        local_y[local_i] += local_A[local_in+j*l+x[j]*l];
}
free(x);
} /* Mat_vect_mult */
```


MPI DERIVED DATATYPES

- Used to represent any collection of data items in memory by storing both the types of the items and their relative locations in memory.
- The idea is that if a function that sends data knows this information about a collection of data items, it can collect the items from memory before they are sent.
- Similarly, a function that receives data can distribute the items into their correct destinations in memory when they're received.

Derived DATATYPES

- Formally, consists of a sequence of basic MPI data types together with a displacement for each of the data types.
- Trapezoidal Rule example:

[Variable Address]

a 24

b 40

n 48

{(MPI_DOUBLE,0), (MPI_DOUBLE, 16), (MPI_INT,24)}

MPI_Type_create_struct

www.rejinpaul.com

- Builds a derived datatype that consists of individual elements that have different basic types.

```
int MPI_Type_create_struct(
    int count /+ ln /,
    int array_of_blocklengths[] /+ ln +/,
    MPI_Aint array_of_displacements[] /+ ln /,
    MPI_Datatype array_of_types[] /+ ln +/,
    MPI_Datatype* new_type_p /+ out ±/);
```

MPI_Get_address

www.rejinpaul.com

- Returns the address of the memory location referenced by `location_p`.
- The special type `MPI_Aint` is an integer type that is big enough to store an address on the system.

```
int MPI_Get_address(
    void* location_p,
    MPI_Aint* address_p);
```

MPI_Type_commit

www.rejinpaul.com

- Allows the MPI implementation to optimize its internal representation of the datatype for use in communication functions.

```
int MPI_Type_commit(MPI_Datatype *new_mpi_t_p) /* in/out */;
```

MPI_Type_free

www.rejinpaul.com

- When we're finished with our new type, this frees any additional storage used.

```
int MPI_Type_free(MPI_Datatype *old_mpi_t_p /* in/out ↗);
```

Get input function with a derived datatype (1)

```
void Build_mpi_type(
    double + a_P /*# n /
    double + b_P /*+ n /
    int + n_P /*% ln /
    MPI_Datatype+ input_mpi_t_p /*# out +/) {

    int array_of_blocklengths[3] = {1, 1, 1};
    MPI_Datatype array_of_types[3] = {MPI_DOUBLE, MPI_DOUBLE, MPI_INT};
    MPI_Aint a_addr, b_addr, n_addr;
    MPI_Aint array_of_displacements[3] = {0};
```


Get input function with a derived datatype (2)

```
MP I_Get_address(a_p. &a_addr );
MP I_Get_address(b_p. &b_addr);
MP I_Get_address(n_p. &n_addr):
array_of_displacements[1] = b_addr-a_addr;
array_of_displacements[2] = n_addr-a_addr;
MP I_Type_create_struct(3, array_of_blocklengths,
                        array_of_displacements, array_of_types,
                        input_mpi_t_p):
MP I_Type_commit(input_mpi_t_p);
} h» Build_mpi_type «/
```

Get input function with a derived datatype (3)

```
void Get_input(int my_rank, int comm_sz, double# a_p, double+ b_p.  
    int+ n_p) {  
    MPI_Datatype input_mpi_t;  
  
    Build_mpi_type(a_p. b_p. n_p. &input_mpi_t);  
  
    if (my_rank == 0) {  
        printf("Enter a, b, and n\n");  
        scanf("&lf %lf %d", a_p. b_p. n_p);  
    }  
    MPI_Bcast(a_p. I. input_mpi_t, 0, MPI_COMM_WORLD);  
  
    MPI_Type_free(&input_mpi_t);  
} /* Get_input */
```

PERFORMANCE EVALUATION

Elapsed parallel time

- Returns the number of seconds that have elapsed since some time in the past.

```
double MPI_Wtime(void);
```

```
double start, finish;
```

```
start = MPI_Wtime();
```

```
/* Code to be timed */
```

```
finish = MPI_Wtime();
```

```
printf("Proc %d > Elapsed time = %e seconds\n",  
       my_rank, finish-start);
```

Elapsed serial time

- In this case, you don't need to link in the MPI libraries.
- Returns time in microseconds elapsed from some point in the past.

```
#include "timer.h"
```

```
double now;
```

```
GET_TIME(now);
```

Elapsed serial time

```
#include "timer.h"
```

```
double start, finish;
```

```
GET_TIME(start);
```

```
/* Code to be timed */
```

```
GET_TIME(finish);
```

```
printf("Elapsed time = %e seconds\n", finish-start);
```

- Ensures that no process will return from calling it until every process in the communicator has started calling it.

```
int MPI_Barrier(MPI_Comm comm /+ in 1);
```

MPI_Barrier

```
double local_start, local_finish,
local_elapsed, elapsed;

MPI_Barrier(comm);
. . .
local_start = MPI_Wtime(0);
Code to be timed el
local_elapsed = local_finish - local_start;
MPI_Reduce(&local_elapsed, &elapsed, 1,
MPI_DOUBLE, MPI_MAX, 0, comm);

if (my_rank == 0)
printf("Elapsed time = %e seconds\n", elapsed);
```


UNIT V- PARALLEL PROGRAM DEVELOPMENT

Case studies - n-Body solvers – Tree Search – OpenMP and MPI implementations and comparison.

Introduction

Many physical phenomena directly or indirectly (when solving a discrete version of a continuous problem) involve, or can be simulated with particle systems, where each particle interacts with all other particles according to the laws of physics. Examples include the gravitational interaction among the stars in a galaxy or the Coulomb forces exerted by the atoms in a molecule. The challenge of efficiently carrying out the related calculations is generally known as the *N-body problem*.

Mathematically, the N-body problem can be formulated as

$$U(\mathbf{x}_0) = \sum_i F(\mathbf{x}_0, \mathbf{x}_i) \quad (1)$$

where $U(\mathbf{x}_0)$ is a physical quantity at $\mathbf{x}_0$ which can be obtained by summing the pairwise interactions $F(\mathbf{x}_0, \mathbf{x}_i)$ over the particles of the system. For instance, assume a system of N particles, located at $\mathbf{x}_i$ and having a mass of m_i . The gravitational force exerted on a particle $\mathbf{x}$ having a mass m is then expressed as

$$F(\mathbf{x}) = -Gm \sum_{i=1}^N m_i \frac{\mathbf{x} - \mathbf{x}_i}{|\mathbf{x} - \mathbf{x}_i|^3} \quad (2)$$

where G is the gravitational constant.

The task of evaluating the function $U(\mathbf{x}_0)$ for all N particles using (1) requires $O(n)$ operations for each particle, resulting in a total complexity of $O(n^2)$. In this paper we will see how this complexity can be reduced to $O(n \log n)$ or $O(n)$ by using efficient methods to approximate the sum in the right hand term of (1), while still preserving such important physical properties as energy and momentum.

Example N-body problem: 10 million-star galaxy

The summation techniques presented in this paper may be applied to a wide range of N-body problems. However, in order to clarify it is beneficial to have an actual problem to apply the ideas on. In this case, I have chosen the problem of simulating the movements of the stars of a galaxy. Let's assume that there are about $N:=10$ million stars in the galaxy, although this is clearly much less than "in the real world". Furthermore, for the sake of clarity the simulation will be done in two dimensions.

In the model, each star has the following quantities associated with it:

mass, m_i

position, $\mathbf{x}_i$ (depends on time)

Newtonian (let's not allow Einstein to mess things up) physics tells us that for each star

$$\frac{d^2 \mathbf{x}_j}{dt^2} = -Gm_j \sum_{i=1, i \neq j}^N m_i \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3} \quad (3)$$

Then, for each timestep from time t_k to $t_{k+1} := t_k + \Delta t$ we need to integrate the right hand term of equation (3) in order to obtain the change in position:

$$\dot{\mathbf{x}}_j = \int_{t_k}^{t_{k+1}} \mathbf{F}(\mathbf{x}_j(t)) dt \quad (4)$$

where

$$\mathbf{F}(\mathbf{x}_j) = \sum_{i=1}^N G m_i \frac{\mathbf{x}_j - \mathbf{x}_i}{|\mathbf{x}_j - \mathbf{x}_i|^3} \quad (5)$$

(4) is a somewhat difficult integral equation, since $\mathbf{x}_j$ is present on both sides. Also, $\mathbf{x}_i$ is dependent on t , which means we have a system of N coupled integral equations for each time step.

A discrete version of (4) (which can be obtained by making certain assumptions) has the general form¹

$$\dot{\mathbf{x}}_j = \sum_{i=1}^k c_i \mathbf{F}(\mathbf{x}_j(t - h_i)), \quad k \quad (6)$$

and is thus a linear combination of the function $\mathbf{F}$ evaluated at different time points; different discrete integration schemes yield different coefficients c_i and h_i . A commonly used integrator is the so-called *Leapfrog* integration scheme.

We can now formulate an algorithm for the simulation:

1. Set initial positions
2. for each timestep t do
3. for each particle j do
4. evaluate $\mathbf{F}(\mathbf{x}_j(t))$ at timepoints required by the integrator
5. use the integrator to calculate $\dot{\mathbf{x}}_j$
6. $\mathbf{x}_j(t + \Delta t) = \mathbf{x}_j(t) + \dot{\mathbf{x}}_j \Delta t$
7. endfor
8. endfor

The function $\mathbf{F}$ is of the form (1), and thus the N-body force calculation algorithms presented in this paper can be used to speed up step 4 of the algorithm.

The Particle-Particle (PP) method

The method of evaluating the right hand side of (1) directly is generally referred to as the Particle-Particle (PP) method². This brute-force approach is clearly not feasible for large amount of particles due to the $O(n^2)$ time requirement, but can effectively be used for small amounts of particles³. As this method does not approximate the sum, the accuracy equals machine precision.

Tree codes

Let a be the radius of the smallest disc, call it D , so that the set of particles $P := (\mathbf{x}_{i1}, \dots, \mathbf{x}_{iN})$ are inside the disc. Many physical systems have the property that the field $U(\mathbf{x})$ generated by the the particle set P may be very complex inside D , but smooth (“low on information content”) at some distance $c a$ from D . The gravitational force, for instance, has this property.⁴

This observation is used in the so-called *tree code*-approach to the N-body problem: Clusters of particles at enough distance from the “target particle” $\mathbf{x}_0$ of equation 1 are

given a computationally simpler representation in order to speed up summation⁵. This approach can be illustrated by the following example: when calculating the gravitational force exerted by earth on an orbiting satellite, we do not try to sum the gravitational forces of all atoms that constitute planet earth. Instead we approximate the force with that of an infinitesimal particle, which have the same mass as earth and is located at earth's center of mass.

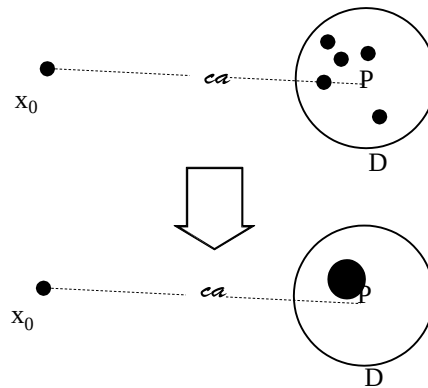


Figure 1 Approximating the set P

The tree-code approach is effective because it can be applied recursively⁶. For instance, in Figure 5, we might also be able to use a subset of P' P to approximate the force on the particle down to the left, as the radius a' of the disc D' enclosing P' is less than a , and thus the minimal distance for approximation, ca' , is also smaller.

We need a data structure that supports the recursive nature of the idea, in 2D this is a *quadtree* and in 3D an *octree*. Each node in the quadtree is a square, and has four child nodes (unless it is a leaf node), representing a break-up into four smaller squares, $\frac{1}{4}$ the size of the original square (see Figure 2). An octree is built in a similar manner (each node has 8 children).

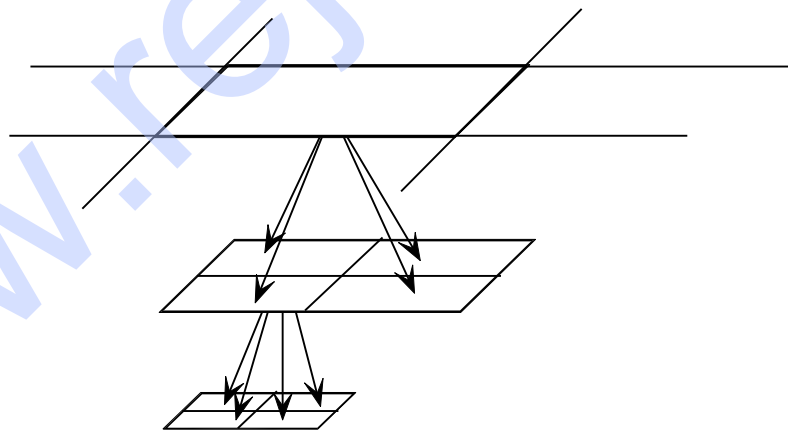


Figure 2 Quadtree

We now construct a quadtree, so that each leaf node contains only 1 particle. This is done by recursively subdividing the computational box; each node is further subdivided if it contains more than one particle.

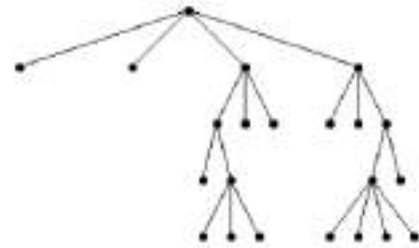
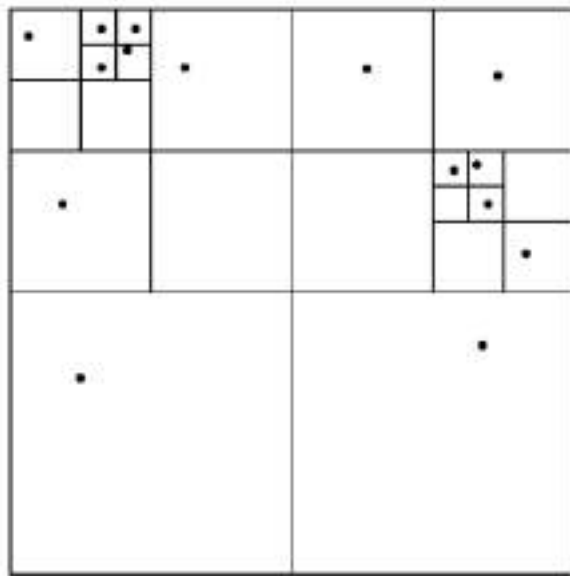


Figure 3 Adaptive quadtree with one particle/leaf. The picture is from [Demmel1]

Assuming the particles are not at arbitrarily small distances from each other (at least the machine precision sets a limit) a quadtree can be built in $O(n \min(b, \log n))$ time, where b is the machine precision⁷.

Now assume that the distance between a cluster and a particle must be at least the length of the side of the cluster box in order to obtain an accurate approximation. When calculating the force on a particle x_0 , the tree is recursively traversed from the root. At each level, there may be no more than 9 boxes (the ones surrounding the box containing the particle) which need further subdivision, limiting the number of force calculations on the next level to 27 ($=2 \cdot 3 \cdot 2 \cdot 3 - 9$, see Figure 4). Thus as each level a maximum of 27 $O(1)$ operations are performed. The depth of the tree is $\min(b, \log(n))$ yielding a total complexity (for all N particles) of $O(n \min(b, \log n))$.

1	2	2	2	2	2	2
	2	2	3	3	3	3
1	2	2	3	3	3	3
	2	2	3	3	3	3
1	2	2	3	3	3	3
	2	2	3	3	3	3
1	2	2	3	3	3	3
	2	2	3	3	3	3
1	1	1	1	1	1	1

Figure 4 Quadtree nodes calculated at each level for particle x

Tree codes thus reduces the computational complexity from $O(n^2)$ to $O(n \log n)$ or $O(n)$ depending on your point of view - certainly a vast improvement! But as the

saying goes, there's no such thing as a free lunch: tree codes are less accurate than simple PP, and require more auxiliary storage⁸.

The Barnes-Hut algorithm: applying tree codes.

The Barnes-Hut algorithm is a good example of how to use tree codes in an algorithm. The algorithm was presented in [Barnes&Hut] in 1986, and is widely used in astrophysics. The algorithm has also been successfully parallized.⁹ This discussion of the algorithm is mainly based on [Demmel1], which describes Barnes-Hut for a 2D N-body system similar to our example.

The main idea is to approximate long-range forces by aggregating particles into one particle, and using the force exerted by this particle. A quadtree structure (or octree in 3D¹⁰), as described in the previous section, is used to store the particles. The tree steps of the algorithm are

1. Build quadtree as described in section 0
2. Traverse the quadtree from the leaves to the root, computing center of mass and total mass for each parent node.
3. For each particle, traverse the tree from the root, calculating the force during the traversal

Step 2

Step 2 calculates the approximations for the long-range force. The approximation is made by considering several particles as one, with a position equal to the center of mass of the approximated particles, and a mass the sum of the approximated particles' masses. More formally, to find the mass and position associated with a node N:

```

calculate_approxiamtions( N )
  if N is a leaf node
    return; // Node has a (real) particle => has mass & position
  for all children n of N do
    calculate_approximations( n )
  M := 0
  cm := (0,0)
  for all children n of N do
    M := M + mass of n
    cm := cm + mass of n * position of n
  endfor
  cm := 1/M * cm
  mass of N := M
  position of N := cm
end

```

Step 3

Consider the ratio,

$$\frac{D}{r} \quad (7)$$

where D is the size of the current node (call it A) "box" and r is the distance to the center of mass of another node (called B). If this ratio is sufficiently small, we can use the center of mass and mass of B to compute the force in A. If this is not the case, we need to go to the children of B and do the same test. Figure 4 shows this for $\epsilon=1.0$; the numbers indicate the relative depth of the nodes. It can clearly be seen that further away from the particle, x , large nodes are used and closer to it smaller. The method is accurate to approximately 1% with $\epsilon=1.0$ ¹¹. Expressed in pseudocode

```

treeForce(x, N)
  if N is a leaf or size(N)/x-N<
    return force(x,N)
  else
    F := 0
    for all children n of N do
      F := F + treeForce(x, n)
    endfor
    return F
  endif
end

```

The Particle-Mesh (PM) method

Whereas we have discretized time, position is still continuous. The PM method goes one step further: it effectively discretizes position too¹². However, before we explore this idea further, we need the concept of potential.

Let us assume that there exists a quantity ρ , which is related to the physical quantity U we are studying according to:

$$U = c \rho \quad (8)$$

and, furthermore, that

$$\nabla^2 U = c \quad (9)$$

where ρ is the density function (e.g. mass, charge) obtained from the particle distribution and c is a constant. This leads to the Poisson equation¹³

$$\nabla^2 U = c \quad (10)$$

In the sample N-body problem U corresponds to the force, and ρ to the potential energy in the gravitational field generated by the particles. ρ is the mass density (mass/area unit). In the continued discussion of the PM method the quantities used will be these.

The idea of the PM method is that we set up a mesh (grid) over the computational box, and then solve the potential (i.e. Poisson's equation) at the meshpoints. Forces at the meshpoints can then be obtained by calculating the gradient of the potential. To find the force on a particle not located at a meshpoint we can either use the force at the nearest meshpoint, or interpolate the force from the closest meshpoints.¹⁴

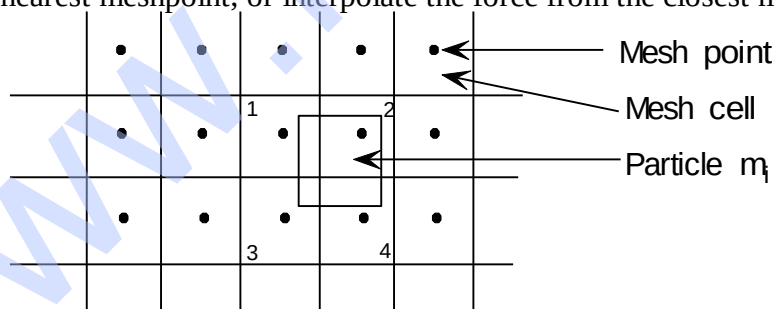


Figure 5 Mesh density function assignment

Creating the density function

There are several ways of assigning the particle masses to the density function¹⁵:

Nearest gridpoint (NGP): The mass of each particle m_i is assigned to the gridpoint closest to the mass. In Figure 5, this would mean assigning the entire mass of the particle to cell 2. NGP is also referred to as zero-order interpolation

Cloud-in-Cell (CIC): The mass of each particle is weighted over the four (in 2D) closest cells; the weighting is proportional to the intersection of the “cloud” surrounding the particle and the cell. In Figure 5 almost all mass would be assigned to cell 2, then approximately the same amount to cells 1 and 4, and finally about 1/16 to cell 3. CIC implements first order (linear) interpolation.

Higher order interpolations: The “cloud” (weighting function) around the particle can be made to cover even more cells, resulting in higher order interpolations, e.g. TSC (triangular shaped cloud).

Calculating the potentials

Now that we have the density function ρ , we can solve (10) in order to obtain the potential at the meshpoints. This is done by rewriting (10) as a system of discrete difference equations, and solving the system. The system can be solved in $O(G \log G)$ time, where G is the number of gridpoints, by using the Fast Fourier Transform.¹⁶

Finding the forces

Thus the potential is solved. Calculating the force is easy, we only need to calculate the gradient of the potential by taking the difference between two potential values. Some special considerations regarding the degree of force interpolation must however be taken in order to conserve such physical properties as momentum and energy.¹⁷

PM pros and cons

The PM method has a complexity of $O(n)$ with respect to the particles, the slowest steps being the FFT requiring $O(G \log G)$ operations.¹⁸ The cost of the speed is the limited spatial resolution of PM - phenomena at a smaller scale than the mesh spacing are not modelled accurately, i.e. it is only suited for modelling of systems where collisions occur. On the other hand, large-scale phenomena can be shown quite accurately.¹⁹

The Particle-Particle/Particle-Mesh algorithm (P³M)

The Particle-Particle/Particle-Mesh algorithm is a hybrid algorithm that strives to correct the shortcomings of PM when it comes to modelling of short-range phenomena. The following discussion is based on [Hockney]²⁰ and applied to our sample N-body problem.

The general idea is to split up the gravitational force in short-range and long-range forces, just as we did in section 0. The PM method is used to determine the long range force, and PP to correct the PM force at short distances.

Let r_e be the distance (typically 2-3 times the size of the mesh cell), below which we want to use the PP method for force calculation. Denote by $\mathbf{x}_0$ the position where the force is to be evaluated. In order to be able to efficiently find the particles $\mathbf{x}_i$ whose distance from $\mathbf{x}_0$ is less than r_e we employ as so-called *chaining mesh* for the particles,

see Figure 6. As can be see from the figure, those $\mathbf{x}_i$ closer to $\mathbf{x}_0$ than r_e must be in the neighbouring 8 chaining mesh cells of $\mathbf{x}_0$'s chaining mesh cell.

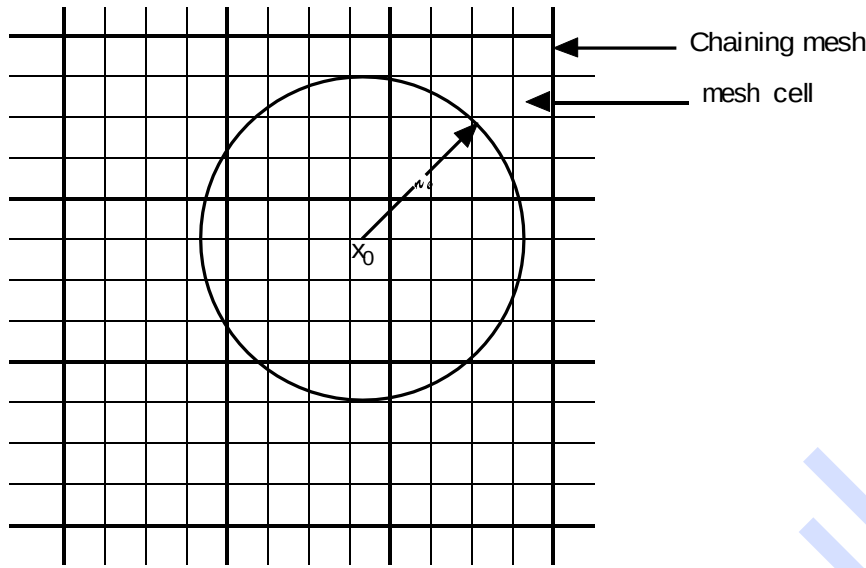


Figure 6 Mesh cells and chaining mesh

Mathematically, we define the correction, or short-range, force $\mathbf{f}^{sr}$ as

$$\mathbf{f}^{tot} = \mathbf{R} + \mathbf{f}^{sr} \quad \mathbf{f}^{sr} = \mathbf{f}^{tot} - \mathbf{R} \quad (11)$$

where $\mathbf{f}^{tot}$ is the total force and $\mathbf{R}$ is the force obtained using the PM method.

Suppose we have a particle $\mathbf{x}_i$ closer to $\mathbf{x}_0$ than r_e . We'd like to calculate the total force on $\mathbf{x}_0$, using direct PP to correct the force $\mathbf{R}$ obtained by PM calculations. With some lengthy mathematical manipulations it can be shown that PM treats the particle $\mathbf{x}_i$ as a "mass cloud" (the density and shape of which depend on the details of the PM method). Thus, we can correct the PM force by subtracting the force induced by such a "mass cloud" (this force is included in the PM force) and then adding the direct PP interaction force. Mathematically, in the case of pointlike masses

$$\mathbf{f}^{sr} = Gmm_i \frac{\mathbf{x}_0 - \mathbf{x}_i}{|\mathbf{x}_0 - \mathbf{x}_i|^3} \mathbf{e}(\mathbf{x}_0, \mathbf{x}_i) \quad (12)$$

where $\mathbf{e}(\mathbf{x}_0, \mathbf{x}_i)$ is the function for the gravitational force of the "mass cloud" the PM method treats $\mathbf{x}_i$ as. We can now formulate the P³M algorithm:

1. for each particle $\mathbf{x}$ do
2. find force on $\mathbf{x}$:= $\mathbf{F}$ using the PM method
3. locate particles closer than r_e for PP calculation using the chaining mesh. Calculate short range force using these particles
4. force on $\mathbf{x}$:= $\mathbf{F}$ + short range forces
5. endfor

The P³M method has been used widely in cosmological simulations, and is easy to use when forces can be easily split into short and long-range (i.e. gravity). A problem is that the algorithm easily becomes dominated by the PP phase.²¹

Fast multipole method (FMM)

Multipole expansion can be viewed as a tree code, with the following main differences

Potential instead of force is used

Multipole expansions of the potential of a box are more accurate than a simple center of mass substitution, but also computationally “heavier”. This is compensated by having more than 1 particle per tree leaf.²²

FMM only require evaluation of $\mathbf{F}(\mathbf{x}_0, \mathbf{x}_i)$ at leaf nodes, and possesses a “richer analytical structure”, meaning it can be adapted to a wider variety of problems²³

Potentials and multipole expansion of the gravitational field in 2D

The mathematics associated with FMM is somewhat lengthy, but not excessively advanced. This section will deal with multipole expansions of the gravitational field of our example N-body problem, similar methods are used when expanding other quantities and/or in other dimensionalities²⁴. The following presentation of the mathematical ideas is based on [Demmel2].

Recall that the potential of a particle satisfies Poissons equation (equation 10). A solution to (10) for a point mass located at $\mathbf{x}_0$ in 2D is

$$\phi(\mathbf{x}) = -\log(|\mathbf{x} - \mathbf{x}_0|) \quad (13)$$

Using the complex number $z = a + bi$ to represent the point $\mathbf{x} = (a, b)$ the potential can be rewritten as the real part of the complex logarithm, which is analytic. Remembering that potentials are additive, the total potential from n particles can be expressed as

$$\phi(z) = -\sum_{i=1}^n m_i \log(z - z_i) = -\sum_{i=1}^n m_i (\log(z) - \log(1 - z_i/z)) \quad (14)$$

Since $\log(z)$ is analytic, it is possible to do a Taylor expansion around origo of $\log(1 - z_i/z)$, yielding

$$\phi(z) = -M \log(z) - \sum_{j=1}^{\infty} \frac{m_j z_i^j}{j} z^{-j} \quad (12)$$

where M is the combined mass of all particles and

$$m_j = \sum_{i=1}^n m_i z_i^j \quad (13)$$

We approximate the potential $\phi(z)$ by summing a finite number, p , of terms in (12). The error is then proportional to

$$\frac{\max(|z_i|)^{p+1}}{|z|} \quad (14)$$

Now, suppose that all z_i lie inside a DD square centered at origin and z is evaluated outside a DD square centered at origin, then $\max(|z_i|) \leq \sqrt{2} \cdot \Delta$ (see Figure 7). We say that z and z_i are well-separated when this condition holds. Also note that the potential is expressed as a power series, and the gradient can thus easily be computed analytically, avoiding further discretization errors (compare to PM!).

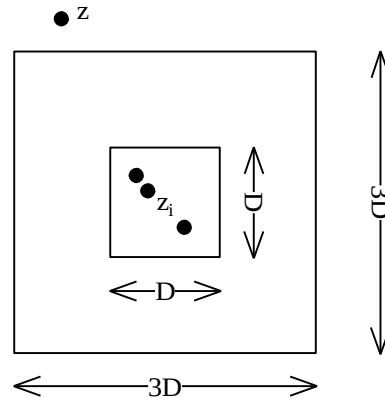


Figure 7 The expansion of the particles z_i converge outside the outer box

Expansion around a point z_c instead of origo is similar. Denote by $\text{outer}(M, \dots, p, z_c)$ this expansion.

Similarly, we can do an *inner expansion* which approximates the potential from particles outside a 3D box and is valid in a $D \times D$ box in the center of the 3D box. This is denoted by $\text{inner}(M, \dots, p, z_c)$

Furthermore, we'll need functions to translate the center of expansion for the inner and outer expansions. These functions are defined as

$$\text{outer}(M, \dots, p, z_c') = \text{outer_shift}(\text{outer}(M, \dots, p, z_c), z_c')$$

and

$$\text{inner}(M, \dots, p, z_c') = \text{inner_shift}(\text{inner}(M, \dots, p, z_c), z_c')$$

These functions can be implemented in $O(p^2)$

Finally a conversion function between outer and inner expansions is needed (i.e. when an outer expansion of a box A is valid in the region of another box B, the outer expansion of A can be converted to an inner expansion around B's center). This can also be performed in $O(p^2)$ and is denoted with

$$\text{inner}(M, \dots, p, z_c') = \text{convert}(\text{outer}(M, \dots, p, z_c), z_c')$$

To further compress the notation, I'll use only the cell as identifier for an expansion, e.g. the inner expansion of cell A centered at z_c is written as:

$$\text{inner}(A)$$

The FMM algorithm: Treecodes with a twist

Now that we are done with the math, let's see how all these expansions help us to build a fast and accurate force calculation algorithm. The discussion is based on [Demmel2]

We begin by constructing a quadtree, as in section 0, with the exception that subdivision is not continued until there is 1 particle per cell, but less particles than some limit s . Then the quadtree is traversed from the leaves toward the root, computing outer expansions for each quadtree node. Next, the quadtree is traversed from the root to the leaves, computing inner expansions for the nodes (using the outer expansions provided in the previous step). Now, the force on the particles can be calculated by

adding the inner expansion for the node (accounts for the potential for all particles well separated from the node) and the direct sum of potentials from particles in nearby (non well-separated) nodes. A more detailed description follows:

Step 1: Building the quadtree

For simplicity, we will assume that the tree is fully populated, that is each leaf is at the same distance from the root (this can be achieved by augmenting the tree with empty leaves). A version of FMM using adaptive trees can be found in [Carrier].

Step 2: Computing `outer()` for each node

Recall that we were able to move the center of expansion with the `outer_shift` function. Now we can move the center of expansion for the child cells $B_1..B_4$ to the center of parent cell A (z_a), and then simply add the coefficients c_j of the shifted expansions to obtain an outer expansion for cell A. The expansions of $B_1..B_4$ converge at distances larger than D from z_a , and thus the criterion of convergence for the outer expansion of A is satisfied.

We start from the leaves, by calculating their outer expansions directly, and then proceed towards the root, “merging” outer expansions as described above. Note the similarity to the center of mass calculation step in the Barnes&Hut algorithm.

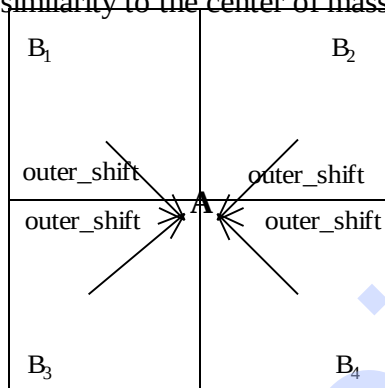


Figure 8 Step 2 of the FMM

Step 3: Computing `inner()` for each node

The idea is that we want to be able to calculate an expansion, which is valid inside a cell A, by using the outer expansions of other cells. Since the outer expansion of a cell B_i only converges (with the desired accuracy) at a distance $1.5D$ from the center of the cell, the immediate neighbour cells of A cannot be used to calculate the inner expansion. More formally, we define the interaction set I of A, in which our particle resides:

$I(A) := \{\text{nodes } B_i \text{ such that } B_i \text{ is a child of a neighbour of parent}(A), \text{ but } B_i \text{ is not itself a neighbour of } A\}$

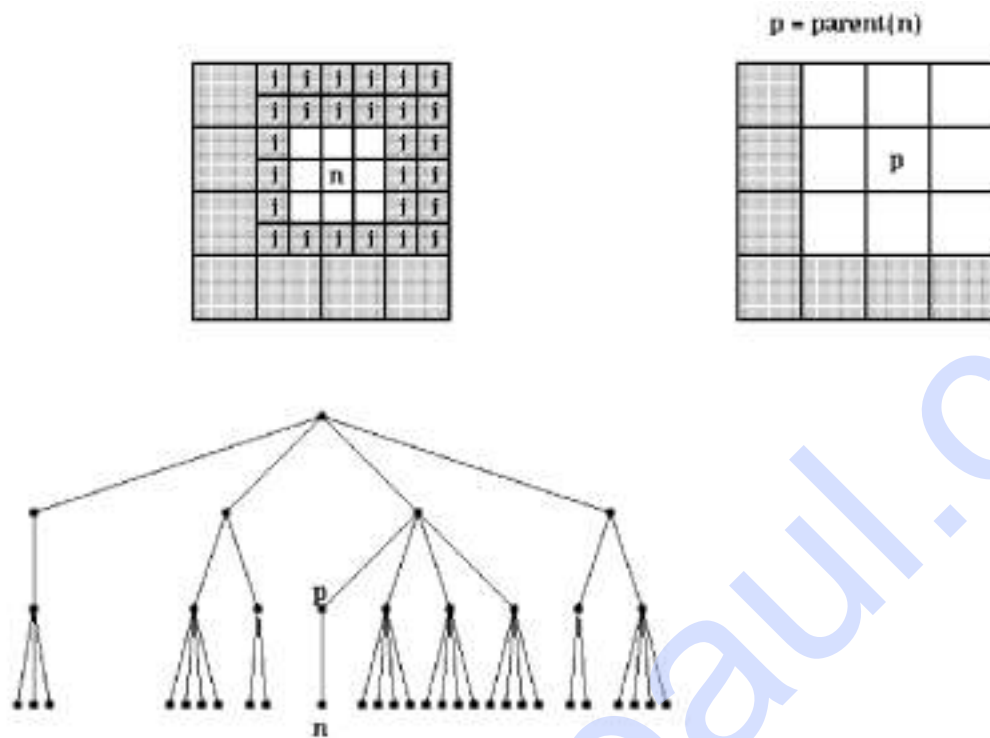


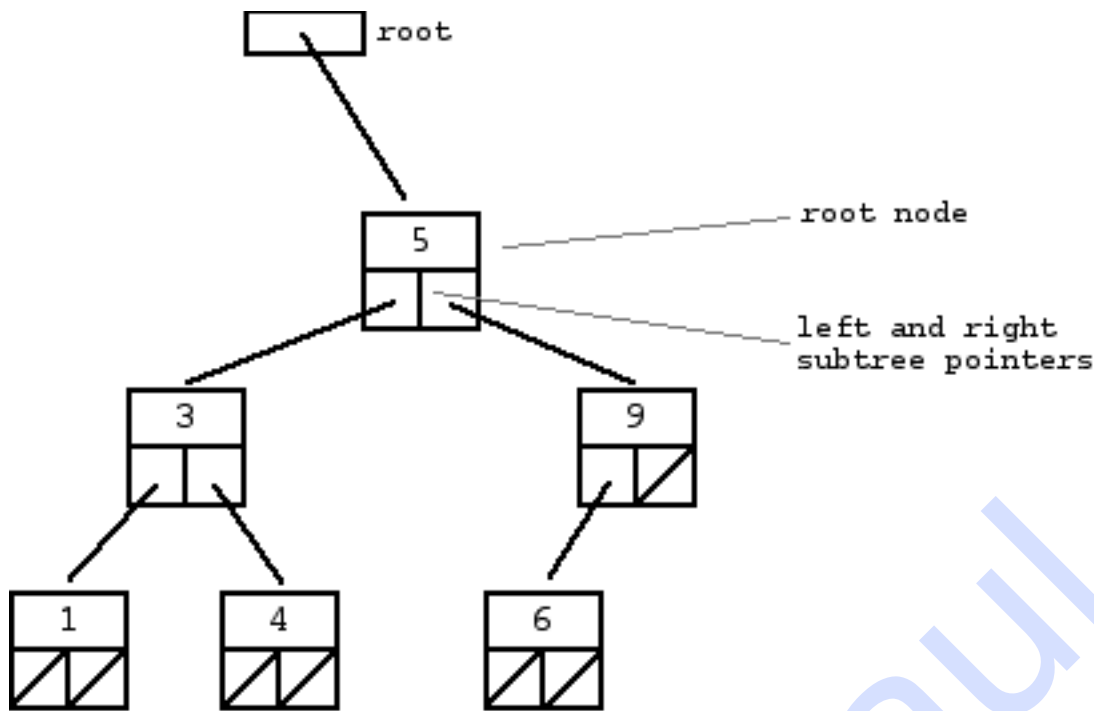
Figure 9 Interaction set for a cell A. From [Demmel2]

The interaction set of A is pictured in Figure 9.

In this stage, we start traversing the tree from the root. At each level, we have an inner expansion which is generated from the interaction set of the previous level (the root does not have any interaction set, and thus it's expansion from "previous levels" is empty). Now, by converting the outer expansions of the interaction set at the current level to inner expansions for A, summing them up, and finally adding the shifted inner expansion of A's parent, we have obtained an inner expansion for A that includes the potential of all cells except the neighbours of A. If A is itself a parent node, we then recurse once more. More formally:

```
Build_inner(A)
  P:=parent(A)
  inner(A):=EMPTY
  for all  $B_i \in I(A)$  do
    inner(A):=inner(A)+convert(outer( $B_i$ ),A)
  endfor
  inner(A):=inner(A)+inner_shift(inner(P),A)
  for all C:=children of A do
    build_inner(C)
  endfor
end
```

Note the difference from Barnes-Hut: This step does not calculate the force on a single particle, rather the potential of an entire leaf cell.



A "binary search tree" (BST) or "ordered binary tree" is a type of binary tree where the nodes are arranged in order: for each node, all elements in its left subtree are less-or-equal to the node ($\leq$), and all the elements in its right subtree are greater than the node ($>$). The tree shown above is a binary search tree -- the "root" node is a 5, and its left subtree nodes (1, 3, 4) are ≤ 5 , and its right subtree nodes (6, 9) are > 5 . Recursively, each of the subtrees must also obey the binary search tree constraint: in the (1, 3, 4) subtree, the 3 is the root, the $1 \leq 3$ and $4 > 3$. Watch out for the exact wording in the problems -- a "binary search tree" is different from a "binary tree".

The nodes at the bottom edge of the tree have empty subtrees and are called "leaf" nodes (1, 4, 6) while the others are "internal" nodes (3, 5, 9).

Binary Search Tree Niche

Basically, binary search trees are fast at insert and lookup. The next section presents the code for these two algorithms. On average, a binary search tree algorithm can locate a node in an N node tree in order $\lg(N)$ time (log base 2). Therefore, binary search trees are good for "dictionary" problems where the code inserts and looks up information indexed by some key. The $\lg(N)$ behavior is the average case -- it's possible for a particular tree to be much slower depending on its shape.

Strategy

Some of the problems in this article use plain binary trees, and some use binary search trees. In any case, the problems concentrate on the combination of pointers and recursion. (See the articles linked above for pointer articles that do not emphasize recursion.)

For each problem, there are two things to understand...

- The node/pointer structure that makes up the tree and the code that manipulates it
- The algorithm, typically recursive, that iterates over the tree

When thinking about a binary tree problem, it's often a good idea to draw a few little trees to think about the various cases.

Typical Binary Tree Code in C/C++

As an introduction, we'll look at the code for the two most basic binary search tree operations -- lookup() and insert(). The code here works for C or C++.

In C or C++, the binary tree is built with a node type like this...

```
struct node {
    int data;
    struct node* left;
    struct node* right;
}
```

Lookup()

Given a binary search tree and a "target" value, search the tree to see if it contains the target. The basic pattern of the lookup() code occurs in many recursive tree algorithms: deal with the base case where the tree is empty, deal with the current node, and then use recursion to deal with the subtrees. If the tree is a binary search tree, there is often some sort of less-than test on the node to decide if the recursion should go left or right.

```
/*
 * Given a binary tree, return true if a node
 * with the target data is found in the tree. Recurs
 * down the tree, chooses the left or right
 * branch by comparing the target to each node.
 */
static int lookup(struct node* node, int target) {
    // 1. Base case == empty tree
    // in that case, the target is not found so return false
    if (node == NULL) {
        return(false);
    }
    else {
        // 2. see if found here
        if (target == node->data) return(true);
        else {
            // 3. otherwise recur down the correct subtree
            if (target < node->data) return(lookup(node->left, target));
            else return(lookup(node->right, target));
        }
    }
}
```

The lookup() algorithm could be written as a while-loop that iterates down the tree. Our version uses recursion to help prepare you for the problems below that require recursion.

Pointer Changing Code

There is a common problem with pointer intensive code: what if a function needs to change one of the pointer parameters passed to it? For example, the insert() function below may want to change the root pointer. In C and C++, one solution uses pointers-to-pointers (aka "reference parameters"). That's a fine technique, but here we will use the simpler technique that a function that wishes to change a pointer passed to it will **return** the new value of the pointer to the caller. The caller is responsible for using the new value. Suppose we have a change() function

that may change the the root, then a call to change() will look like this...

```
// suppose the variable "root" points to the tree
root = change(root);
```

We take the value returned by change(), and use it as the new value for root. This construct is a little awkward, but it avoids using reference parameters which confuse some C and C++ programmers, and Java does not have reference parameters at all. This allows us to focus on the recursion instead of the pointer mechanics.

Insert()

Insert() -- given a binary search tree and a number, insert a new node with the given number into the tree in the correct place. The insert() code is similar to lookup(), but with the complication that it modifies the tree structure. As described above, insert() returns the new tree pointer to use to its caller. Calling insert() with the number 5 on this tree...

```

  2
 / \
1   10
```

returns the tree...

```

  2
 / \
1   10
  /
 5
```

The solution shown here introduces a newNode() helper function that builds a single node. The base-case/recursion structure is similar to the structure in lookup() -- each call checks for the NULL case, looks at the node at hand, and then recurs down the left or right subtree if needed.

```
/*
  Helper function that allocates a new node
  with the given data and NULL left and right
  pointers.
*/
struct node* NewNode(int data) {
    struct node* node = new(struct node);    // "new" is like "malloc"
    node->data = data;
    node->left = NULL;
    node->right = NULL;

    return(node);
}

/*
  Give a binary search tree and a number, inserts a new node
  with the given number in the correct place in the tree.
  Returns the new root pointer which the caller should
  then use (the standard trick to avoid using reference
  parameters).
*/
struct node* insert(struct node* node, int data) {
```

```
// 1. If the tree is empty, return a new, single node
if (node == NULL) {
    return(newNode(data));
}
else {
    // 2. Otherwise, recur down the tree
    if (data <= node->data) node->left = insert(node->left, data);
    else node->right = insert(node->right, data);

    return(node); // return the (unchanged) node pointer
}
}
```

The shape of a binary tree depends very much on the order that the nodes are inserted. In particular, if the nodes are inserted in increasing order (1, 2, 3, 4), the tree nodes just grow to the right leading to a linked list shape where all the left pointers are NULL. A similar thing happens if the nodes are inserted in decreasing order (4, 3, 2, 1). The linked list shape defeats the $\lg(N)$ performance. We will not address that issue here, instead focusing on pointers and recursion.

Binary Tree Problems

Here are 14 binary tree problems in increasing order of difficulty. Some of the problems operate on binary search trees (aka "ordered binary trees") while others work on plain binary trees with no special ordering. The next section, [shows the solution code in C/C++](#), gives the [background](#) and [solution code in Java](#). The basic structure and recursion of the solution code is the same in both languages -- the differences are superficial.

Reading about a data structure is a fine introduction, but at some point the only way to learn is to actually try to solve some problems starting with a blank sheet of paper. To get the most out of these problems, you should at least attempt to solve them before looking at the solution. Even if your solution is not quite right, you will be building up the right skills. With any pointer-based code, it's a good idea to make memory drawings of a few simple cases to see how the algorithm should work.

build123()

This is a very basic problem with a little pointer manipulation. (You can skip this problem if you are already comfortable with pointers.) Write code that builds the following little 1-2-3 binary search tree...

```

      2
     / \
    1   3
```

Write the code in three different ways...

- a: by calling newNode() three times, and using three pointer variables
- b: by calling newNode() three times, and using only one pointer variable
- c: by calling insert() three times passing it the root pointer to build up the tree

(In Java, write a build123() method that operates on the receiver to change it to be the 1-2-3 tree with the given coding constraints.

```
struct node* build123() {
```

```
size()
```


This problem demonstrates simple binary tree traversal. Given a binary tree, count the number of nodes in the tree.

```
int size(struct node* node) {
```

maxDepth()

Given a binary tree, compute its "maxDepth" -- the number of nodes along the longest path from the root node down to the farthest leaf node. The maxDepth of the empty tree is 0, the maxDepth of the tree on the first page is 3.

```
int maxDepth(struct node* node) {
```

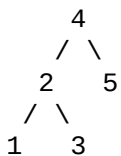
minValue()

Given a non-empty binary search tree (an ordered binary tree), return the minimum data value found in that tree. Note that it is not necessary to search the entire tree. A maxValue() function is structurally very similar to this function. This can be solved with recursion or with a simple while loop.

```
int minValue(struct node* node) {
```

printTree()

Given a binary search tree (aka an "ordered binary tree"), iterate over the nodes to print them out in increasing order. So the tree...



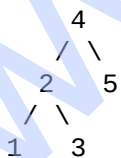
Produces the output "1 2 3 4 5". This is known as an "inorder" traversal of the tree.

Hint: For each node, the strategy is: recur left, print the node data, recur right.

```
void printTree(struct node* node) {
```

printPostorder()

Given a binary tree, print out the nodes of the tree according to a bottom-up "postorder" traversal -- both subtrees of a node are printed out completely before the node itself is printed, and each left subtree is printed before the right subtree. So the tree...

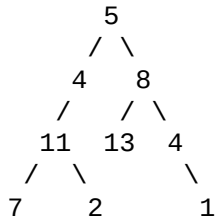


Produces the output "1 3 2 5 4". The description is complex, but the code is simple. This is the sort of bottom-up traversal that would be used, for example, to evaluate an expression tree where a node is an operation like '+' and its subtrees are, recursively, the two subexpressions for the '+'.

```
void printPostorder(struct node* node) {
```

hasPathSum()

We'll define a "root-to-leaf path" to be a sequence of nodes in a tree starting with the root node and proceeding downward to a leaf (a node with no children). We'll say that an empty tree contains no root-to-leaf paths. So for example, the following tree has exactly four root-to-leaf paths:



Root-to-leaf paths:

```

path 1: 5 4 11 7
path 2: 5 4 11 2
path 3: 5 8 13
path 4: 5 8 4 1

```

For this problem, we will be concerned with the sum of the values of such a path -- for example, the sum of the values on the 5-4-11-7 path is $5 + 4 + 11 + 7 = 27$.

Given a binary tree and a sum, return true if the tree has a root-to-leaf path such that adding up all the values along the path equals the given sum. Return false if no such path can be found. (Thanks to Owen Astrachan for suggesting this problem.)

```
int hasPathSum(struct node* node, int sum) {
```

printPaths()

Given a binary tree, print out all of its root-to-leaf paths as defined above. This problem is a little harder than it looks, since the "path so far" needs to be communicated between the recursive calls. **Hint:** In C, C++, and Java, probably the best solution is to create a recursive helper function `printPathsRecur(node, int path[], int pathLen)`, where the path array communicates the sequence of nodes that led up to the current call. Alternately, the problem may be solved bottom-up, with each node returning its list of paths. This strategy works quite nicely in Lisp, since it can exploit the built in list and mapping primitives. (Thanks to Matthias Felleisen for suggesting this problem.)

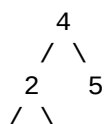
Given a binary tree, print out all of its root-to-leaf paths, one per line.

```
void printPaths(struct node* node) {
```

mirror()

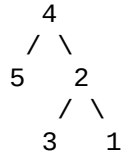
Change a tree so that the roles of the left and right pointers are swapped at every node.

So the tree...



1 3

is changed to...



The solution is short, but very recursive. As it happens, this can be accomplished without changing the root node pointer, so the return-the-new-root construct is not necessary. Alternately, if you do not want to change the tree nodes, you may construct and return a new mirror tree based on the original tree.

```
void mirror(struct node* node) {
```

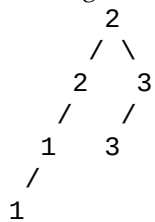
doubleTree()

For each node in a binary search tree, create a new duplicate node, and insert the duplicate as the left child of the original node. The resulting tree should still be a binary search tree.

So the tree...



is changed to...



As with the previous problem, this can be accomplished without changing the root node pointer.

```
void doubleTree(struct node* node) {
```

sameTree()

Given two binary trees, return true if they are structurally identical -- they are made of nodes with the same values arranged in the same way. (Thanks to Julie Zelenski for suggesting this problem.)

```
int sameTree(struct node* a, struct node* b) {
```

countTrees()

This is not a binary tree programming problem in the ordinary sense -- it's more of a math/combinatorics recursion problem that happens to use binary trees. (Thanks to Jerry Cain for suggesting this problem.)

Suppose you are building an N node binary search tree with the values 1..N. How many structurally different binary search trees are there that store those values? Write a recursive function that, given the number of distinct values, computes the number of structurally unique binary search trees that store those values. For example,

countTrees(4) should return 14, since there are 14 structurally unique binary search trees that store 1, 2, 3, and 4. The base case is easy, and the recursion is short but dense. Your code should not construct any actual trees; it's just a counting problem.

```
int countTrees(int numKeys) {
```

Binary Search Tree Checking (for problems 13 and 14)

This background is used by the next two problems: Given a plain binary tree, examine the tree to determine if it meets the requirement to be a binary search tree. To be a binary search tree, for every node, all of the nodes in its left tree must be $\leq$ the node, and all of the nodes in its right subtree must be $>$ the node. Consider the following four examples...

a. 5 -> TRUE
 / \
 2 7

b. 5 -> FALSE, because the 6 is not ok to the left of the 5
 / \
 6 7

c. 5 -> TRUE
 / \
 2 7
 /
 1

d. 5 -> FALSE, the 6 is ok with the 2, but the 6 is not ok with the 5
 / \
 2 7
 / \
 1 6

For the first two cases, the right answer can be seen just by comparing each node to the two nodes immediately below it. However, the fourth case shows how checking the BST quality may depend on nodes which are several layers apart -- the 5 and the 6 in that case.

isBST() -- version 1

Suppose you have helper functions minVal() and maxVal() that return the min or max int value from a non-empty tree (see problem 3 above). Write an isBST() function that returns true if a tree is a binary search tree and false otherwise. Use the helper functions, and don't forget to check every node in the tree. It's ok if your solution is not very efficient. (Thanks to Owen Astrachan for the idea of having this problem, and comparing it to problem 14)

Returns true if a binary tree is a binary search tree.

```
int isBST(struct node* node) {
```

isBST() -- version 2

Version 1 above runs slowly since it traverses over some parts of the tree many times. A better solution looks at each node only once. The trick is to write a utility helper function isBSTRecur(struct node* node, int min, int max) that traverses down the tree keeping track of the narrowing min and max allowed values as it goes, looking at each node only once. The initial values for min and max should be INT_MIN and INT_MAX -- they narrow from there.

```
/*
Returns true if the given tree is a binary search tree
(efficient version).
*/
int isBST2(struct node* node) {
    return(isBSTRecur(node, INT_MIN, INT_MAX));
}

/*
Returns true if the given tree is a BST and its
values are >= min and <= max.
*/
int isBSTRecur(struct node* node, int min, int max) {
```

Tree-List

The Tree-List problem is one of the greatest recursive pointer problems ever devised, and it happens to use binary trees as well. CLibrary works [through the Tree-List problem in detail](#) and includes solution code in C and Java. The problem requires an understanding of binary trees, linked lists, recursion, and pointers. It's a great problem, but it's complex.

C/C++ Solutions

Make an attempt to solve each problem before looking at the solution -- it's the best way to learn.

Build123() Solution (C/C++)

```
// call newNode() three times
struct node* build123a() {
    struct node* root = newNode(2);
    struct node* lchild = newNode(1);
    struct node* rchild = newNode(3);

    root->left = lchild;
    root->right = rchild;

    return(root);
}

// call newNode() three times, and use only one local variable
struct node* build123b() {
    struct node* root = newNode(2);
    root->left = newNode(1);
    root->right = newNode(3);

    return(root);
}
```

```
/*
Build 123 by calling insert() three times.
Note that the '2' must be inserted first.
*/
struct node* build123c() {
    struct node* root = NULL;
    root = insert(root, 2);
    root = insert(root, 1);
    root = insert(root, 3);
    return(root);
}
```

size() Solution (C/C++)

```
/*
Compute the number of nodes in a tree.
*/
int size(struct node* node) {
    if (node==NULL) {
        return(0);
    } else {
        return(size(node->left) + 1 + size(node->right));
    }
}
```

maxDepth() Solution (C/C++)

```
/*
Compute the "maxDepth" of a tree -- the number of nodes along
the longest path from the root node down to the farthest leaf node.
*/
int maxDepth(struct node* node) {
    if (node==NULL) {
        return(0);
    }
    else {
        // compute the depth of each subtree
        int lDepth = maxDepth(node->left);
        int rDepth = maxDepth(node->right);

        // use the larger one
        if (lDepth > rDepth) return(lDepth+1);
        else return(rDepth+1);
    }
}
```

minValue() Solution (C/C++)

```
/*
Given a non-empty binary search tree,
return the minimum data value found in that tree.
```

Note that the entire tree does not need to be searched.

```
*/
int minValue(struct node* node) {
    struct node* current = node;

    // loop down to find the leftmost leaf
    while (current->left != NULL) {
        current = current->left;
    }

    return(current->data);
}
```

printTree() Solution (C/C++)

```
/*
Given a binary search tree, print out
its data elements in increasing
sorted order.
*/
void printTree(struct node* node) {
    if (node == NULL) return;

    printTree(node->left);
    printf("%d ", node->data);
    printTree(node->right);
}
```

printPostorder() Solution (C/C++)

```
/*
Given a binary tree, print its
nodes according to the "bottom-up"
postorder traversal.
*/
void printPostorder(struct node* node) {
    if (node == NULL) return;

    // first recur on both subtrees
    printTree(node->left);
    printTree(node->right);

    // then deal with the node
    printf("%d ", node->data);
}
```

hasPathSum() Solution (C/C++)

```
/*
Given a tree and a sum, return true if there is a path from the root
down to a leaf, such that adding up all the values along the path
equals the given sum.
```

Strategy: subtract the node value from the sum when recurring down, and check to see if the sum is 0 when you run out of tree.

```
*/
int hasPathSum(struct node* node, int sum) {
    // return true if we run out of tree and sum==0
    if (node == NULL) {
        return(sum == 0);
    }
    else {
        // otherwise check both subtrees int
        subSum = sum - node->data;
        return(hasPathSum(node->left, subSum) ||
               hasPathSum(node->right, subSum));
    }
}
```

printPaths() Solution (C/C++)

```
/*
Given a binary tree, print out all of its root-to-leaf
paths, one per line. Uses a recursive helper to do the work.
*/
void printPaths(struct node* node) {
    int path[1000];

    printPathsRecur(node, path, 0);
}

/*
Recursive helper function -- given a node, and an array containing
the path from the root node up to but not including this node,
print out all the root-leaf paths.
*/
void printPathsRecur(struct node* node, int path[], int pathLen) {
    if (node==NULL) return;

    // append this node to the path array
    path[pathLen] = node->data;
    pathLen++;

    // it's a leaf, so print the path that led to here
    if (node->left==NULL && node->right==NULL) {
        printArray(path, pathLen);
    }
    else {
        // otherwise try both subtrees
        printPathsRecur(node->left, path, pathLen);
        printPathsRecur(node->right, path, pathLen);
    }
}

// Utility that prints out an array on a line.
void printArray(int ints[], int len) {
    int i;
    for (i=0; i<len; i++) {
```



```
    printf("%d ", ints[i]);
}
printf("\n");
}
```

mirror() Solution (C/C++)

/*
Change a tree so that the roles of the
left and right pointers are swapped at every node.

So the tree...

```
    4
   / \
  2   5
 / \
1   3
```

is changed to...

```
    4
   / \
  5   2
   / \
  3   1
```

```
*/
void mirror(struct node* node) {
    if (node==NULL) {
        return;
    }
    else {
        struct node* temp;

        // do the subtrees
        mirror(node->left);
        mirror(node->right);

        // swap the pointers in this node
        temp = node->left;
        node->left = node->right;
        node->right = temp;
    }
}
```

doubleTree() Solution (C/C++)

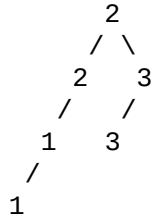
/*
For each node in a binary search tree,
create a new duplicate node, and insert
the duplicate as the left child of the original node.
The resulting tree should still be a binary search tree.

So the tree...

```
    2
   / \
```

1 3

Is changed to...



```

*/
void doubleTree(struct node* node) {
    struct node* oldLeft;

    if (node==NULL) return;

    // do the subtrees
    doubleTree(node->left);
    doubleTree(node->right);

    // duplicate this node to its left
    oldLeft = node->left;
    node->left = newNode(node->data);
    node->left->left = oldLeft;
}

```

sameTree() Solution (C/C++)

```

/*
Given two trees, return true if they are
structurally identical.
*/
int sameTree(struct node* a, struct node* b) {
    // 1. both empty -> true
    if (a==NULL && b==NULL) return(true);

    // 2. both non-empty -> compare them
    else if (a!=NULL && b!=NULL) {
        return(
            a->data == b->data &&
            sameTree(a->left, b->left) &&
            sameTree(a->right, b->right)
        );
    }
    // 3. one empty, one not -> false
    else return(false);
}

```

countTrees() Solution (C/C++)

```

/*
For the key values 1...numKeys, how many structurally unique

```

binary search trees are possible that store those keys.

Strategy: consider that each value could be the root.
Recursively find the size of the left and right subtrees.

```
*/
int countTrees(int numKeys) {

    if (numKeys <=1) {
        return(1);
    }
    else {
        // there will be one value at the root, with whatever remains
        // on the left and right each forming their own subtrees.
        // Iterate through all the values that could be the root...
        int sum = 0;
        int left, right, root;

        for (root=1; root<=numKeys; root++) {
            left = countTrees(root - 1);
            right = countTrees(numKeys - root);

            // number of possible trees with this root == left*right
            sum += left*right;
        }

        return(sum);
    }
}
```

isBST1() Solution (C/C++)

```
/*
Returns true if a binary tree is a binary search tree.
*/
int isBST(struct node* node) {
    if (node==NULL) return(true);

    // false if the min of the left is > than us
    if (node->left!=NULL && minValue(node->left) > node->data)
        return(false);

    // false if the max of the right is <= than us
    if (node->right!=NULL && maxValue(node->right) <= node->data)
        return(false);

    // false if, recursively, the left or right is not a BST
    if (!isBST(node->left) || !isBST(node->right))
        return(false);

    // passing all that, it's a BST
    return(true);
}
```

Message Passing Interface

The Message Passing Interface, MPI is a controlled API standard for programming a wide array of parallel architectures. Though MPI was originally intended for classic distributed memory architectures, it is used on various architectures from networks of PCs via large shared memory systems, such as the SGI Origin 2000, to massive parallel architectures, such as Cray T3D and Intel paragon. The complete MPI API offers 186 operations, which makes this is a rather complex programming API. However, most MPI applications use only six to ten of the available operations.

MPI is intended for the *Single Program Multiple Data* (SPMD) programming paradigm – all nodes run the same application-code. The SPMD paradigm is efficient and easy to use for a large set of scientific applications with a regular execution pattern. Other, less regular, applications are far less suited to this paradigm and implementation in MPI is tedious.

MPI's point-to-point communication comes in four shapes: standard, ready, synchronous and buffered. A *standard-send* operation does not return until the send buffer has been copied, either to another buffer below the MPI layer or to the network interface, (NIC). The *ready-send* operations are not initiated until the addressed process has initiated a corresponding receive-operation. The *synchronous* call sends the message, but does not return until the receiver has initiated a read of the message. The fourth model, the *buffered* send, copies the message to a buffer in the MPI-layer and then allows the application to continue. Each of the four models also comes in *asynchronous* (in MPI called *non-blocking*) modes. The non-blocking calls return immediately, and it is the programmer's responsibility to check that the send has completed before overwriting the buffer. Likewise a non-blocking receive exist, which returns immediately and the programmer needs to ensure that the receive operation has finished before using the data.

MPI supports both group broadcasting and global reductions. Being SPMD, all nodes have to meet at a group operation, i.e. a broadcast operation blocks until all the processes in the context have issued the broadcast operation. This is important because it turns all group-operations into synchronization points in the application. The MPI API also supports scatter-gather for easy exchange of large data-structures and virtual architecture topologies, which allow source-code compatible MPI applications to execute efficiently across different platforms.

Experiment Environment

Cluster

The cluster comprises 51 Dell Precision Workstation 360s, each with a 3.2GHz Intel Prescott processor, 2GB RAM and a 120GB Serial ATA hard-disk². The nodes are connected using Gigabit Ethernet over two HP Procurve 2848 switches. 32 nodes are connected to the first switch, and 19 nodes to the second switch. The two switches are trunked³ with 4 copper cables, providing 4Gbit/s bandwidth between the switches, see Figure 1. The nodes are running RedHat Linux 9 with a patched Linux 2.4.26 kernel to support Serial ATA. Hyperthreading is switched on, and Linux is configured for Symmetric Multiprocessor support.

² The computers have a motherboard with Intel's 875P chipset. The chipset supports Gigabit Ethernet over Intel's CSA (Communication Streaming Architecture) bus, but Dell's implementation of the motherboards use an Intel 82540EM Gigabit Ethernet controller connected to the PCI bus instead.

³ Trunking is a method where traffic between two switches is loadbalanced across a set of links in order to provide a higher available bandwidth between the switches.

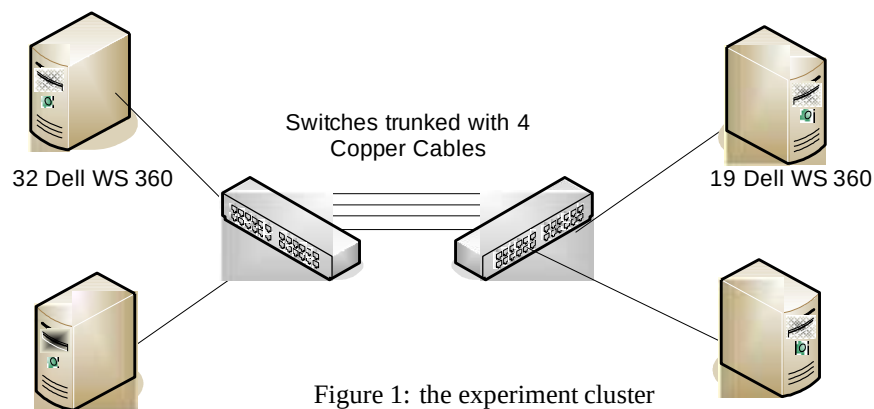


Figure 1: the experiment cluster

MPICH

MPICH is the official reference implementation of MPI and has a high focus on being portable. MPICH is available for all UNIX flavors and for Windows, a special GRID enabled version, MPICH-G2, is available for Globus[11]. Many of the MPI implementations for specialized hardware, i.e. cluster interconnects, are based on MPICH. MPICH version 1.2.52 is used for the below experiments.

LAM-MPI

Local Area Multicomputer-MPI, LAM-MPI, started out as an implementation for running MPI applications on LANs. An integrated part of this model was 'on-the-fly' endian-conversion to support different architectures to collaborate on an MPI execution. While endian-conversion still is supported, it is no longer performed per default as it is assumed that most executions will be on homogenous clusters. The experiments in this paper are performed with LAM-MPI 7.0.5.

MESH-MPI

MESH-MPI is only just released and the presented results are thus brand-new. MESH-MPI is 'yet-another-commercial-MPI', but with a strong focus on performance, rather than simply improved support over the open-source versions. In addition to improved performance, MESH-MPI also promotes true non-blocking operations, thread safety, and scalable collective operations. Future versions have announced support for a special Low Latency Communication library (LLC) and a Runtime Data Dependency Analysis (RDDA) functionality to schedule communication. These functions are not available in the current version which is 1.0a.

Benchmarks

This section describes the benchmark suites we have chosen for examining the performance of the three MPI implementations. One suite, Pallas, is a micro-benchmark suite, which gives a lot of information about the performance of the different MPI functions, while the other, NPB, is an application/kernel suite, which describes the application level performance. The NPB suite originates from NASA and is used as the basis for deciding on new systems at NASA. This benchmark tests both the processing power of the system and the communication performance.

Pallas Benchmark Suite

The Pallas benchmark suite[9] from Pallas GmbH is a suite, which measures the performance of different MPI functions. The performance is measured for individual operations rather than on the application level. The results can thus be used in two ways; either to choose an MPI implementation that performs well for the operations one uses, or to determine which operations performs poorly on the available MPI implementation so that one can avoid them when coding applications. The tests/operations that are run in Pallas are:

- PingPong
The time it takes to pass a message between two processes and back
- PingPing
The time it takes to send a message from one process to another
- SendRecv
The time it takes to send and receive a message in parallel
- Exchange
The time it takes to exchange contents of two buffers
- Allreduce
The time it takes to create a common result, i.e. a global sum
- Reduce
The same as Allreduce but the result is delivered to only one process
- Reduce Scatter
The same as Reduce but the result is distributed amongst the processes
- Allgather
The time it takes to collect partial results from all processes and deliver the data to all processes
- Allgatherv
Same as Allgather, except that the partial results need not have the same size
- Alltoall
The time it takes for all processes to send data to all other processes and receive from all other processes – the data that is sent is unique to each receiver
- Bcast - *the time it takes to deliver a message to all processes*

NAS Parallel Benchmark Suite

The NAS Parallel Benchmark, NPB, suite is described as:

The NAS Parallel Benchmarks (NPB) are a set of 8 programs designed to help evaluate the performance of parallel supercomputers. The benchmarks, which are derived from computational fluid dynamics (CFD) applications, consist of five kernels and three pseudo-applications. The NPB come in several flavors. NAS solicits performance results for each from all sources.
[10]

NPB is available for threaded, OpenMP and MPI systems and we naturally run the MPI version. NPB is available with five different data-sets, A through D, and W which is for workstations only. We use dataset C since D won't fit on the cluster, and also since C is the most widely reported dataset.

The application kernels in NPB are:

- MG – *Multigrid*
- CG – *Conjugate Gradient*
- FT – *Fast Fourier Transform*
- IS – *Integer Sort*
- EP – *Embarrassingly Parallel*
- BT – *Block Tridiagonal*
- SP – *Scalar Pentadiagonal*
- LU – *Lower Upper Gauss-Seidel*

Results

In this section we present and analyze the results of running the benchmarks from section 3 on the systems described in section 2. All the Pallas benchmarks are run on 32 CPUs (they run on 2^x sized systems) as are the NPB benchmarks except BT and SP which are run on 36 CPUs (they run on X^2 sized systems).

Pallas Benchmark Suite

First in the Pallas benchmark is the point to point experiments, the extreme case is the concurrent Send and Recv experiments where MPICH uses more than 12 times longer than MESH-MPI, but otherwise all three are fairly close. MPICH performs worse than the other two and the commercial MESH-MPI loses only on the ping-ping experiment.

The seemingly large differences on ping-pong and ping-ping are not as significant as they may seem since they are the result of the interrupt throttling rate on the Intel Ethernet chipsets which – when set at the recommended 8000, discretises latencies in chunks of 125us, thus the difference between 62.5 us and 125us is not as significant as it may seem and would probably be much smaller on other Ethernet chipsets.

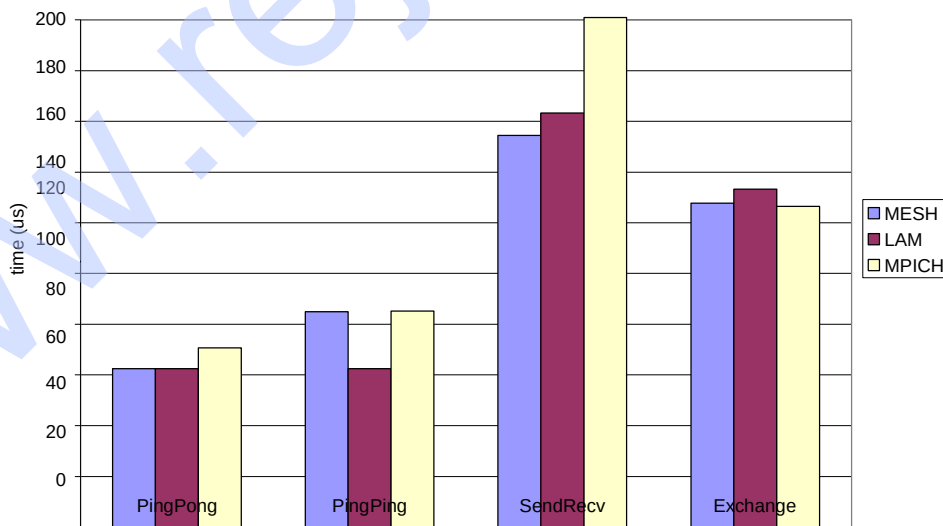


Figure 2: point-to-point latencies from Pallas 0B messages.

Switching to large messages, 4MB, the picture is more uniform and MPICH consistently looses to the other two. LAM-MPI and MESH-MPI are quite close in all these experiments and are running within 2% of each other. The only significant exception is in the ping-ping experiment where LAM-MPI outperforms MESH-MPI with 5%.

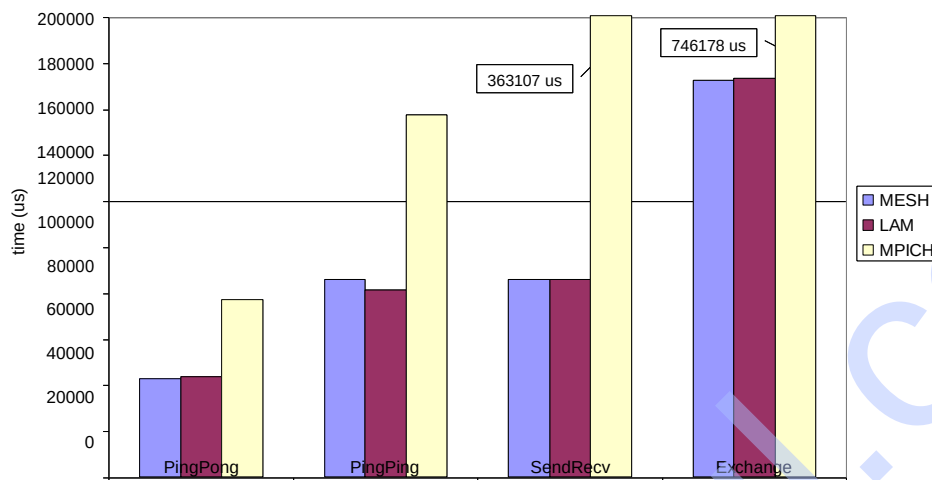


Figure 3: point-to-point latencies from Pallas 4MB messages.

In the collective operations, the small data is tested on 8B (eight bytes) rather than 0B because 0B on group-operations are often not performed at all and resulting times are reported in the 0.05us range, thus to test the performance on small packages we use the size of a double precision number. The results are shown in Figure 4.

In the collective operations, the extreme case is Allgatherv using LAM-MPI which reports a whopping 4747us or 11 times longer than when using MESH-MPI. Except for the Alltoall benchmark where LAM-MPI is fastest, MESH-MPI is consistently the faster, and for most experiments, the advantage is significant, measured in multiples rather than percentages. The Bcast operation, which is a frequently used operation in many applications, shows MESH-MPI to be 7 times faster than MPICH and 12 times faster than LAM-MPI.

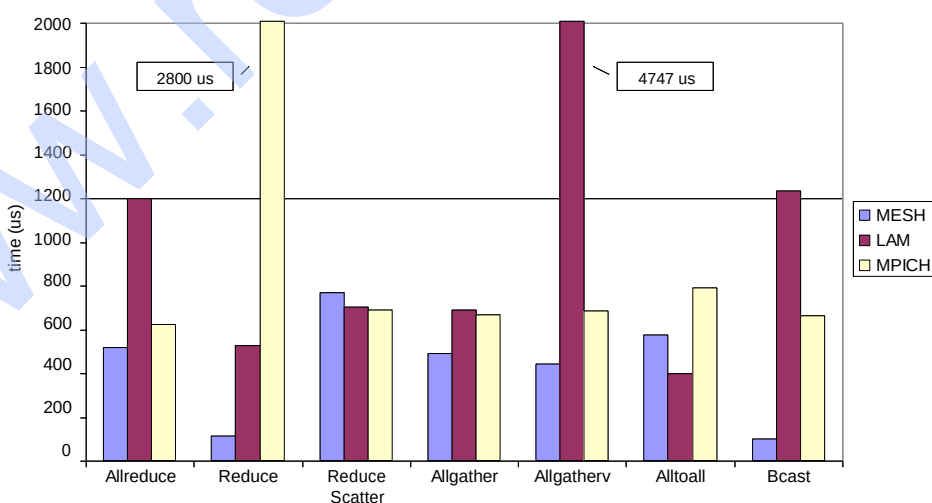


Figure 4: collective Operations latencies from Pallas 8B messages.

For large messages, the results have been placed in two Figures, 5 and 6, in order to fit the time-scale better. With the large messages, MESH-MPI is consistently better than both open-source candidates, ranging from nothing, -1%, to a lot: 11 times. On average MESH-MPI outperforms LAM-MPI with 4.6 times and MPICH with 4.3 times. MESH-MPI is on average 3.5 times faster than the best of the two open-source implementations.

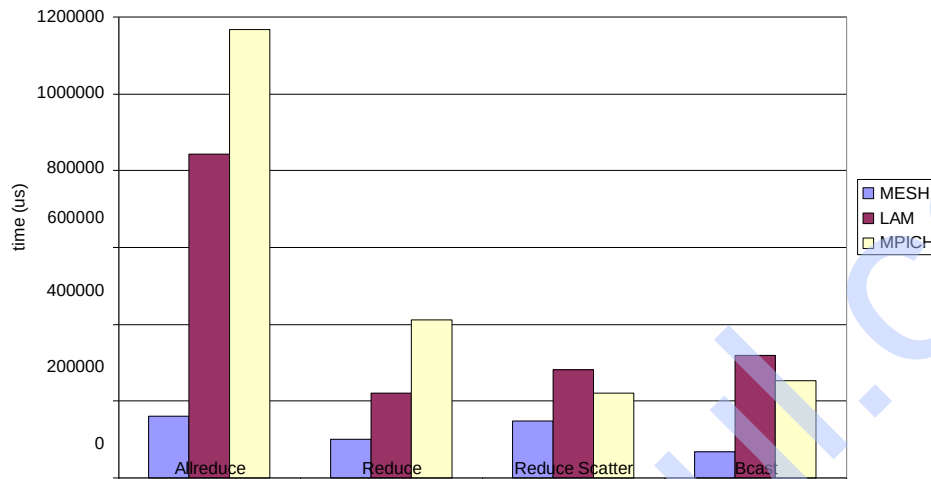


Figure 5: collective Operations latencies from Pallas 4MB messages.

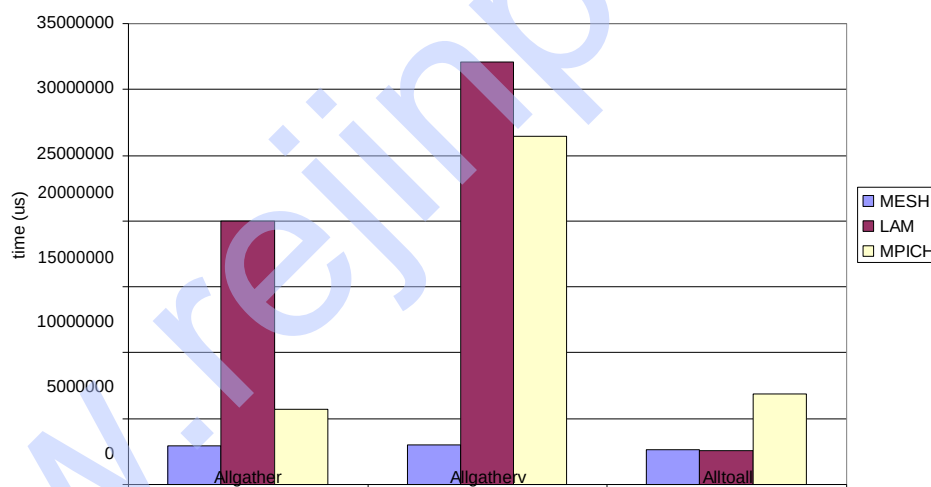


Figure 6: collective Operations latencies from Pallas 4MB messages.

NPB Benchmark Suite

While micro-benchmarks are interesting from an MPI perspective, users are primarily interested in the performance at application level. Here, according to Amdahl's law, improvements are limited by the fraction of time spent on MPI operations. Thus the runtime of the NPB suite is particularly interesting, since it allows us to predict the value of running a commercial MPI, and it will even allow us to determine if the differences at the operation level performance can be seen at the application level.

The results are in favour of the commercial MPI; MESH-MPI finished the suite 14.5% faster than LAM and 37.1% faster than MPICH. Considering that these are real-world applications doing real work and taking Amdahl's law into consideration, this is significant.

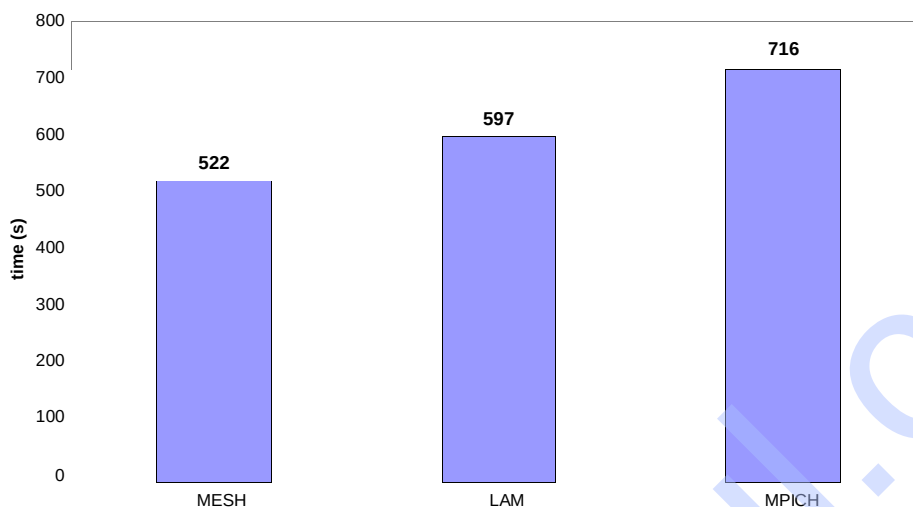


Figure 7: runtime of the NPB benchmark

If we break down the results in the individual applications the picture is a little less obvious and LAM-MPI actually outperforms MESH-MPI on two of the experiments; the FT by 3% and the LU by 6%. Both of these makes extensive use of the Alltoall operation where MESH-MPI has the biggest problems keeping up with LAM-MPI in the Pallas tests.

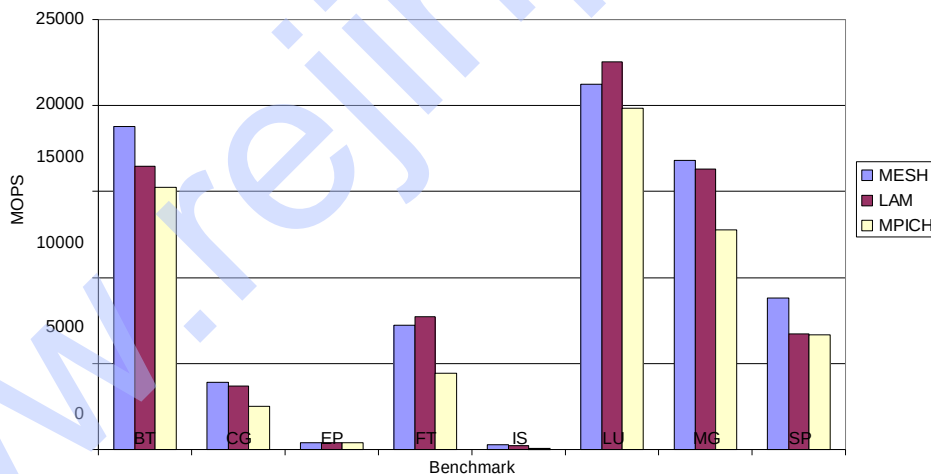


Figure 8: runtime of the individual NPB benchmarks

OpenMP and MPI implementations and comparison

MPI	OpenMP
1 . Available from different vendor and can be compiled in desired platform with desired compiler. One can use any of MPI API i.e MPICH, OpenMPI or other	1 .OpenMP are hooked with compiler so with gnu compiler and with Intel compiler one have specific implementation. User is at liberty with changing compiler but not with openmp implementation.
2. MPI support C,C++ and FORTRAN	2.OpenMP support C,C++ and FORTRAN
3.OpenMPI one of API for MPI is providing provisional support for Java	3.Few projects try to replicate openmp for Java.
4. MPI target both distributed as well shared memory system	4.OpenMP target only shared memory system
5.Based on both process and thread based approach .(Earlier it was mainly process based parallelism but now with MPI 2 and 3 thread based parallelism is there too. Usually a process can contain more than 1 thread and call MPI subroutine as desired	5.Only thread based parallelism.
6. Overhead for creating process is one time	6. Depending on implementation threads can be created and joined for particular task which add overhead
7.There are overheads associated with transferring message from one process to another	7.No such overheads, as thread can share variables
8. Process in MPI has private variable only, no shared variable	8. In OpenMP , threads have both private as well shared variable

9.Data racing is not there if not using any thread in process .

9. Data racing is inherent in OpenMP model

10.Compilation of MPI program require

1. Adding header file : #include "mpi.h"
2. compiler as:(in linux)
mpic++ mpi.cxx -o mpiExe

10. Need to add omp.h and then can directly compile code with -fopenmp in Linux environment

g++ -fopenmp openmp.cxx -o openmpExe

(User need to set environment variable PATH and LD_LIBRARY_PATH to MPI as OpenMPI installed folder or binaries) (For Linux)

11 . Running MPI program .

- a) User need to make sure that bin and library folder from MPI installation are included in environmental variable PATH and LD_LIBRARY_PATH.
- b) For running executable from command line ,user need to supply following command and specify number of processor as in example below it is four .

11.User can launch executable OpenMPExe in normal way.

mpirun -np 4 mpiExe

./openmpExe

Sample MPI program

```
#include <iostream>
#include <mpi.h>
```

```
/******
```

This is a simple hello world program. Each processor print its id

```
*****/
```

```
using namespace std;
int main(int argc,char** argv)
{
```

```
    int myid, numprocs;
```

```
    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD,&numprocs);
    MPI_Comm_rank(MPI_COMM_WORLD,&myid);
```

```

    /* output my rank */
    cout<<"Hello from "<<myid<<endl;
    MPI_Finalize();

}

```

Command to run executable with name a.out in Linux = **mpirun -np 4 a.out**

Output

```

Hello from 1
Hello from 0
Hello from 2
Hello from 3

```

Sample OpenMP Program

```

#include<iostream>
#include<omp.h>
using namespace std;
/*****
Sample OpenMP program which at stage 1 has 4 threads and at stage 2 has 2 threads
*****/
int main()
{
#pragma omp parallel num_threads(4) /*create 4 threads and region inside it will be
executed by all threads . */
{
#pragma omp critical/*allow one thread at a time to access below statement
cout<<" Thread Id in OpenMP stage 1= "<<omp_get_thread_num()<< endl;
} //here all thread get merged into one thread id

cout<<"I am alone"<<endl;

#pragma omp parallel num_threads(2)/*create two threads
{
cout<<" Thread Id in OpenMP stage 2= "<<omp_get_thread_num()<< endl;;
}
}

```

Command to run executable with name a.out on Linux : /a.out

Output

```

Thread Id in OpenMP stage 1= 2
Thread Id in OpenMP stage 1=0
Thread Id in OpenMP stage 1=3
Thread Id in OpenMP stage 1= 1
I am alone
Thread Id in OpenMP stage 2= 1
Thread Id in OpenMP stage 2=0

```

Summary

MPI and OpenMP have its own advantages and limitations . OpenMP is relatively easy to implement and involves few pragma directives to achieve desired tasks. OpenMP can be used in recursive function as well i.e as traversing in binary tree. However it suffers from problem of memory limitations for memory intensive calculations.

MPI usually serve those problem well which involve large memory. With MPI 3 , shared memory advantage can be utilized within MPI too. Also one can use OpenMP with MPI i.e for shared memory in targeted platform OpenMP can be used whereas for distributed one, MPI can be used.